



Oprogramowanie systemowe (System software)

Tomasz Dziubich
p. 524 EA, tomasz.dziubich@pg.edu.pl



1. Zakres tematyczny przedmiotu
2. Literatura
3. Warunki zaliczenia przedmiotu
4. Strona przedmiotu



**POLITECHNIKA
GDAŃSKA**

**Tryb chroniony
(Protected mode)**

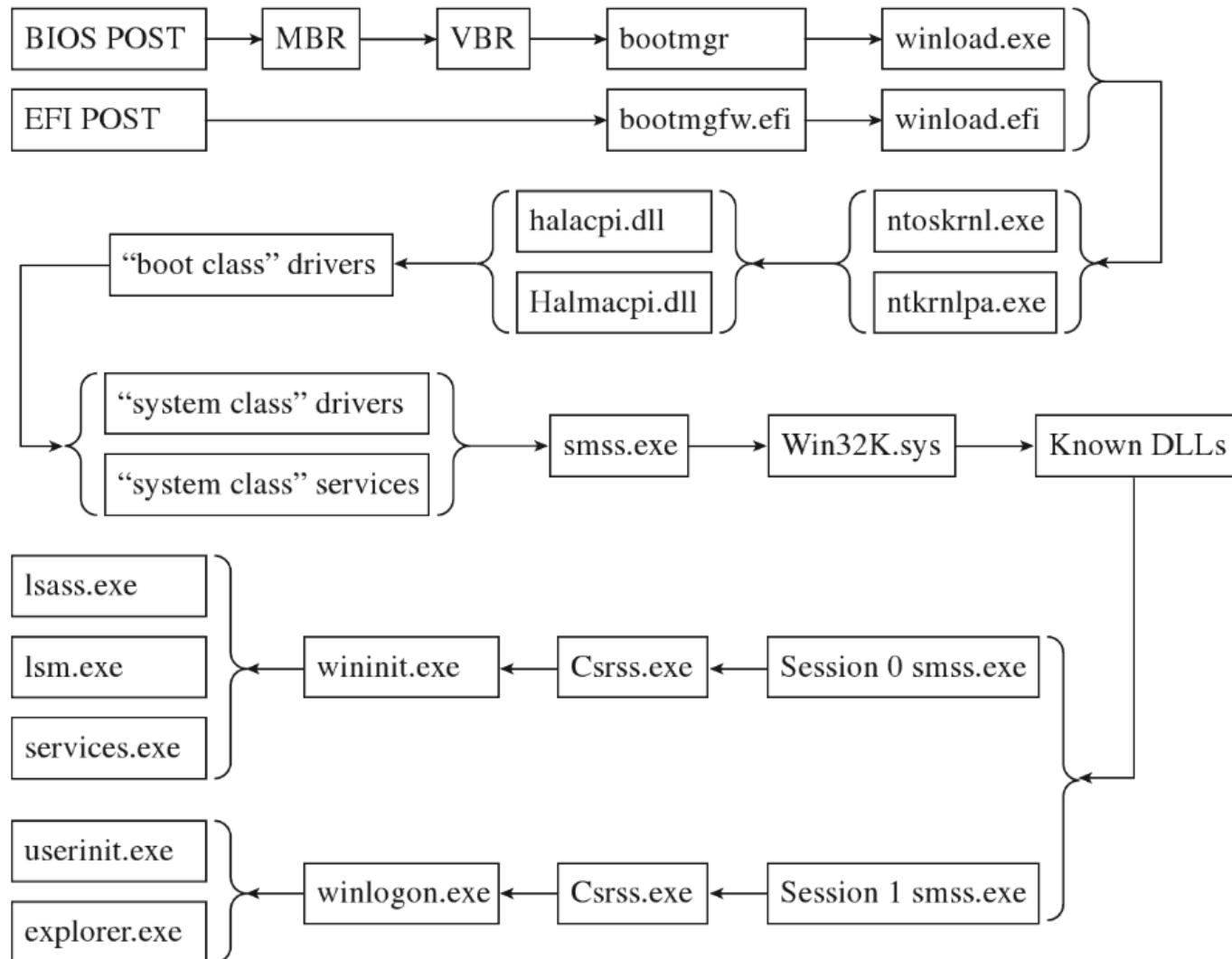
Tomasz Dziubich



1. Segmentowy i płaski model pamięci
2. Tryb rzeczywisty i chroniony
3. Adresacja w trybie chronionym z segmentacją i stronicowaniem
4. Separacja przestrzeni adresowych procesów
5. Obsługa wyjątków i przerwań
6. Poziomy ochrony i bramy funkcji systemowych



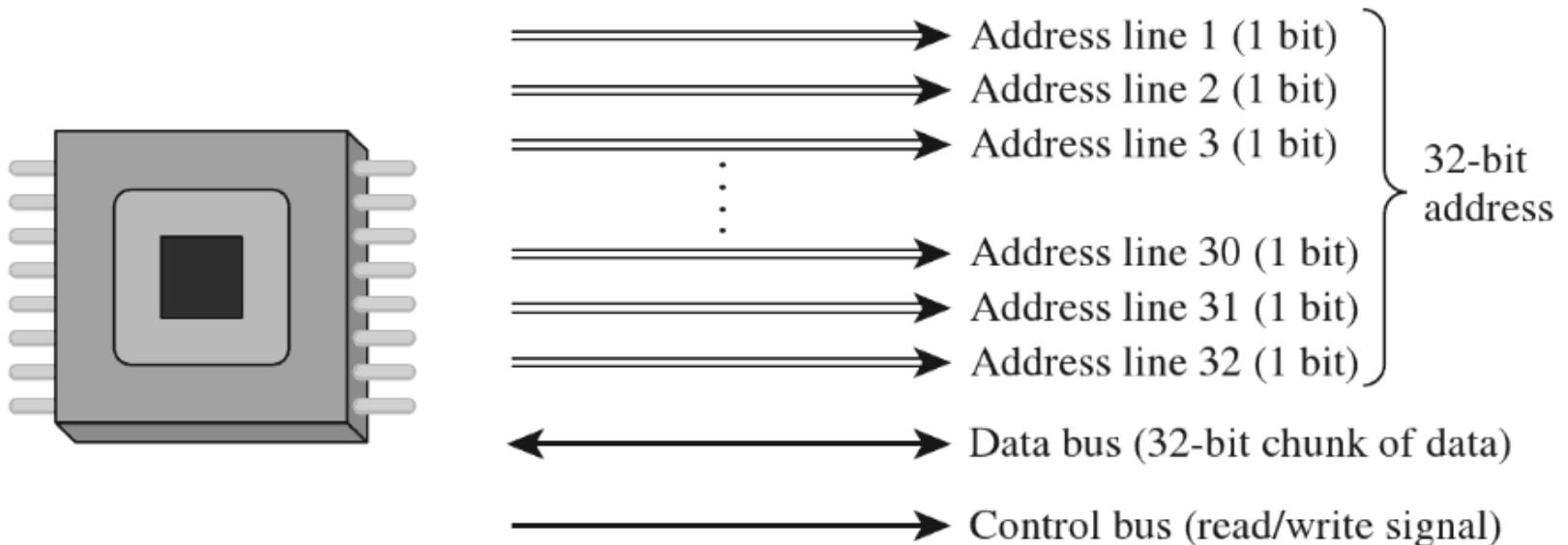
Proces uruchamiania SO na przykładzie Windows (Boot Process)





Od czego więc zacząć?

- Poznanie sprzętu, na którym działa SO:
 - jaka jest architektura procesora?
 - jakie są obszary, w których procesor wspiera twórcę SO?





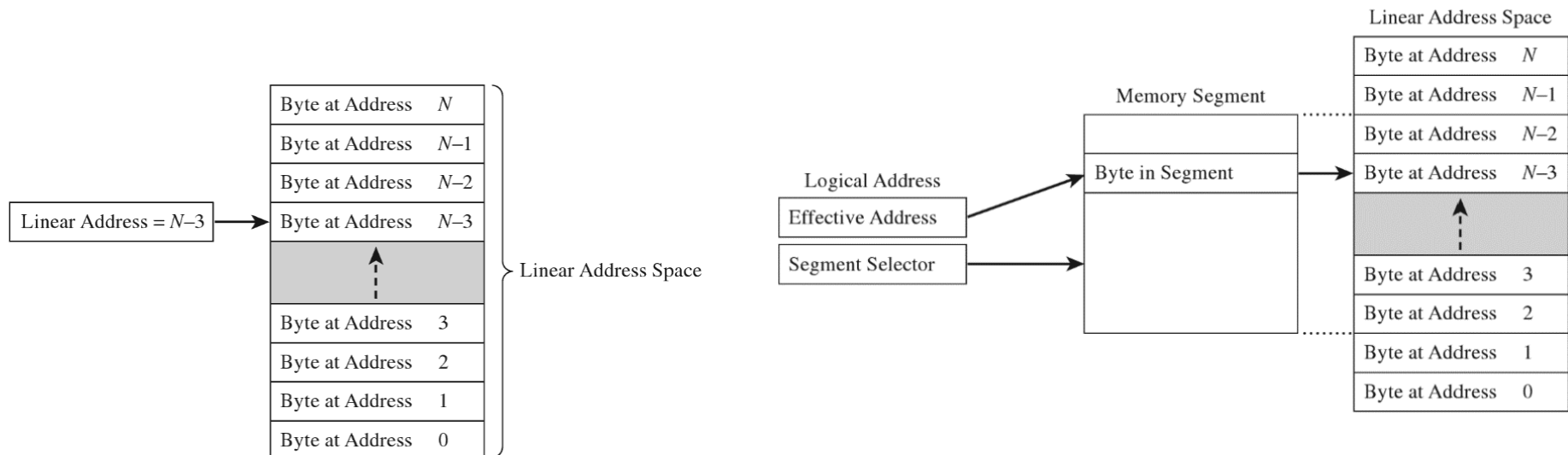
Pamięć RAM a magistrala adresowa

- Rozpatrzmy n -bitową (np. 32 bitowa) magistralę adresową i pamięć RAM o pojemności:
 - $= 2^n$ GB (4 GB) - dokładne dopasowanie
 - $< 2^n$ GB (2 GB) - ignorujemy najstarsze bity, ustawiając je np. na 0
 - $> 2^n$ GB (8 GB) - ?? zmiana sposobu wyznaczania adresu
- Co z rozmiarem rejestrów adresowych (niedopasowanie rozmiaru) ?
- Problem wielozadaniowości
- Istnieją dwa modele pamięci fizyczny (rzeczywisty, sprzętowy) i liniowy (abstrakcyjny, widziany powyżej warstwy sprzętowej, przez programistę).
 - PAE (Physical Address Extension) – adresacja 36 bitowa (do 52 bit)
 - Sprawdzenie MAXPHYADDR zwrócony przez rozkaz [CPUID](#) (funkcja 80000008h)



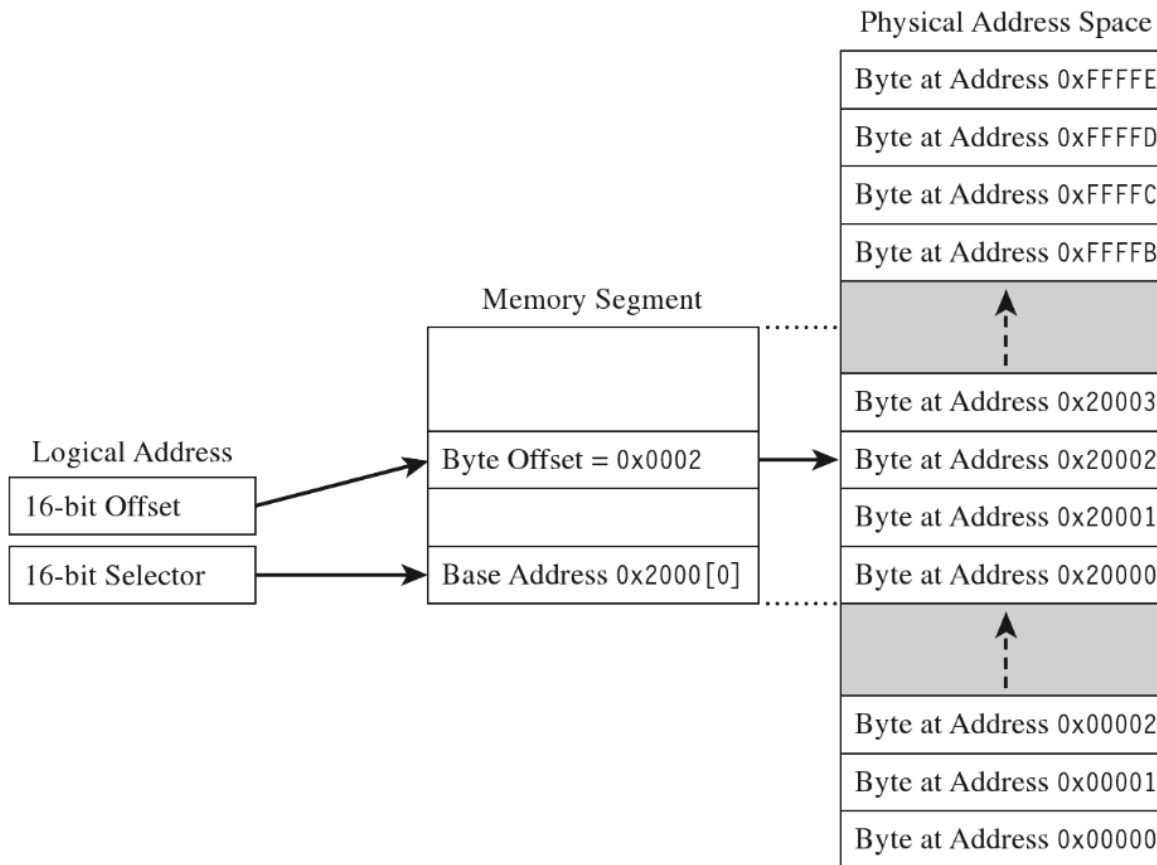
Segmentowy i płaski model pamięci

- Adres fizyczny – adres wskazujący na położenie lokacji pamięci (komórki) w przestrzeni pamięci fizycznej kości pamięci RAM (adres musi wskazywać na istniejącą komórkę)
- Adres liniowy – adres w procesie wskazujący na położenie lokacji pamięci w jego przestrzeni adresów (wirtualnych, model programowy)
- Adres logiczny – reprezentacja adresu liniowego przy pomocy pewnego zestawu identyfikatorów dostępnych w procesorze (np. selektora i offsetu)





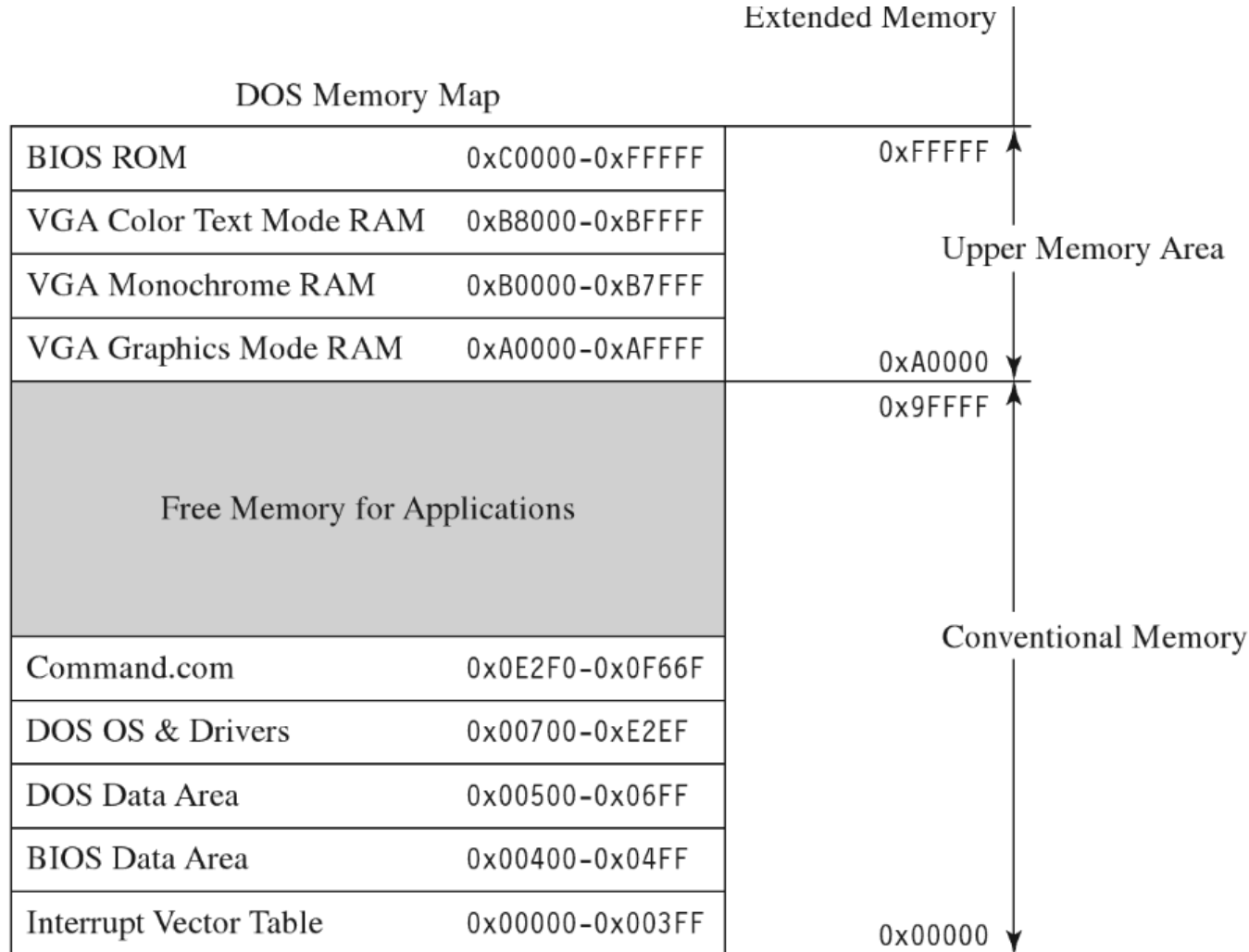
Tryby pracy procesora x86



- Tryb rzeczywisty (real mode) – używa segmentacji
- Tryb chroniony (protected mode) – wymagana segmentacja, umożliwia stronicowanie



DOS (Memory Map/Layout)



Tryb rzeczywisty
jest podstawą
przejęcia w tryb
chroniony dla
systemów
wyposażonych w
oprogramowanie
układowe BIOS



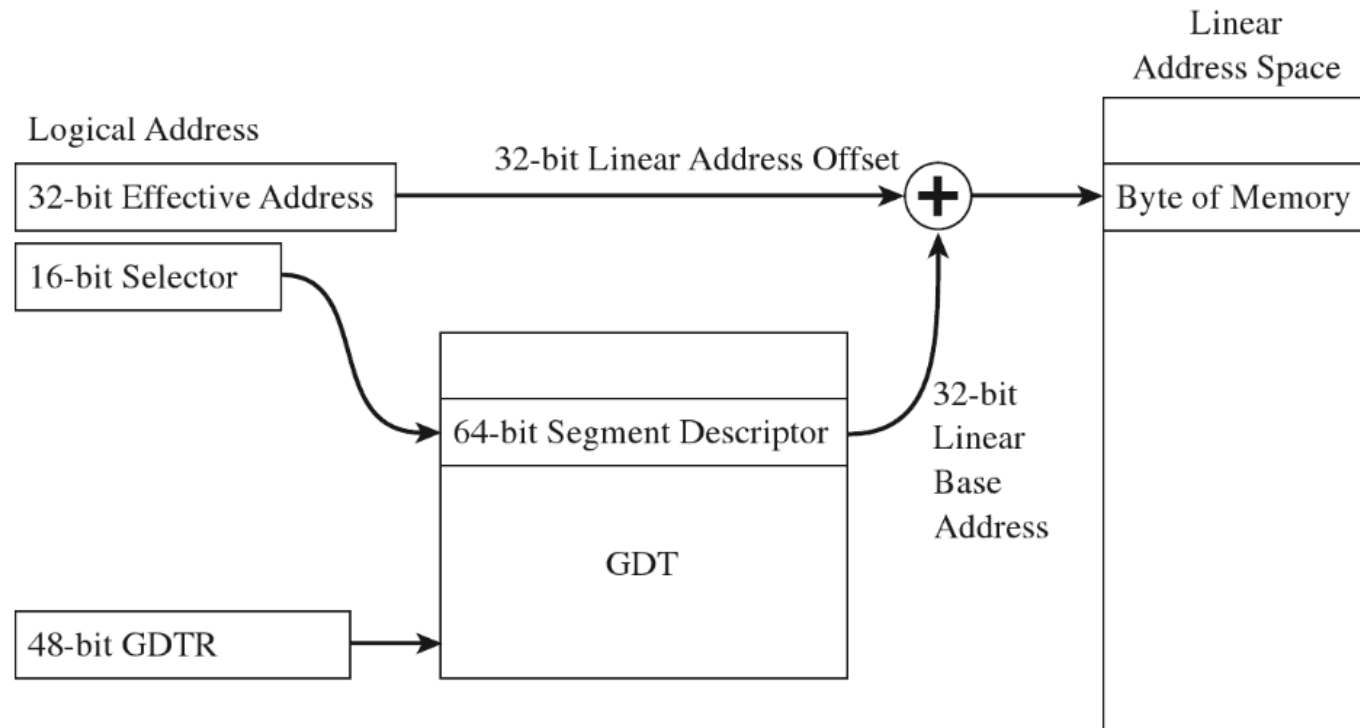
Wady trybu rzeczywistego

- Modyfikowanie adresów funkcji systemowych (hooking)
- Zdolność istniejących sterowników do przejęcia dowolnej informacji
- Manipulowanie strukturami danych systemowych do ukrycia procesu
- Modyfikacja programów wykonywalnych w celu zmiany przepływu sterowania



Tryby chroniony z segmentacją (bez stronicowania)

- Adresacja przestrzeni identyczna dla wszystkich procesów (np. od 0)!

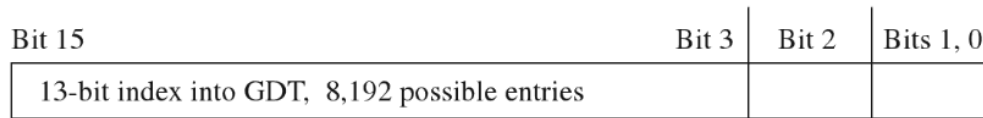


- Segmentacja jest obowiązkowa w x86!



Selektor i deskryptor

16-bit Segment Selector



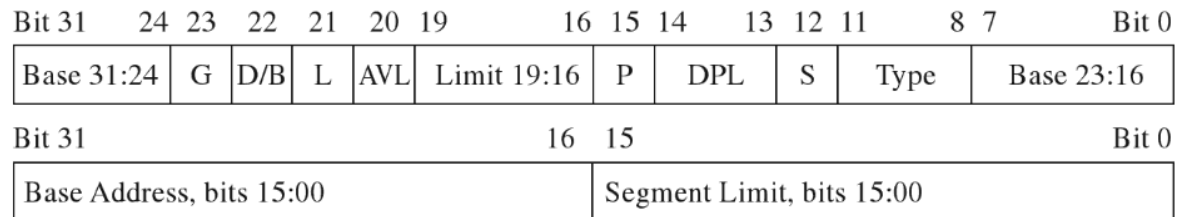
1 = specifies a descriptor in an LDT

0 = specifies a descriptor in a GDT

Requested privilege level (RPL)

(00 = most privilege, 11 = least privilege)

64-bit Segment Descriptor



Segment Limit (20-bits)

Base Address (32-bits)

Type Field

S Flag

DPL

P Flag

AVL

L Flag

D/B

G Flag

Segment size (if G = 0: 1 byte – 1 MB, if G = 1: 4 KB to 4 GB)

Base linear address used to form the final linear address

Type of segment (code or data), access, and growth direction

If S is clear, system segment. If S is set, application segment

Descriptor privilege level (00 = Ring 0, 11 = Ring 3)

If P is set, segment is resident in memory

No explicit purpose, available for use by operating system

IA-32 processors set this bit to zero (indicates 64-bit code)

Meaning varies according to segment type (code, data, or stack)

See description of segment limit field



Rejestry sterujące

CR4

Bit 31	18	17		14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved (i.e., zero)	OSX	PCI		SMX	VMX			OSX	OSF	PCE	PGE	MCE	PAE	PSE	DE	TSD	PVI	VM

CR3

Bit 31	12	11	5	4	3	2	0
Page Directory Base Address				PCD	PWT		

CR2

Bit 31	0
Linear Address that has caused a page fault	

CR1

Bit 31	0
Reserved (i.e., not used, by Intel)	

CR0

31	30	29			18		16					5	4	3	2	1	0
PG	CD	NW			AM		WP					NE	ET	TS	EM	MP	PE

Flags of Particular Interest

Page Directory Base Address in CR3

WP (Write Protect Bit) in CRO—When set, prevents writing into read-only user-level pages

Other Flags of Note

CR0: PG flag—enables paging when set

CR0: PE flag—enables protected mode when set

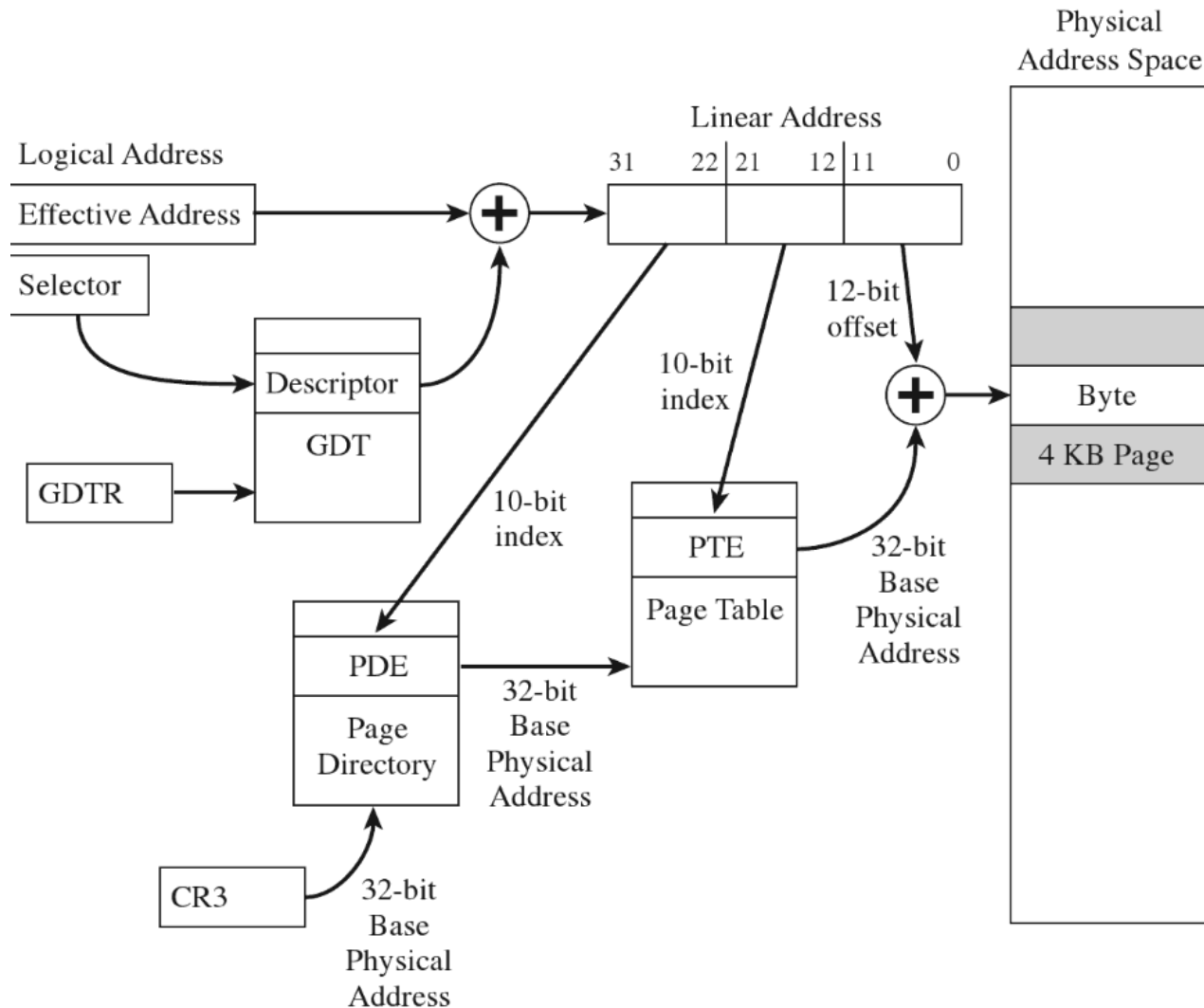
(set by OS when it makes the jump from real mode)

CR4: PSE flag—enables larger page sizes when set (e.g., 2 or 4 MB)

CR4: PAE flag—when set it allows a 36-bit physical address space to be used



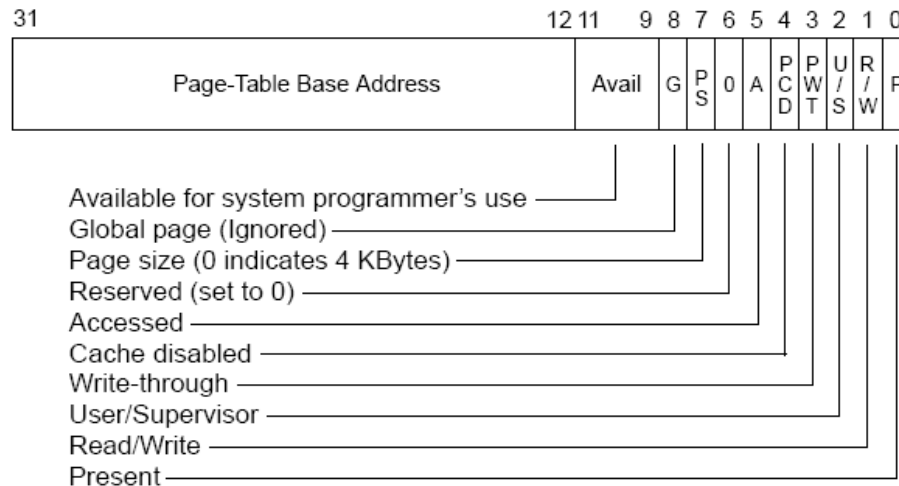
Tryb chroniony z segmentacją i stronicowaniem



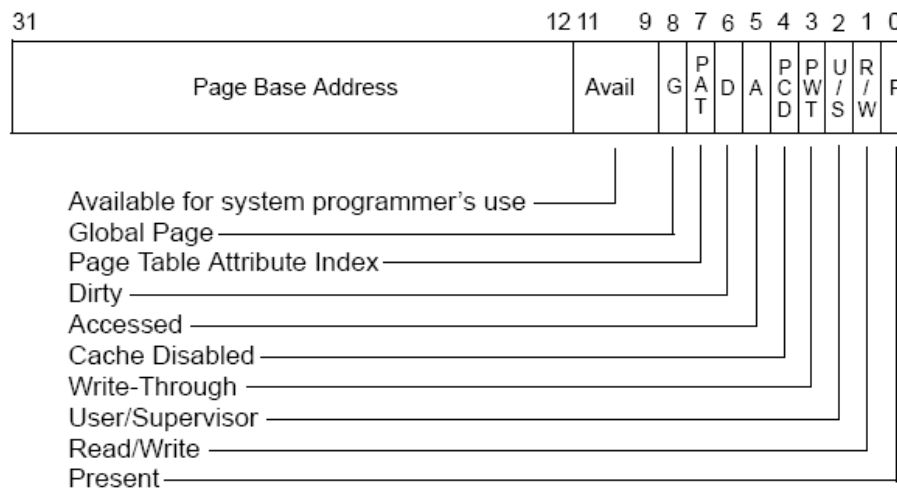


Wpis w tabelach PD i PT

Page-Directory Entry (4-KByte Page Table)

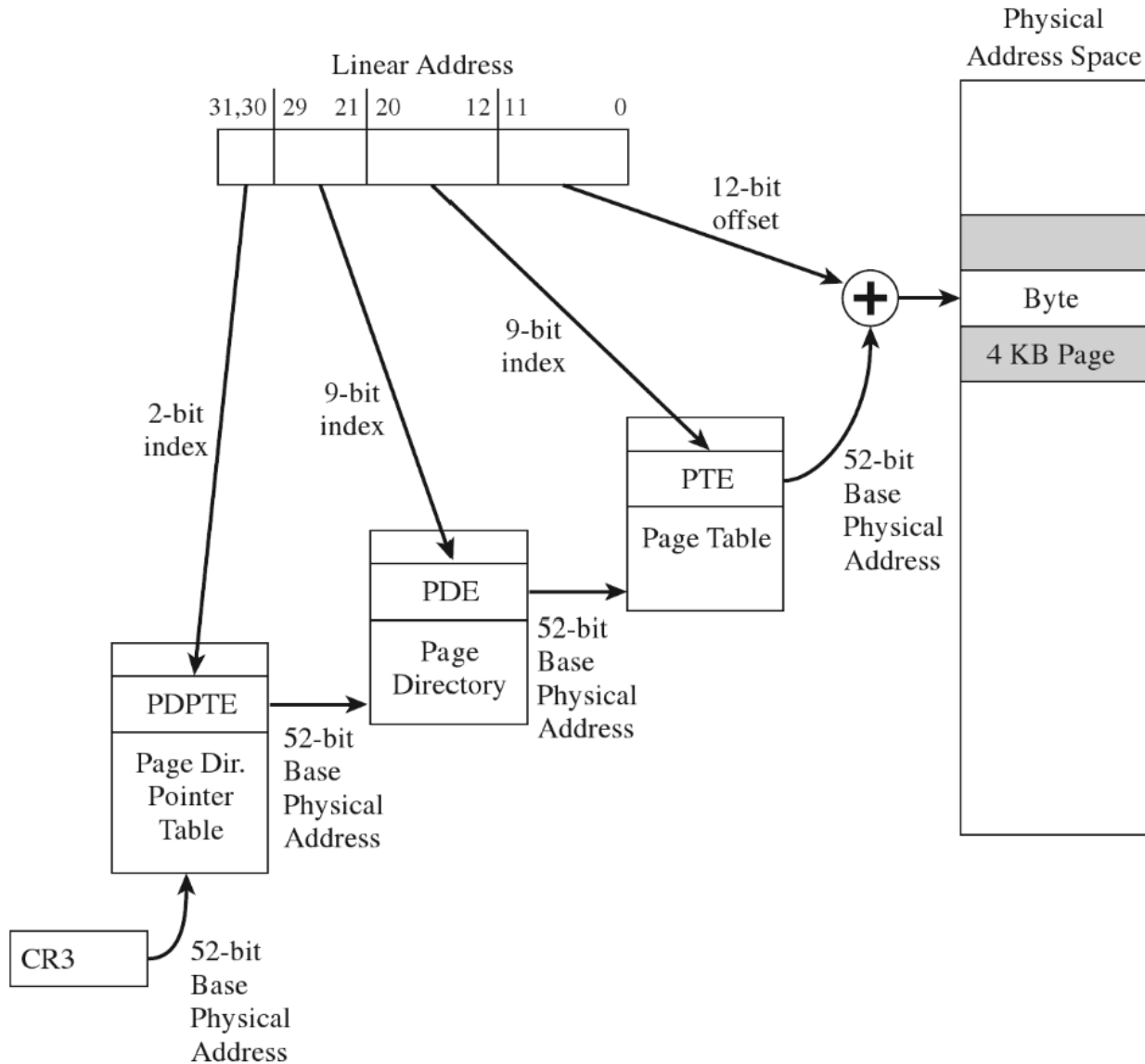


Page-Table Entry (4-KByte Page)



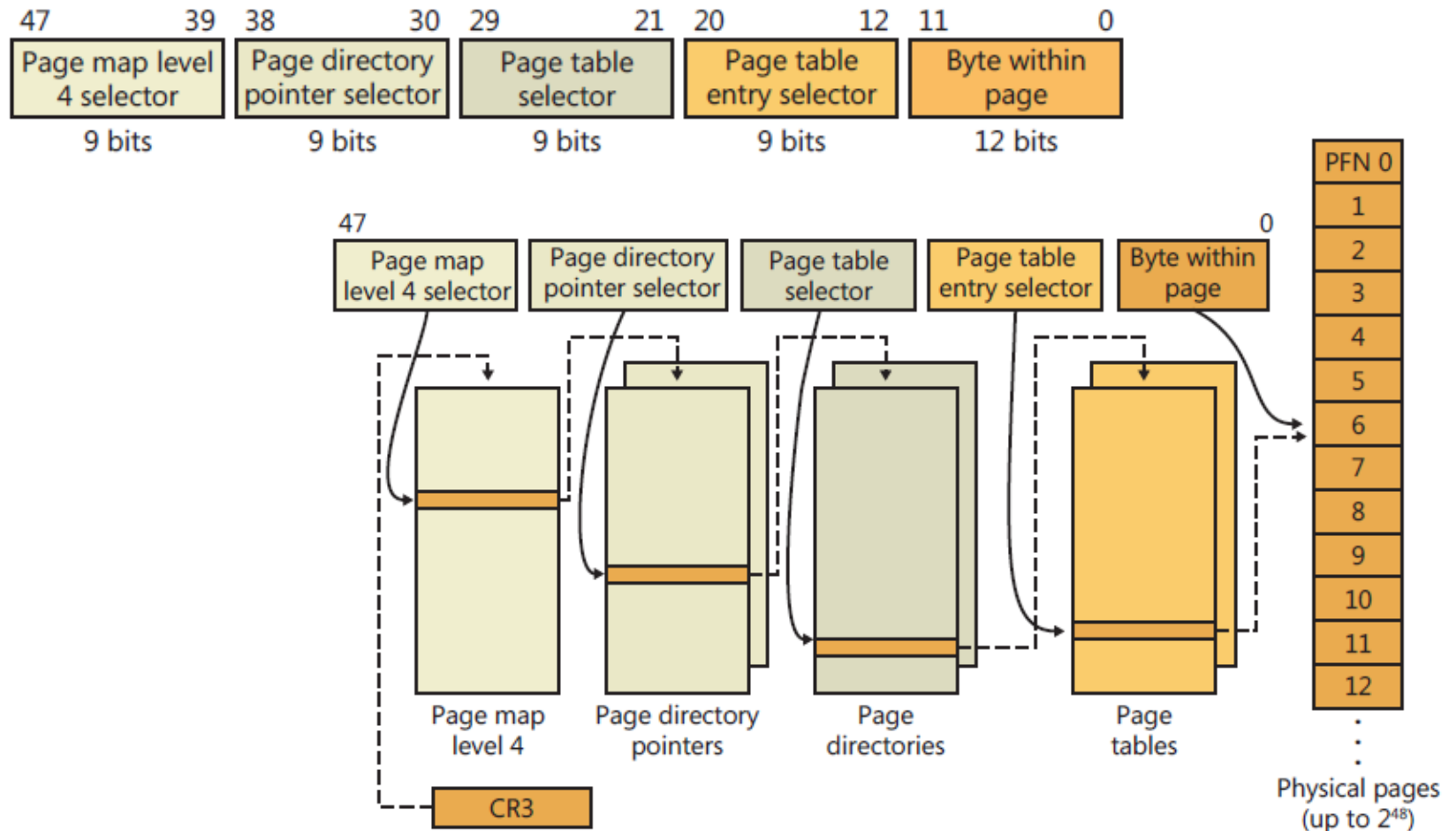


Tryb chroniony z segmentacją i stronicowaniem dla PAE



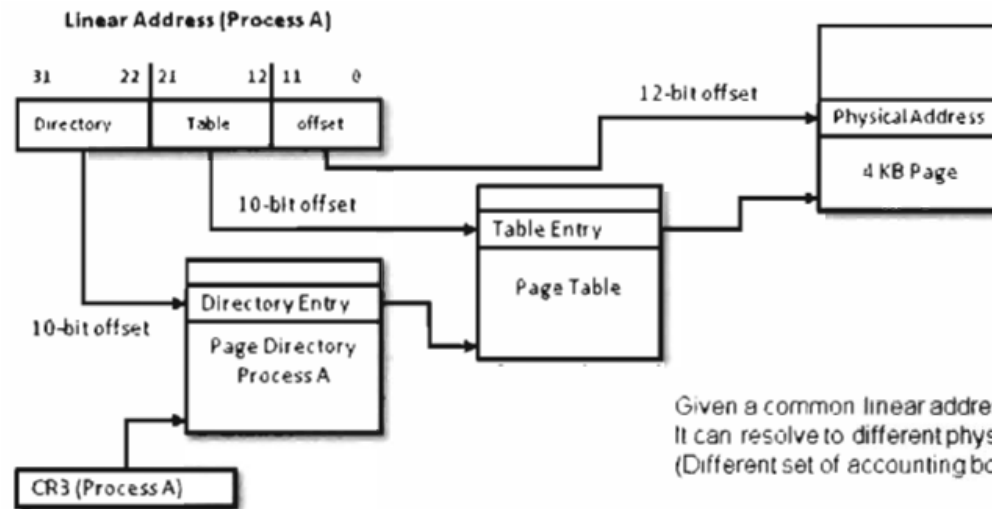


Tryb chroniony z segmentacją i stronicowaniem dla x64

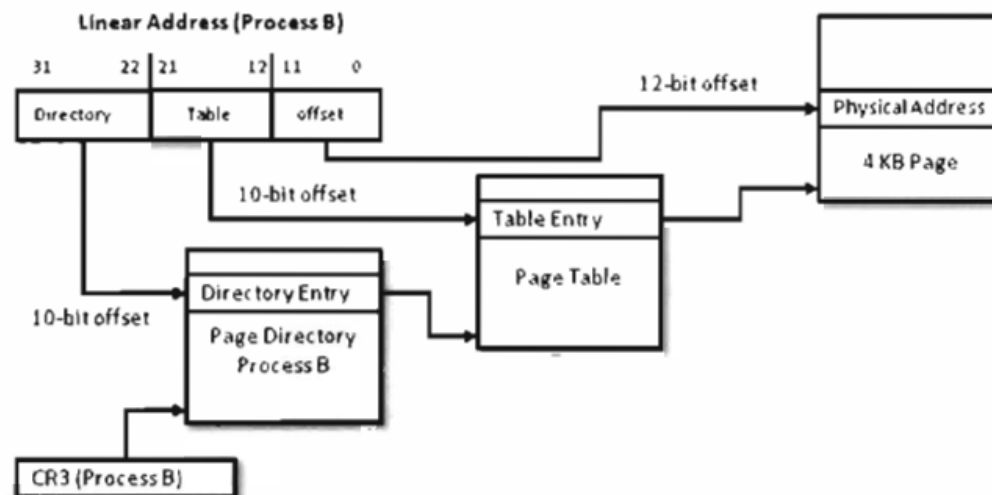




Separacja przestrzeni adresowych

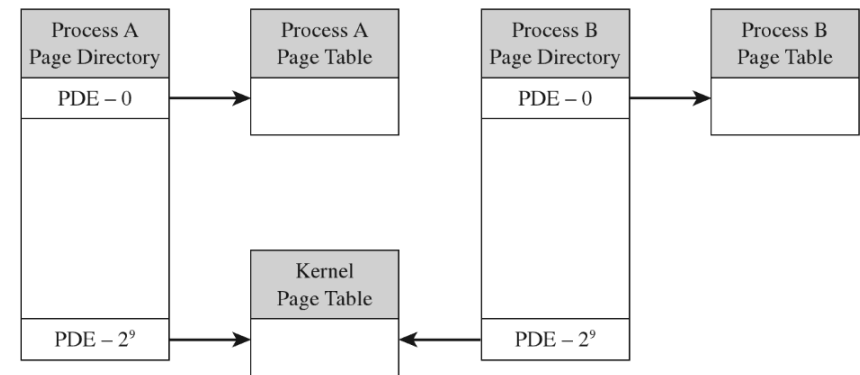
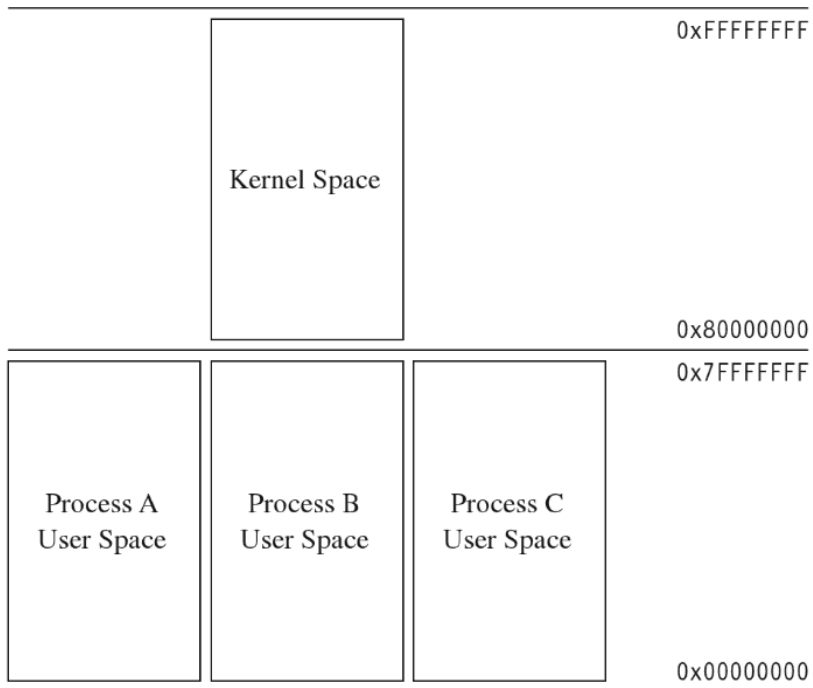


Given a common linear address (e.g., 0x11223344)
It can resolve to different physical address for each process
(Different set of accounting books used in each case)



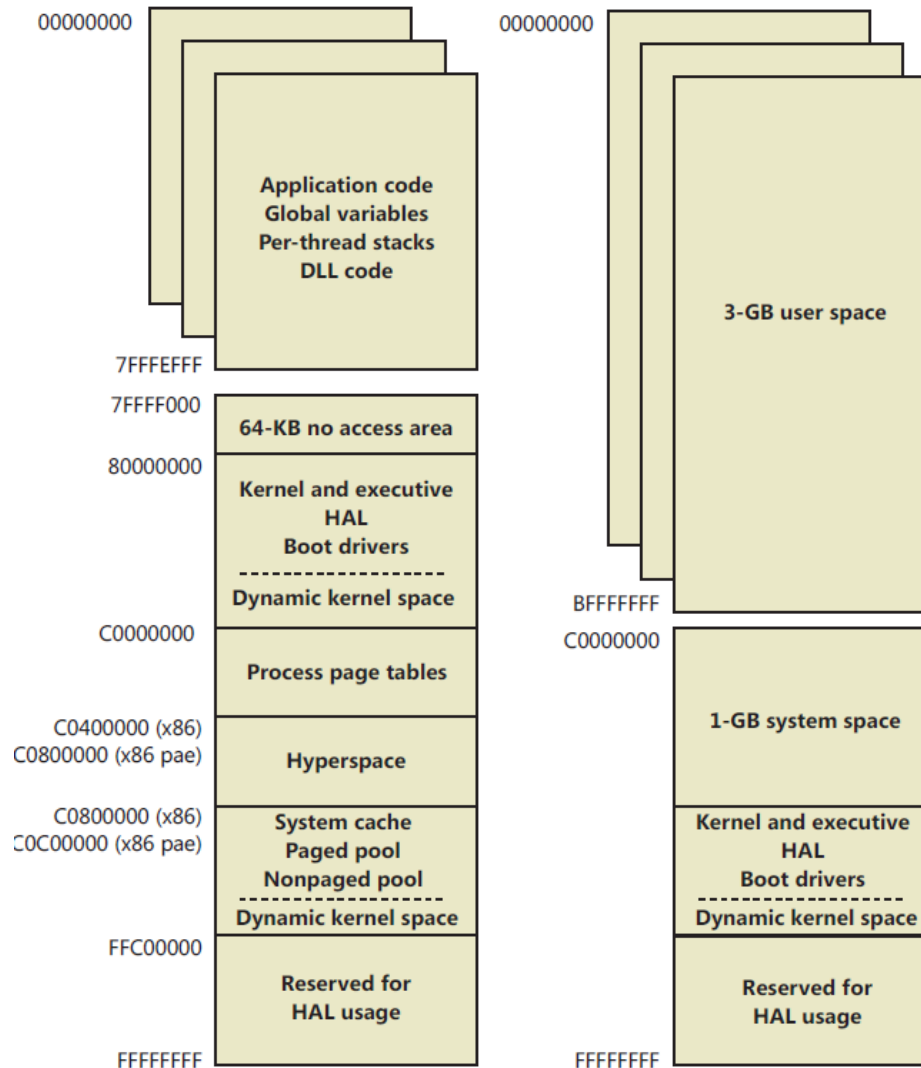


Podział liniowej przestrzeni adresowej





Mapa przestrzeni adresowej w trybie 32-bitowym



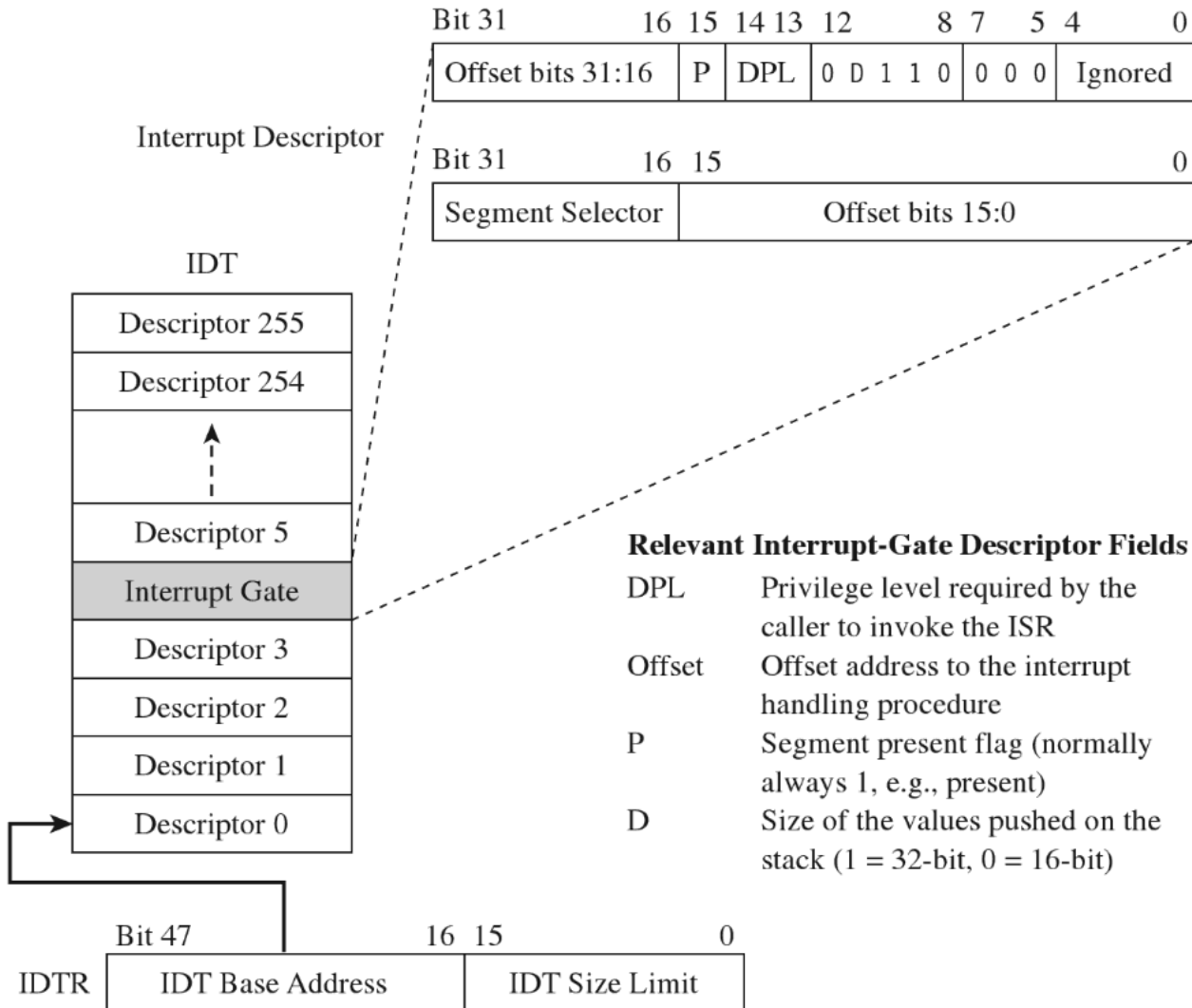


Mapa przestrzeni adresowej w trybie 64-bitowym

Start	End	Size	Description
FFFF0800`00000000	FFFFF67F`FFFFFFFF	238TB	Unused System Space
FFFFF680`00000000	FFFFF6FF`FFFFFFFF	512GB	PTE Space
FFFFF700`00000000	FFFFF77F`FFFFFFFF	512GB	HyperSpace
FFFFF780`00000000	FFFFF780`00000FFF	4K	Shared System Page
FFFFF780`00001000	FFFFF7FF`FFFFFFFF	512GB-4K	System Cache Working Set
FFFFF800`00000000	FFFFF87F`FFFFFFFF	512GB	Initial Loader Mappings
FFFFF880`00000000	FFFFF89F`FFFFFFFF	128GB	Sys PTEs
FFFFF8a0`00000000	FFFFF8bF`FFFFFFFF	128GB	Paged Pool Area
FFFFF900`00000000	FFFFF97F`FFFFFFFF	512GB	Session Space
FFFFF980`00000000	FFFFFa70`FFFFFFFF	1TB	Dynamic Kernel VA Space
FFFFFa80`00000000	*nt!MmNonPagedPoolStart-1	6TB Max	PFN Database
*nt!MmNonPagedPoolStart	*nt!MmNonPagedPoolEnd	512GB Max	Non-Paged Pool
FFFFFFFF`FFc00000	FFFFFFFF`FFFFFFFF	4MB	HAL and Loader Mappings

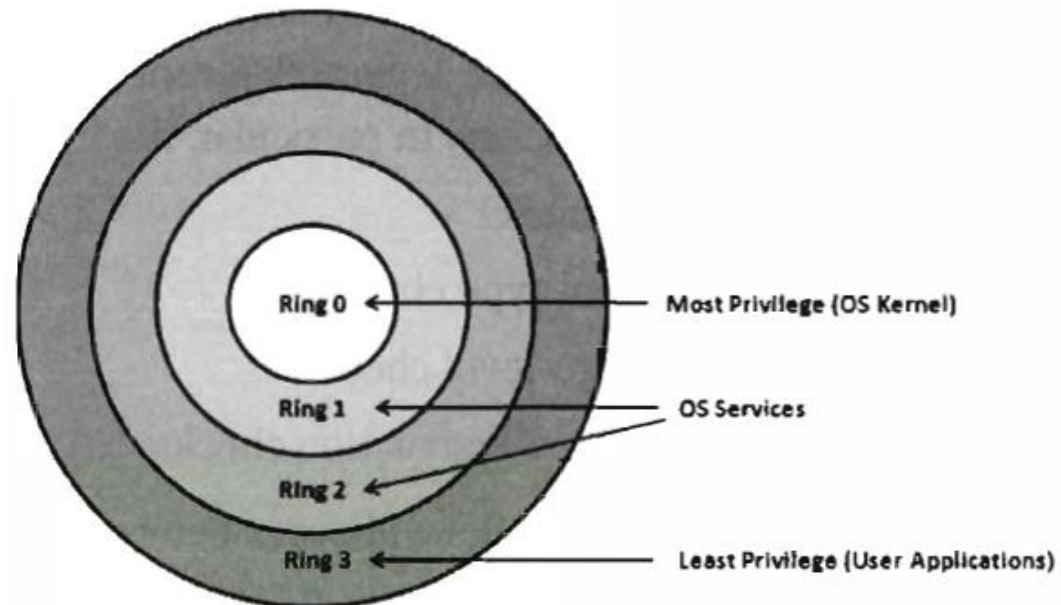


Przerwania sprzętowe



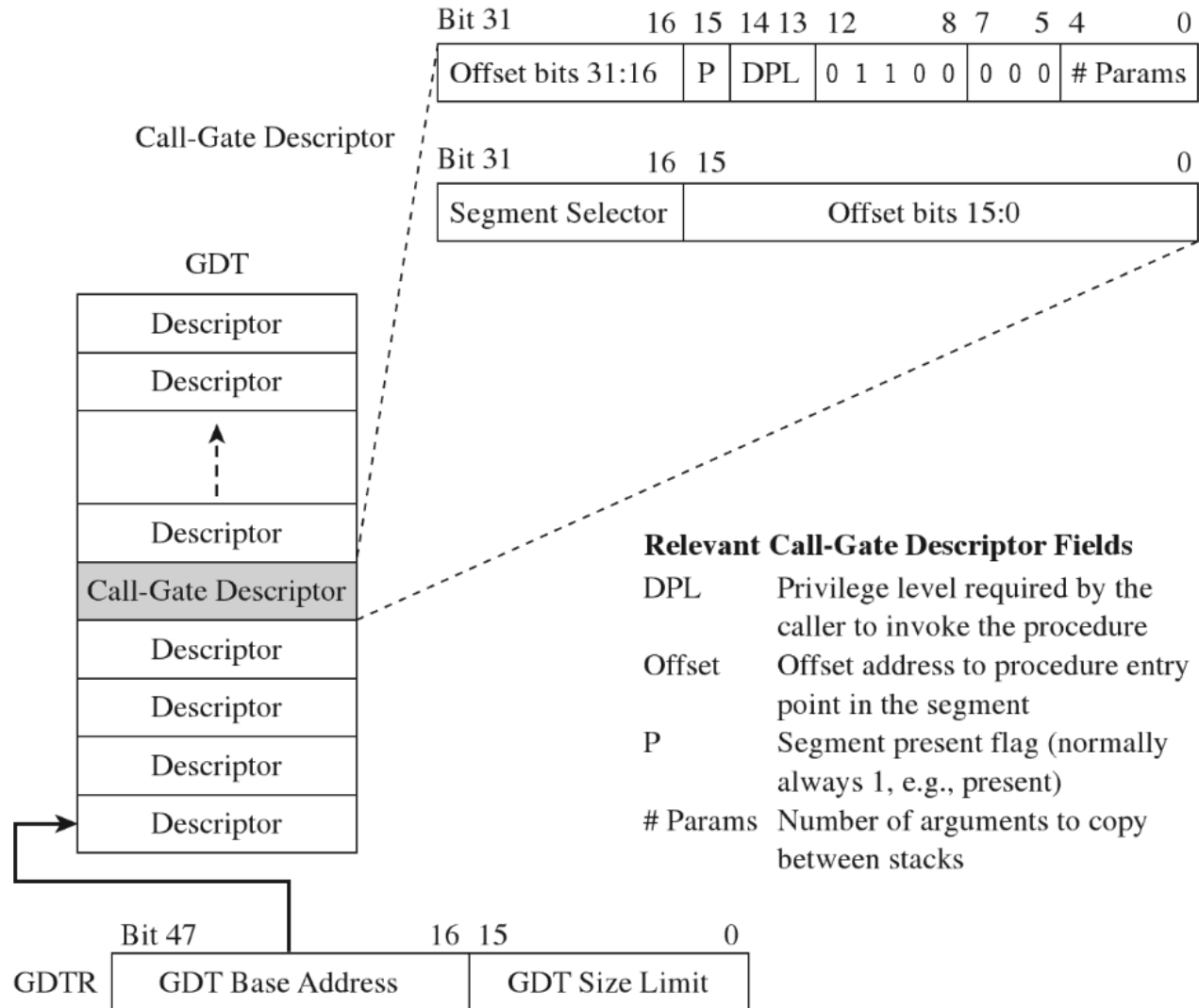


Poziomy uprzywilejowania



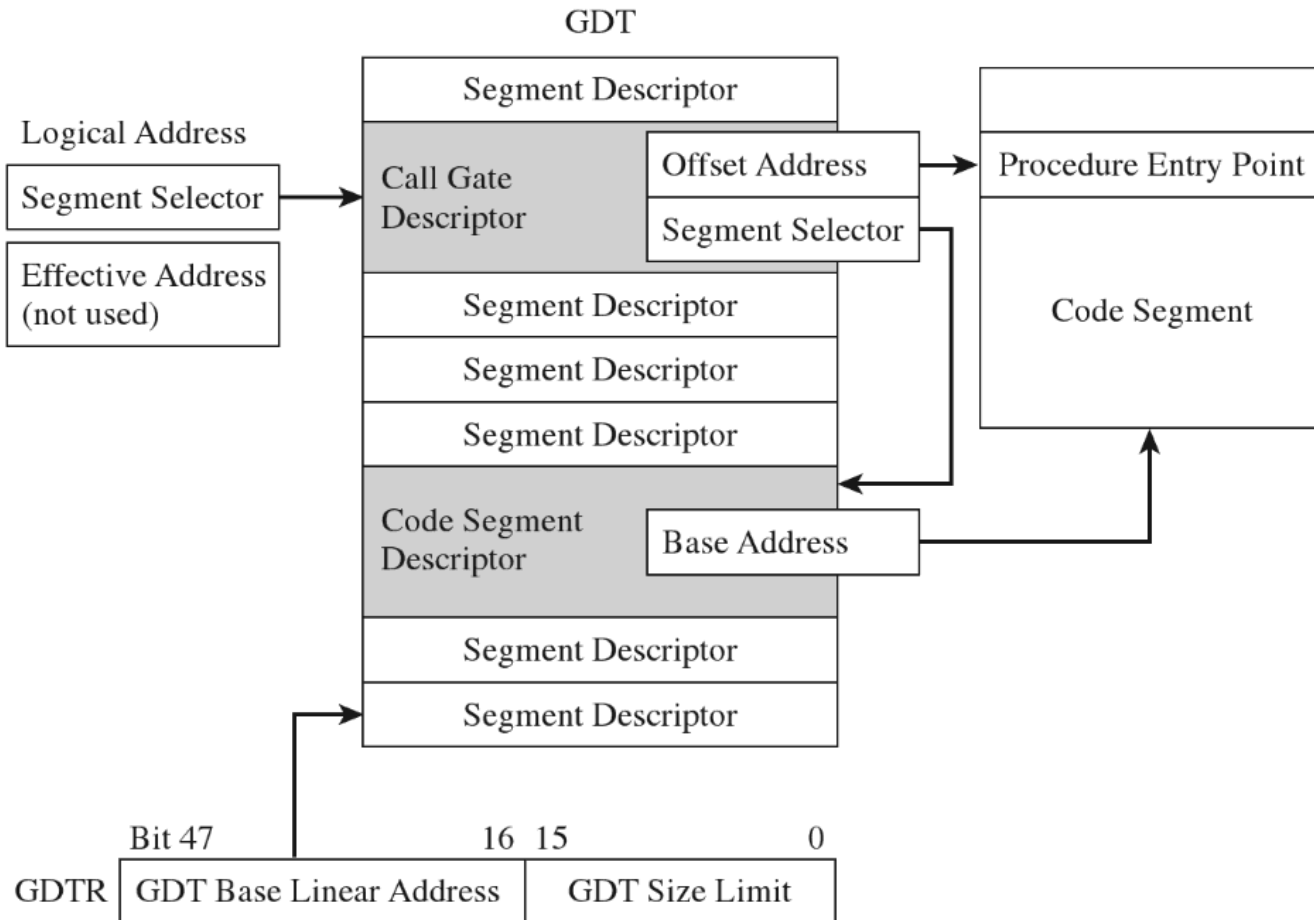


Deskryptory bram



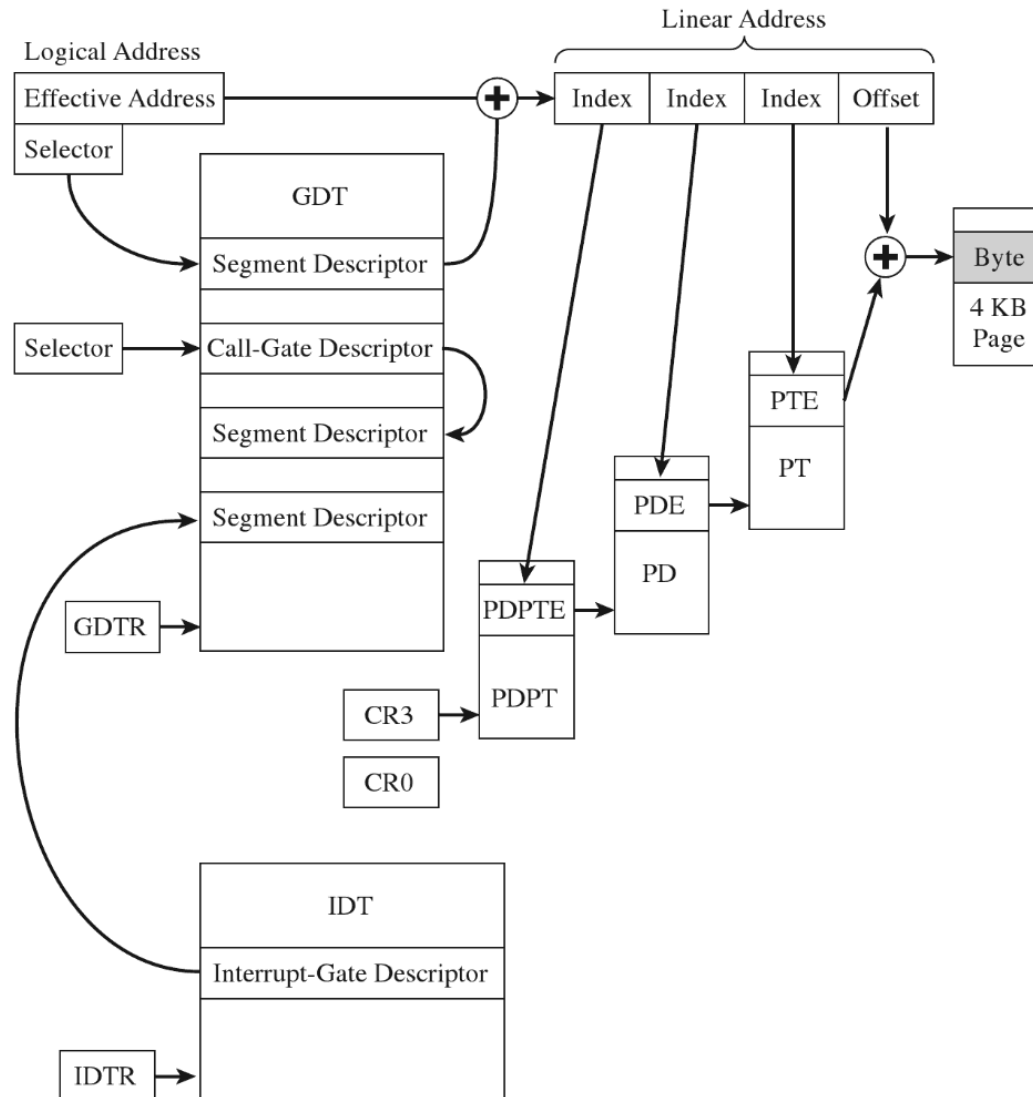


Wywołanie funkcji systemowej





Rzut całościowy na adresowanie





Pola rejestrów - podsumowanie

System Table Registers

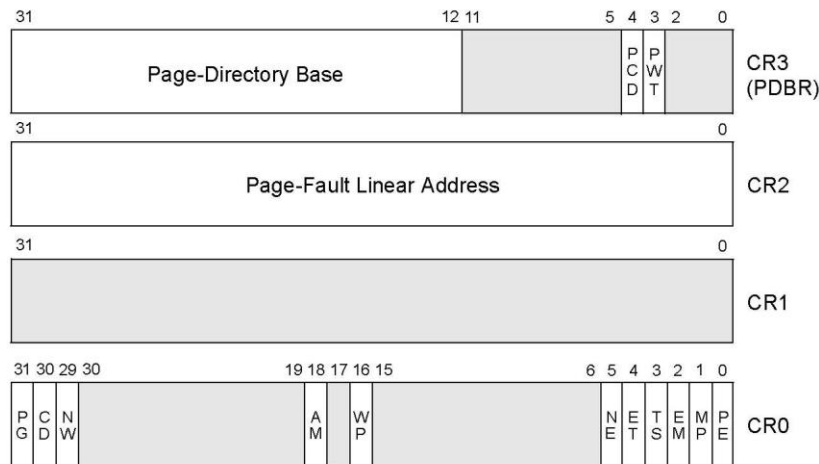
	47(79)	16	15	0
GDTR	32(64)-bit Linear Base Address		16-Bit Table Limit	
IDTR	32(64)-bit Linear Base Address		16-Bit Table Limit	

System Segment Registers

Segment Descriptor Registers (Automatically Loaded)

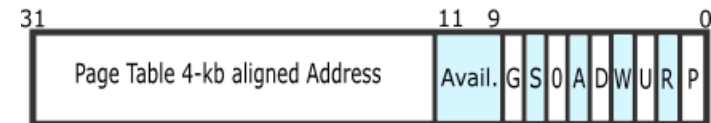
31	15	0	Attributes			
Task Register LDTR	Seg. Sel.	32(64)-bit Linear Base Address		Segment Limit		
	Seg. Sel.	32(64)-bit Linear Base Address		Segment Limit		

OSFXSR



Reserved

Page Directory Entry



G - Ignored
S - Page Size (0 for 4kb)
A - Accessed
D - Cache Disabled
W - Write Through
U - User/Supervisor
R - Read/Write
P - Present



Oprogramowanie układowe (Firmware)

Tomasz Dziubich
dziubich@eti.pg.edu.pl



POLITECHNIKA
GDAŃSKA

Agenda

1. Historia systemów operacyjnych i konieczność stosowania oprogramowania układowego
2. BIOS
3. UEFI



1. (1945-1955) – komputery oparte na lampach próżniowych, bez systemu operacyjnego
2. (1955-1965) – komputery oparte na tranzystorach, przetwarzanie wsadowe, zbiór wspólnych poleceń (załaduj program, uruchom, wydrukuj wynik). Najważniejszymi przedstawicielami „SO” są FMS (Fortran Monitor System) i IBSYS (system IBM dla komputera 7094)
3. (1965-1980) – mikrokomputery, wielozadaniowość; pierwszym krokiem było ujednolicenie listy rozkazów i tworzenie uniwersalnych języków programowania (asemblerzy i język C). Zaczęto zatem wytwarzać procesory o tej samej architekturze ISA (liście rozkazów, przykładowo IBM 360)
4. (1980-1990) – komputery osobiste (DOS, IBM, Microsoft); jednolity model systemu, pamięć wirtualna, sieciowość, interfejsy graficzne użytkownika, UNIX, Windows.
5. ?? Przetwarzanie mobilne i wszechobecne – zarządzanie energią, zorientowanie na zadania, wydajność, bezpieczeństwo.



Oprogramowanie układowe zgodne z BIOS

- Basic Input/Output System – oprogramowanie zawarte w programowalnych układach scalonych, które zawiera zestaw funkcji, procedur oraz danych zapewniających podstawową komunikację między sprzętem a systemem operacyjnym
- Utworzony dla trybu 16-bitowego
- Znajduje się w pamięci ROM-BIOS F000:0000 do FFFF:FFFF
- Zawiera m.in. program inicjujący pracę komputera (tzw. POST)
- Po włączeniu komputera pierwszy rozkaz pobierany jest z adresu F000:FFF0 (5 bajtów kodu FAR JMP do rozpoczęcia POST)
- Po adresem 0000:7C00 umieszczany jest rekord ładujący (z dysku ze schematem partycjonowania MBR)
- Uruchamiany w trybie rzeczywistym 16-bitowym
- Przełączenia na tryb chroniony dokonuje loader systemu operacyjnego



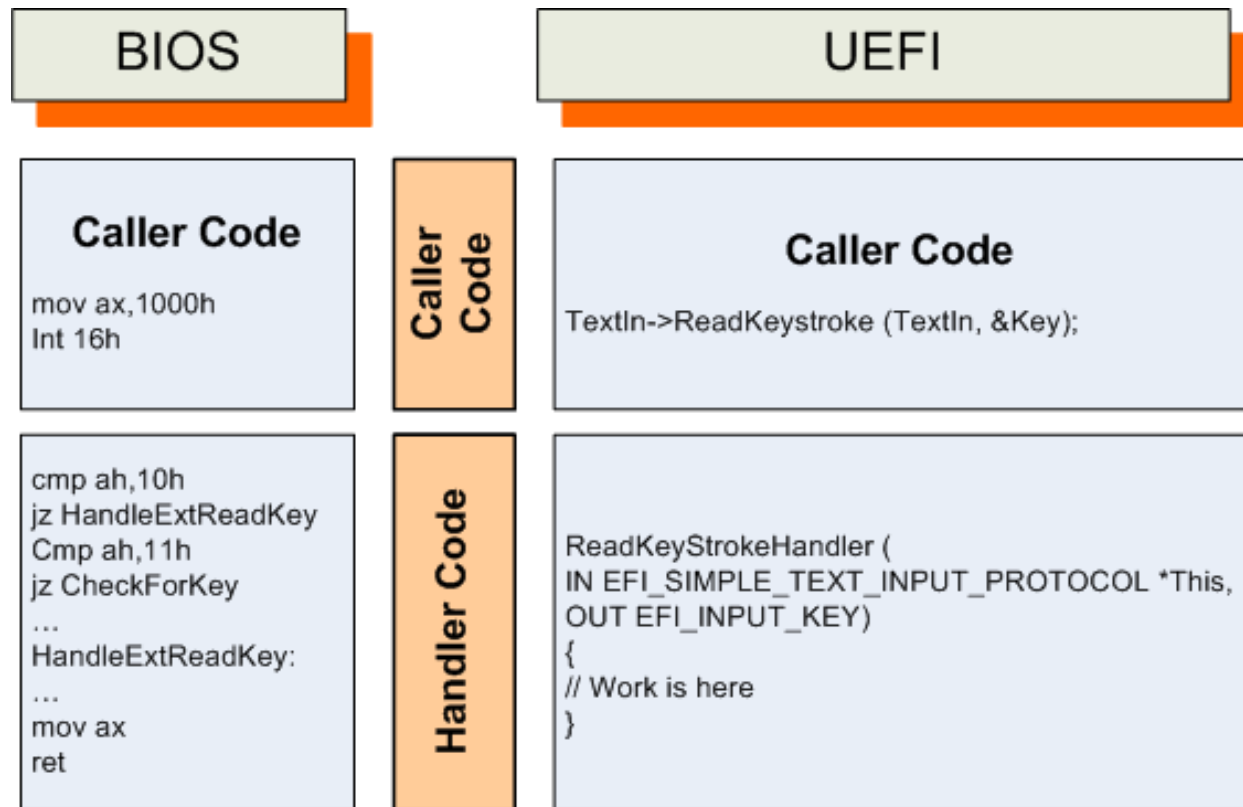
Oprogramowanie układowe zgodne z BIOS (2)

BIOS pełni cztery podstawowe funkcje:

- Test komponentów komputera podczas każdego włączenia komputera, funkcja ta została nazwana Power-On-Self-Test
- Kontrola nad fundamentalnymi operacjami sprzętowymi np. nadzorowanie procesu odświeżania pamięci dynamicznej, komunikacja z urządzeniami zewnętrznymi
- Realizacja buforów z danymi systemowymi: BIOS rezerwuje mały blok pamięci RAM nazywany BIOS Data Area, gdzie są przechowywane informacje dotyczące konfiguracji komputera oraz dane z urządzeń np. bufor klawiatury
- Udostępnienie funkcji niskopoziomowych łączących system operacyjny ze sprzętem komputerowym np. funkcje wyświetlające, odczytujące, komunikacja z dyskiem.
 - 0x10 Karta graficzna
 - 0x13 Przerwanie do obsługi dysków twardych
 - 0x14 Port szeregowy
 - 0x16 Funkcje klawiatury

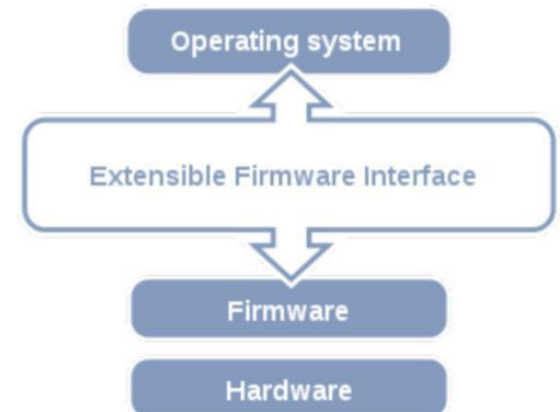


Przykład kodu dla BIOS i UEFI



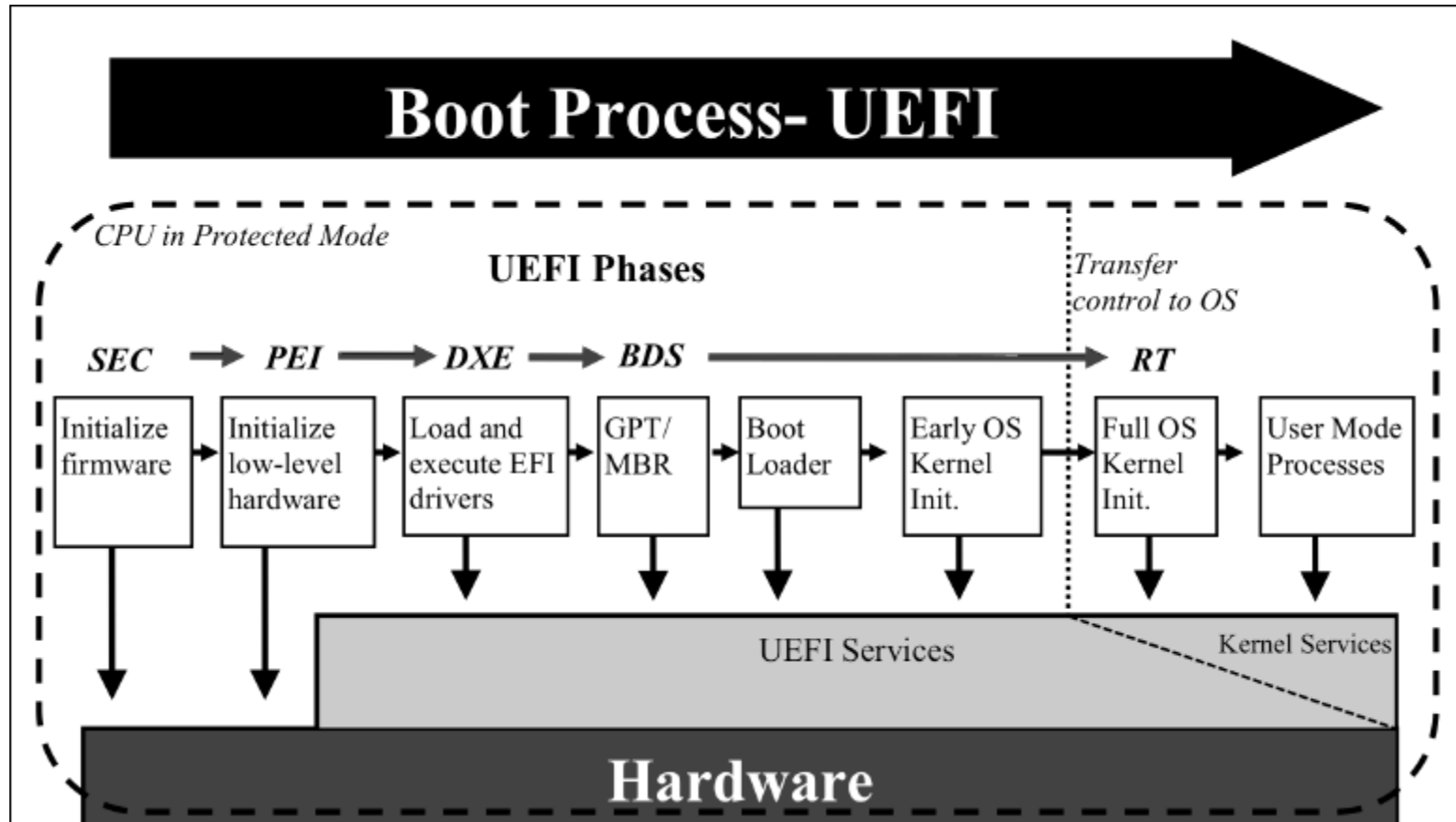


- UEFI - Unified Extensible Firmware Interface
- Celem Unified EFI Forum jest wyeliminowanie BIOS z procesu rozruchu komputerów i zastąpienie go standaryzowanym UEFI.
- Pozwala na szybsze ładowanie i uruchamianie systemu operacyjnego.
- Udostępnia zestaw narzędzi diagnostycznych, bez konieczności instalowania SO
- UEFI specyfikuje środowisko, do którego mogą być dodawane aplikacje wykonywane jeszcze przed uruchomieniem właściwego systemu operacyjnego, a korzystające z systemu plików, trybów graficznych o wysokiej rozdzielczości czy implementacji stosu protokołów sieciowych.
- Początkowo firma Intel i HP – ok.1995 roku
- Unified EFI Forum – 2005 r.





Start komputera wg EFI

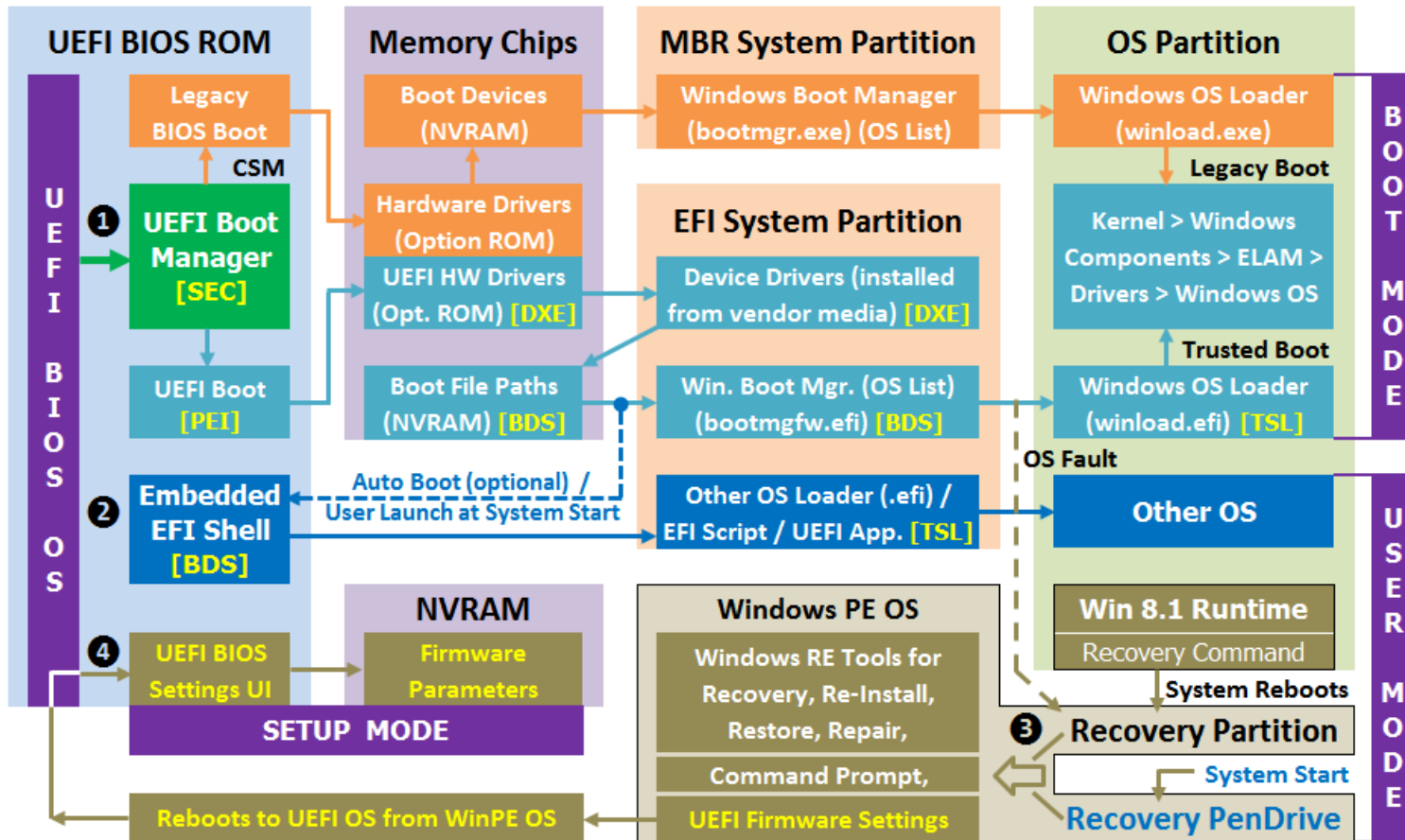




Start komputera wg EFI (2)

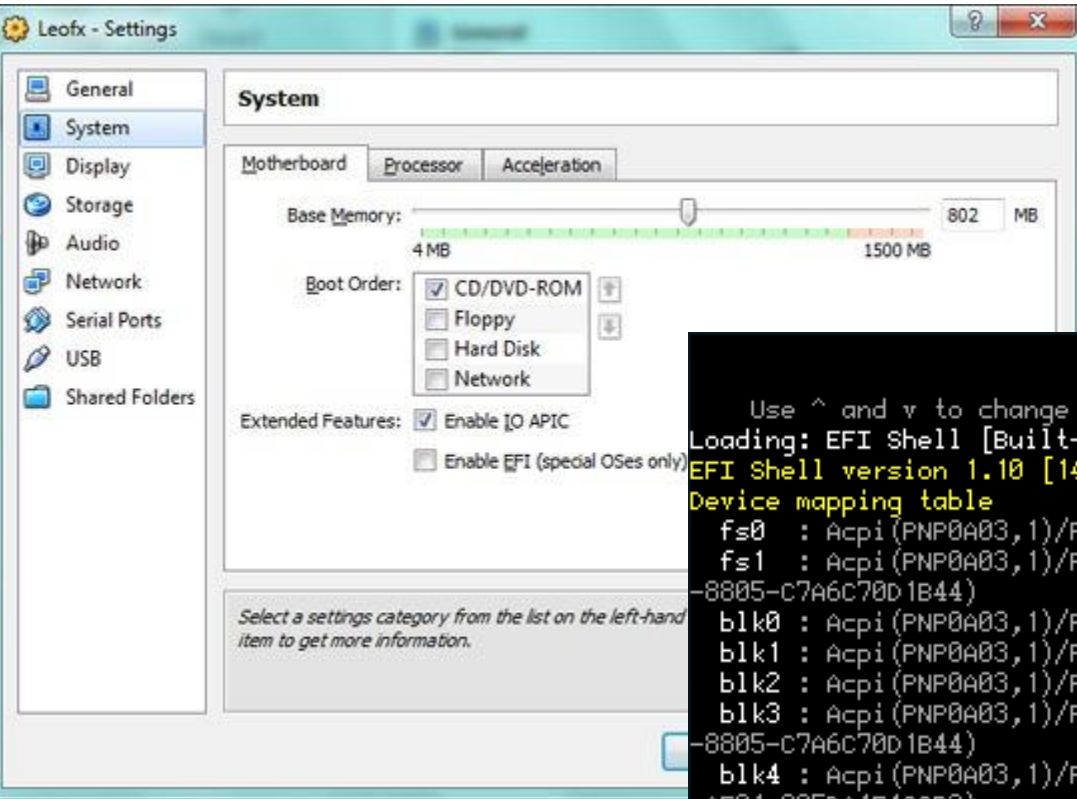
- Sprawdzenie integralności obrazu (SEC)
- Ładowanie lekkiej warstwy PEI (Pre-EFI Initialization), która wykonuje większość rzeczy związanych z rozruchem komputera (=POST BIOS-u) jak np. inicjalizacja chipsetu, pamięci, enumeracja szyn danych itp.
- Ładowanie DXE (Driver Execution Environment) dla zapewnienia podstawowego API dla sterowników
- Wykrywane są partycje dyskowe (BDS)
- Uruchomienie programów rozruchowych OS'ów według kolejności z listy startowej
- Jeśli bootloader OS zadziała poprawnie, zostaną wyłączone usługi rozruchowe a sterowanie zostanie przekazane do systemu operacyjnego
- Jeśli jednak nie ma znanego programu rozruchowego lub wszystkie okazały się błędne zostanie załadowana powłoka EFI Shell

Start komputera wg EFI (3)





Virtualbox, QEMU i EFI



```
Use ^ and v to change option(s). Use Enter to select an option
Loading: EFI Shell [Built-in]
EFI Shell version 1.10 [14.62]
Device mapping table
fs0 : Acpi(PNP0A03,1)/Pci(1|0)/Ata(Primary,Master)/CDROM(Entry0)
fs1 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)/HD(Part1,SigEF114FA1-368E-4764-8805-C7A6C70D1B44)
blk0 : Acpi(PNP0A03,1)/Pci(1|0)/Ata(Primary,Master)
blk1 : Acpi(PNP0A03,1)/Pci(1|0)/Ata(Primary,Master)/CDROM(Entry0)
blk2 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)
blk3 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)/HD(Part1,SigEF114FA1-368E-4764-8805-C7A6C70D1B44)
blk4 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)/HD(Part2,SigA101853C-0871-43FA-AF01-805DA1E4CCB3)
blk5 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)/HD(Part3,SigCEE8B18-6215-45EB-B637-F8AA111F99C7)
blk6 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun1,Lun0)/HD(Part4,SigBCED0BBE-EF0C-40B7-91B5-014F2791C0D7)
blk7 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun2,Lun0)
blk8 : Acpi(PNP0A03,1)/Pci(3|0)/Scsi(Pun2,Lun0)/HD(Part1,Sig3016A371-7779-41E7-B4BB-A7CF684CCDB7)
Shell>
```



Używamy standardowego kompilatora C/asm (format pliku PE).

- **pakiet** – jest to zbiór modułów posiadający pewne wspólne właściwości. (np. pakiet o nazwie); Zawiera plik deklaracyjny z rozszerzeniem .dec , który zawiera informacje publiczne, jak np. ścieżki do wykorzystywanych plików nagłówkowych; plik opisujący proces budowania modułu (rozszerzenie .dsc), który zawiera listę modułów do zbudowania i instancji klas bibliotecznych.
- **moduł** – jest to zbiór plików źródłowych (najczęściej .c lub .asm) i nagłówkowych (.h), które tworzą logiczną całość np. sterownik, bibliotekę, aplikację UEFI, aplikację EFI-SHELL. Moduł posiada swój własny podkatalog i plik opisujący moduł z rozszerzeniem .inf , w którym znajdują się ścieżki do plików tworzących moduł i lista wymaganych przez moduł klas bibliotecznych.
- **klasa biblioteczna (Library Class) i instancja biblioteki (Library Instance)**



Podstawowe pojęcia UEFI

Podstawowym oznaczeniem w UEFI jest GUID, czyli unikalny 128-bitowy identyfikator, który przypisuje się dla każdego obiektu czy zmiennej w systemie, np. partycja zarezerwowana przez Microsoft E3C9E316-0B5C-4DB8-81-7D-F9-2D-F0-02-15-AE.

Drugim podstawowym elementem jest uchwyt (EFI_HANDLE), który jest wskaźnikiem na dowolną strukturę w systemie (typedef void * EFI_HANDLE). Tworząc obraz, plik, protokół, urządzenie wiąże się z nim uchwyt.

Oprócz uchwytu, drugim podstawowym pojęciem UEFI jest protokół. Protokół (protocol) składa się z 128-bitowego GUID oraz struktury interfejsu protokołu (Protocol Interface). Struktura (a zatem protokół) zawiera funkcje i instancje danych, które są wykorzystywane przy dostępie do urządzenia lub ogólniej obiektu.

Protokół jest dodany i udostępniony dla wszystkich aplikacji przez oprogramowanie sterownika UEFI poprzez usługi obsługi protokołów (Protocol Handler Services). Jest to zestaw funkcji (Install, Uninstall, Reinstall, RegisterProtocol, LocateHandle, HandleProtocol, OpenProtocol, CloseProtocol, LocateHandlerBuffer, i inne).

Oznacza to, że oprogramowanie układowe utrzymuje bazę danych (de facto lista jednokierunkowa), która przechowuje wszystkie protokoły (oraz uchwyty do urządzeń). Stanowi on fragment EFI_BOOT_SERVICES.



- EFI_BOOT_SERVICES i EFI_RUNTIME_SERVICES
- Mapa pamięci – EFI_BOOT_SERVICES.GetMemoryMap()
- Zmiana adresów -
EFI_RUNTIME_SERVICES.SetVirtualAddressMap()
- Ładowanie obrazu EFI_BOOT_SERVICES.LoadImage()
- Uruchomienie obrazu EFI_BOOT_SERVICES.StartImage()
- Odczytanie/Ustawienie zmiennych
EFI_RUNTIME_SERVICES.Get/SetVariable()
- Szczegółowe warunki na stronie 25 w dokumentacji



Urządzenia pamięci masowej (Mass storage systems)

Tomasz Dziubich
dziubich@eti.pg.edu.pl



1. Podstawowe pojęcia związane z pamięcią masową
2. Fizyczna budowa dysku HDD
3. Interfejsy ATA(PI) i AHCI
4. Adresacja sektorów
5. API/ABI dla programisty systemowego



Koncepcja zapisu pliku na dysku

Plik to logiczna jednostka/zespół informacji tworzony zazwyczaj przez proces, który z jakichkolwiek względów został wyodrębniony przez twórcę.

T	o		j	e	s	t		p	l	i	k	.
---	---	--	---	---	---	---	--	---	---	---	---	---

Plik z punktu widzenia twórcy stanowi zdefiniowany ciąg danych (bajtów) następujących po sobie.

54	6F	20	6A	65	73	74	20	70	6C	69	6B	2E
----	----	----	----	----	----	----	----	----	----	----	----	----

Chcąc odwoływać się do pliku musimy posiadać informacje o nich (np. nazwę lub numer id, położenie początkowego bajtu, liczbę bajtów i inne).

Dane o danych to metadane.



Koncepcja zapisu pliku na dysku

- Plik przechowywany zatem na dysku do dane i metadane.
- Niektóre z metadanych są istotne z punktu twórcy SO, a nieistotne z punktu widzenia twórcy (np. położenie pliku na dysku).
- Rolą SO jest zatem odwzorowanie pliku/-ów na urządzenia fizyczne; teoretycznie rozmiar pliku może być dowolnie duży, ale w praktyce jego rozmiar jest ograniczony przez ilość miejsca w fizycznym urządzeniu pamięciowym oraz format metadanych.
- System operacyjny traktuje plik jako ciąg bajtów bez żadnej dodatkowej struktury; znaczenie bajtów zawartych w pliku określone jest przez jego twórcę i użytkownika;

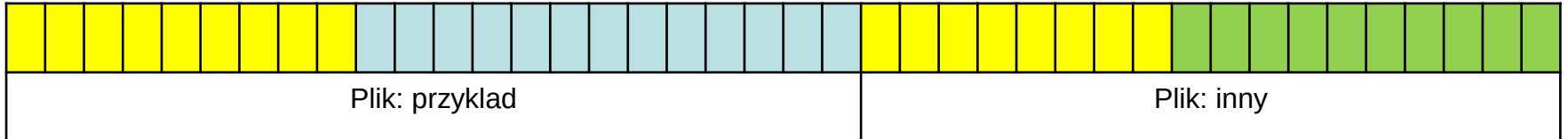


70 p	72 r	7A z	79 y	6B k	6C l	61 a	64 d	0D	54	6F	20	6A	65	73	74	20	70	6C	69	6B	2E
---------	---------	---------	---------	---------	---------	---------	---------	----	----	----	----	----	----	----	----	----	----	----	----	----	----

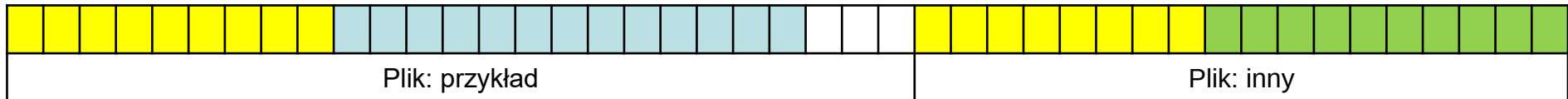
- Na dysku zatem oprócz bajtów przynależnych do pliku, znajdują się:
 - bajty opisujące metadane o plikach,
 - metadane o obszarach, na których znajdują się plik (metadane o systemach plików),
 - Metadane o tych obszarach (metadane o partycjach).
- Mogą się też znajdować obszary, w których zwarto bajty opisujące własności samego dysku lub obszary nie wykorzystane.



Budowa fizyczna i logiczna dysku HDD



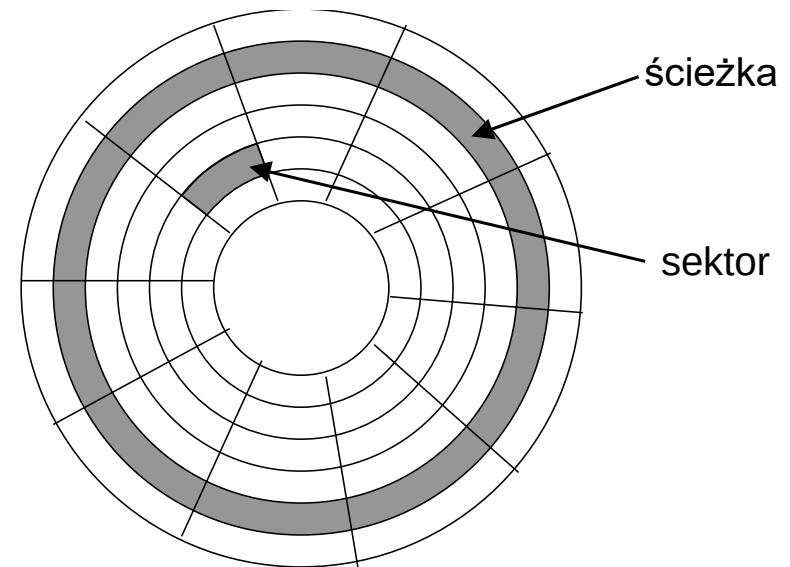
Rysunek: plik, metadane, partycja, dysk



Metadane plik: przykład	Metadane plik: inny	Wolne miejsce na metadane	Plik: przykład wraz ze slackiem	Plik: dane wraz ze slackiem

Budowa dysku twardego HDD

- główną część dysku twardego stanowi zespół (zwykle 2 lub 3) obracających się tarcz (dysków) pokrytych obustronnie materiałem magnetycznym;
- informacje są zapisywane i odczytywane za pomocą sprzężonych ruchomych głowic, a całość sterowana jest przez lokalny mikroprocesor;
- liczba obrotów na minutę wynosi zwykle 7200, a wyjątkowo może dochodzić do 15 tys.;
- głowice odczytu/zapisu znajdują się w odległości kilku μm od powierzchni dysku;





Interfejs ATA/SATA

- Istnieje wiele pojęć (akronimów) związanych z dyskami: IDE, AHCI, ATA, PATA, SATA;
- IDE – wynika z faktu umieszczenia (mikro)kontrolera w urządzeniu pamięci masowej (*Integrated Drive Electronics*)
- Komunikacja w systemie komputerowym następuje obecnie pomiędzy dwoma kontrolerami: kontrolerem SATA znajdującym się w mostku południowym oraz kontrolerem dysku IDE.
- Komunikacja na poziomie fizycznym i łącza danych odbywa się w sposób szeregowy (Serial ATA, SATA) i równoległy (Parallel ATA, PATA). Obecnie w zasadzie występuje tylko szybka transmisja szeregową;
- ATA - AT Attachment, standard opisujący zestaw komend do komunikacji pomiędzy komputerem a urządzeniem pamięci masowej.
- Standard przechodził kolejne metamorfozy (ATAPI, AT Attachment Packet Interface) i obecnie dla urządzeń SATA nosi nazwę AHCI (Advanced Host Controller Interface).



- W standardzie ATA, komputer komunikuje się z dyskiem poprzez 8- i 16-bitowe rejestry, znane w literaturze anglojęzycznej jako Command (Control) Block Registers lub AT-Task-File; rejestry te dostępne w są przestrzeni adresowej wejścia-wyjścia (rozkazy in, out);
- standard ATA przewiduje dołączenie do komputera maksymalnie czterech urządzeń — adresy 1F0H i 3F6H, oraz 170H i 376H
- znaczenie niektórych rejestrów może się zmieniać w zależności od trybu pracy dysku;
- dane mogą być przesyłane w dwóch trybach:
 - PIO (ang. Programmed Input/Output) – prędkość przesyłania do 20 MB/s
 - DMA (ang. Direct Memory Access) – prędkość przesyłania do 150 MB/s
- Przykład: https://wiki.osdev.org/ATA_PIO_Mode



Adresacja przestrzeni dyskowej w ATA

- wykonanie operacji zapisu lub odczytu wskazanego sektora wymaga podania jego położenia fizycznego , czyli:
 - numeru dysku,
 - numeru powierzchni (głowicy): 0, 1, 2, ...
 - numeru cylindra (ścieżki): 0, 1, 2, ...
 - numeru sektora: 1, 2, 3, ...
- taki sposób adresowania sektorów oznaczany jest skrótem CHS (ang. Cylinder / Head / Sector Per Track);
- specyfikacja interfejsu ATA określa m.in. rejestry 8-bitowe, do których należy wpisać parametry opisujące położenie zapisywanego lub odczytywanego sektora:
 - Cylinder Low numer cylindra (część młodsza i starsza)
 - Cylinder High
 - Sector Number numer sektora (pierwszy sektor ma numer 1, nr 0 jest nieużywany)
 - Device / Head numer urządzenia / numer głowicy (podany na 4 młodszych bitach)



Adresacja przestrzeni dyskowej w AHCI

- adresowanie wg schematu CHS polega na podaniu położenia fizycznego potrzebnego sektora poprzez określenie numeru powierzchni (głowicy), numeru ścieżki i numeru sektora; odmienną koncepcję adresowania przyjęto w schemacie adresowania logicznego LBA (ang. Logical Block Addressing);
- w tym schemacie każdy sektor ma przypisany unikatowy numer, podobnie jak bajty w pamięci operacyjnej;
- zatem poszczególne sektory dysku są wskazywane przez liniowo adresowane adresy sektorów, przy czym sektor 1 na ścieżce 0 na powierzchni (głowicy) 0 ma przyporządkowany adres logiczny LBA 0; LBA stanowi obecnie typowy sposób adresowania sektorów dysku;
- Obecnie w związku ze zbyt małą pojemnością na liczbę sektorów (w systemach 32-bitowych) wprowadzono adresację Long LBA



	BIOS	IDE	Ogran.
Maks. liczba sektorów na ścieżce	63	255	63
Maksymalna liczba cylindrów	1024	65536	1024
Maksymalna liczba głowic	256	16	16
Pojemność dysku	8.4 GB	136.9 GB	528 MB

- Int 13h, np..
 - ♦ przykładowo odczytanie sektora wymaga podania następujących parametrów:
 - numer funkcji w AH = 2
 - liczba odczytywanych sektorów w AL,
 - nr sektora w CL na bitach 5-0,
 - nr cylindra (ścieżki) w CH (młodsze 8 bitów) i CL (bity 6, 7 — starsze dwa bity nr cylindra);
 - nr głowicy w DH,
 - nr dysku w DL (w przypadku dysków twardych bit 7 zawiera 1),
 - adres bufora, do którego zostaną wprowadzone odczytane dane w rejestrach ES:BX;



Rozszerzone operacje dyskowe BIOS

- ♦ omówiony wcześniej schemat adresowania pozwala adresować dyski o maksymalnej pojemności do 8.4 GB; w celu adresowania dysków o większej pojemności zdefiniowano nowe funkcje usługowe BIOSu o numerach 41H – 48H; (Device Address Packet)

Offset	Rozm.	Opis pola
00H	1	rozmiar obszaru parametrów (zwykle 16 bajtów)
01H	1	bajt zarezerwowany (= 0)
02H	1	liczba przesyłanych bloków
03H	1	bajt zarezerwowany (= 0)
04H	4	położenie bufora w pamięci RAM podane w postaci <i>segment:offset</i>
08H	8	początkowy numer bloku



Schematy partycjonowania (Partitioning schemes)

Tomasz Dziubich
dziubich@eti.pg.edu.pl



POLITECHNIKA
GDAŃSKA

Agenda

1. Schemat MBR
2. Schemat GPT

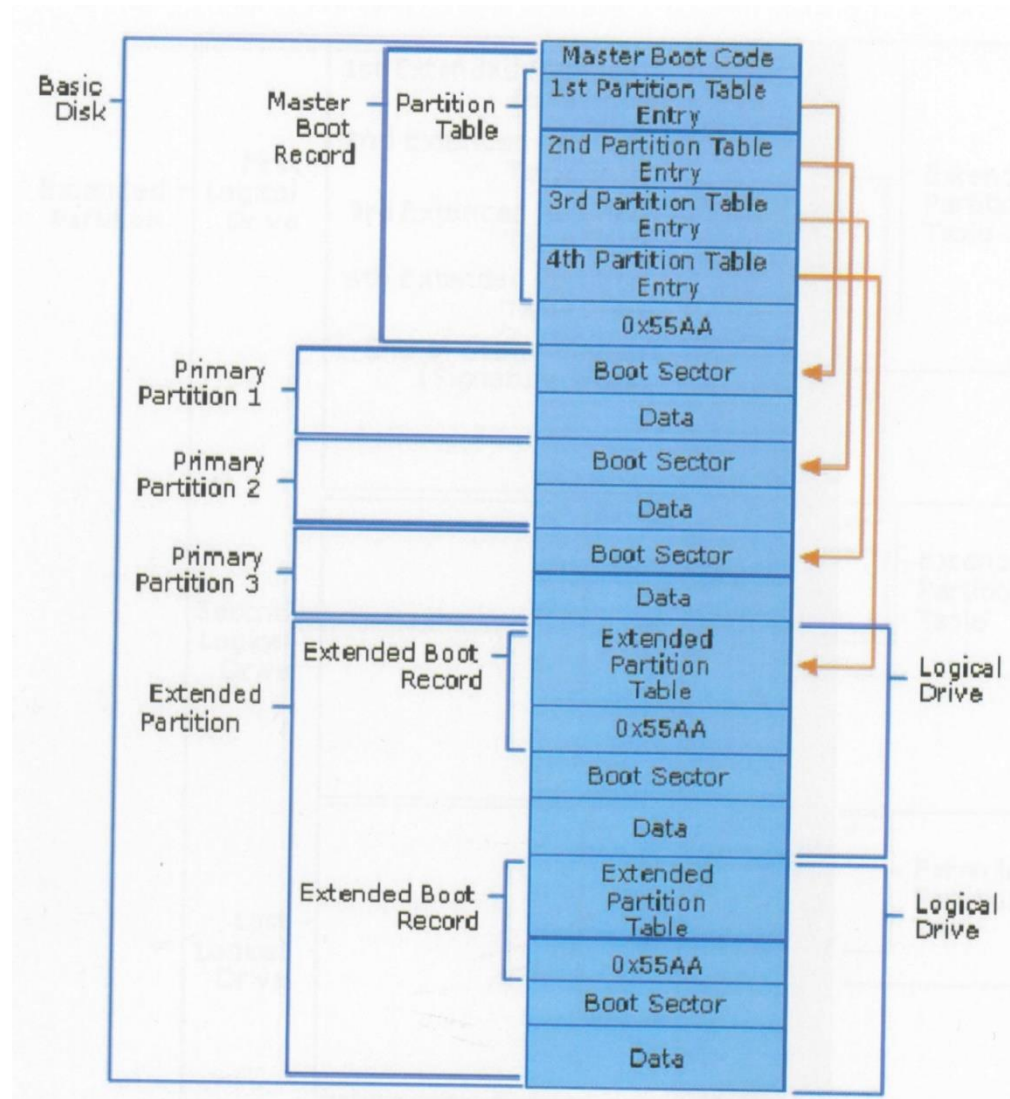


Logiczna struktura dysku MBR

- ◆ podstawowe informacje o strukturze dysku w systemach z BIOS, w tym opisy położenia partycji przechowywane są w rekordzie ładującym MBR (ang. Master Boot Record), który umieszczony jest na dysku w sektorze 1, cylinder 0, głowica 0;
- ◆ Rekord MBR zawiera krótki program, którego wykonanie rozpoczyna ładowanie właściwego systemu operacyjnego (program rozruchowy, bootloader, MBC, masterboot code). Jego rozmiar w tym miejscu jest ograniczony do 446 bajtów.
- ◆ informacje o utworzonych partycjach przechowywane są w 64-bajtowej *tablicy partycji*, która umieszczona jest pod adresem względnym 1BEH, zaraz po kodzie bootladera



Logiczna struktura dysku MBR (2)





Logiczna struktura dysku MBR (3)

- poniżej podano informacje o formacie dyskiety podane w MBR (odczytane za pomocą pakietu Norton Utilities):

OEM ID:	MSDOS5.0
Bytes per sector:	512
Sectors per cluster:	1
Reserved sectors at beginning:	1
FAT Copies:	2
Root directory entries:	224
Total sectors on disk:	2880
Media descriptor byte:	F0 Hex
Sectors per FAT:	9
Sectors per track:	18
Sides:	2
Special hidden sectors:	0
Big total number of sectors:	(Unused)
Physical drive number:	0
Extended Boot Record Signature:	29 Hex
Volume Serial Number:	421415D2 Hex
Volume Label:	ANALIZA2005
File System ID:	FAT12



Logiczna struktura dysku MBR (4)

- ◆ cztery ostatnie bajty obszaru poprzedzającego tablicę partycji zawierają podpis dysku (ang. nt-signature), który ma postać liczby 32-bitowej; w systemie Windows 2000 i następnych sygnatura jest używana jako indeks do odczytywania informacji o dyskach, zapisanych w rejestrze systemowym w podkluczu

HKEY_LOCAL_MACHINE\SYSTEM\MountedDevices

- ◆ ostatnie dwa bajty rekordu MBR zajmuje kod AA55H, identyfikujący rekord MBR;
- ◆ pozostałe 62 sektory na ścieżce nr 0 zajmuje obszar EMBR (ang. Extended MBR), który nie jest używany w systemach Windows; często w obszarze tym umieszczany jest program używany do wyboru systemu operacyjnego (ang. bootmanager).



- w rekordzie MBR zawarty jest krótki program, którego zadaniem jest przygotowanie do uruchomienia systemu operacyjnego; MBC (ang. Master Boot Code) lub IPL (ang. Initial Program Loader);
- po zakończeniu wstępnego testowania komputera system BIOS odczytuje rekord MBR z dysku i zapisuje go w pamięci RAM w obszarze o adresie 0000:7C00H, i wykonuje skok pod ten adres
- uruchomiony program przegląda najpierw tablicę partycji (maksymalnie cztery partycje podstawowe), poszukując w niej partycji oznaczonej jako aktywna
- ze względu na podobieństwo do MBR, sektor ładujący musi zostać załadowany do obszaru pamięci o adresie 0000:7C00H, w którym aktualnie znajduje się rekord MBR;



Tablica partycji w MBR

Offset	Rozmiar	Opis
00H	1	znacznik aktywności partycji: 80H — partycja aktywna (ang. primary partition), 00H — zwykła partycja
01H	1	numer powierzchni (głowicy), od której rozpoczyna się partycja (CHS)
02H	2	numer cylindra i sektora początkowego (CHS)
04H	1	identyfikator systemu plików (ang. operating system indicator), np.: 0CH — system Windows z FAT32 (adresowanie LBA) 0FH — partycja rozszerzona (adresowanie LBA) 83H — Linux (system plików ext2)
05H	1	numer powierzchni (głowicy), w której kończy się partycja (CHS)
06H	2	numer cylindra i sektora końcowego (kodowane tak pole 02H) (CHS)
08H	4	względne położenie partycji (LBA)
0CH	4	liczba sektorów partycji

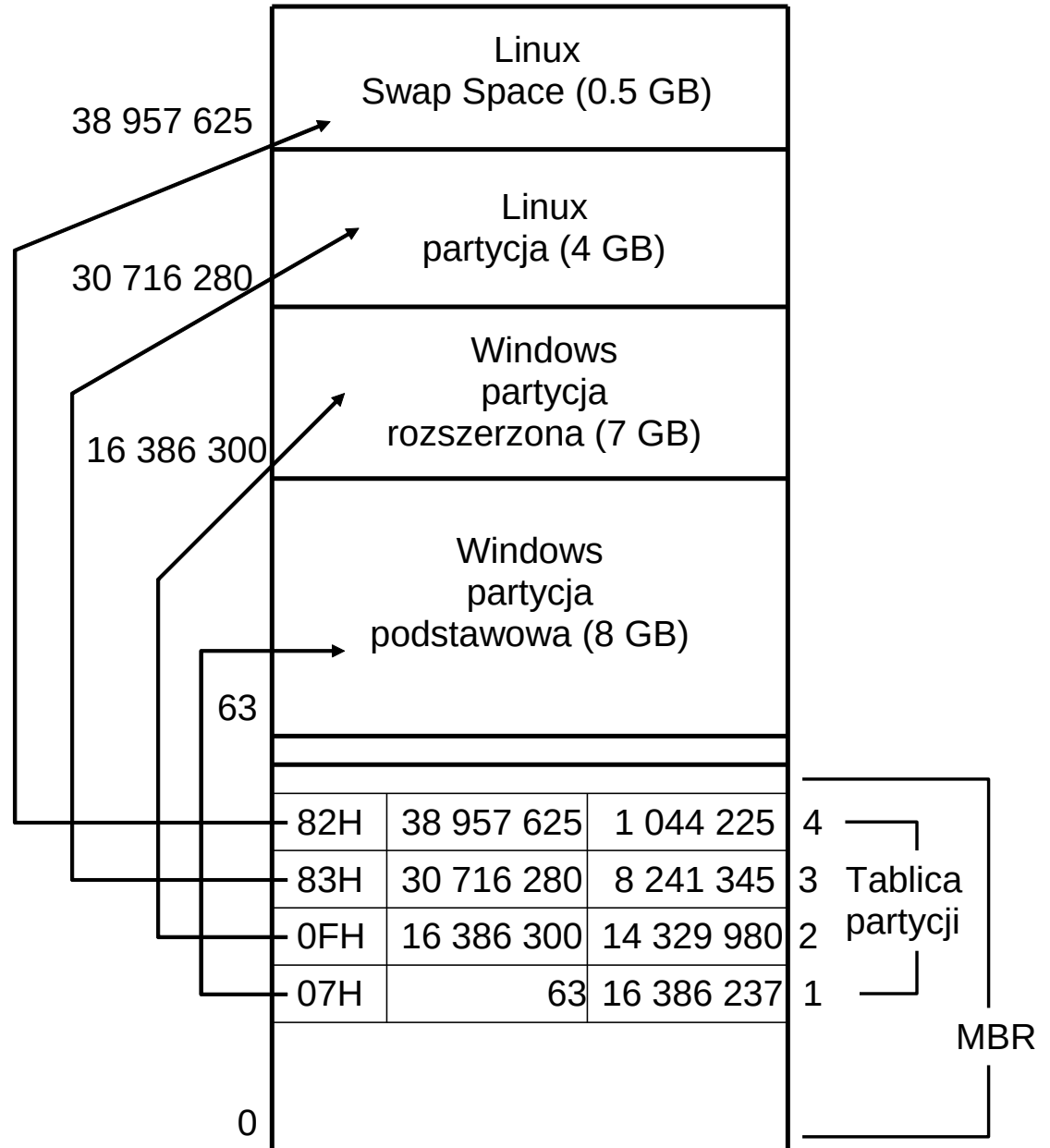


- szczegółowe informacje o partycjach, powiązane z konkretnym systemem plików, zawarte są w *sektorach ładujących partycji* (boot sector); adres partycji podstawowej lub też adresy dysków logicznych w partycji rozszerzonej (zob. opis podany dalej) wskazują na sektory ładujące;
- zazwyczaj na początku sektora ładującego umieszczony jest rozkaz skoku do sekwencji rozkazów inicjującej ładowanie właściwego systemu operacyjnego; na kolejnych 8 bajtach podana jest nazwa systemu plików, np. NTFS; dalej następują dwa bloki parametrów, oznaczone skrótami BPB (ang. BIOS Parameter Block) i EBPB (ang. Extended BPB); w blokach tych podane są różne parametry, jak np. liczba bajtów w sektorze, liczba sektorów na ścieżce, etykieta dysku (11 znaków ASCII) i inne



Partycje rozszerzone

- ♦ pojedyncza tablica partycji może pomieścić informacje tylko o czterech dyskach logicznych; w celu zwiększenia liczby dysków logicznych zdefiniowano tzw. *partycję rozszerzoną* (ang. extended), w skład której może wchodzić wiele dysków (woluminów) logicznych; dyski logiczne widziane są przez użytkownika jako zwykłe dyski; partycje rozszerzone stosowane są w systemie Windows (kod 0FH);
- na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze, w którym zawarta jest tablica partycji, położona w tym samym miejscu (offset) co w przypadku MBR; w tablicy tej wykorzystywane są tylko dwie pierwsze pozycje;
- pierwszy element omawianej tu tablicy partycji wskazuje adres początkowy dysku logicznego (wchodzącego w skład partycji rozszerzonej); z kolei drugi element tablicy partycji wskazuje na sektor dyskowy o takiej samej strukturze, stanowiący kolejny element listy dysków logicznych; element zawierający wartość 0 w polu identyfikatora systemu operacyjnego wskazuje koniec listy; pole o offsecie 08H (względne położenie partycji) określa w tym przypadku odległość partycji od początku partycji rozszerzonej);





POLITECHNIKA
GDAŃSKA

26 619 768

Dysk logiczny
F: (2 GB)

	0	0	4
	0	0	3
	0	0	2
07H	63	4 096 512	1

26 619 705

Dysk logiczny
E: (4 GB)

	0	0	4
	0	0	3
05H	10 233 405	4 096 575	2
07H	63	8 193 087	1

18 426 555

Dysk logiczny
D: (1 GB)

	0	0	4
	0	0	3
05H	2 040 255	8 193 150	2
0BH	63	2 040 192	1

16 386 300



- EFI definiuje GPT (*GUID Partition Table*), nowy schemat partycjonowania, który musi być obsługiwany przez firmware EFI.
- GPT wykorzystuje globalnie unikatowe identyfikatory aby znakować partycje. Jest również wykorzystywany w 64-bitowej wersji systemu Windows i posiada wiele cech czyniących ten schemat lepszym niż MBR.
- GUID lub UUID (*Universally Unique Identifier*) jest określony jako 128-bitowa liczba typu long, która jest unikatowa w czasie (do 3400 roku n.e. na pojedynczy algorytm generujący) i przestrzeni.

Kluczem do generowania unikatowych kluczy bez potrzeby scentralizowanej instytucji jest globalnie unikatowa wartość dostępna każdemu takiemu generatorowi, 48-bitowy adres IEEE 802 (zwykle adres hosta). Dla komputera bez takiego adresu wartość ta jest generowana w sposób probabilistycznie unikatowy. Jednak to nie wystarcza do unikatowości.

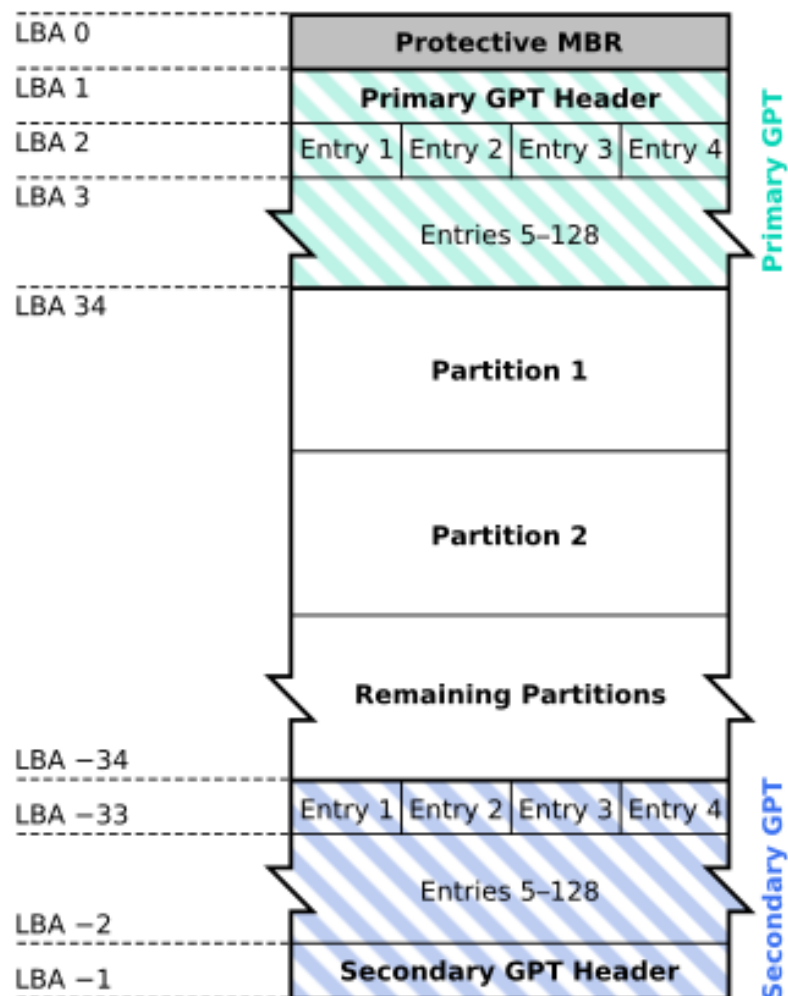
Inne wartości używane przez generatory GUID to np. stempel czasowy, sekwencja zegarowa i numer wersji.



- GPT wykorzystuje 64-bitowe LBA (partycja rzędu 8 EB – 2^{60})
- wspiera wiele partycji (nie trzeba tworzyć zagnieżdżonych struktur jak partycje „rozszerzone”),
- wykorzystuje pola version number/size dla przyszłej rozbudowy,
- wykorzystuje pola CRC32 dla zapewniania spójności danych,
- każda partycja zawiera 36-znakową czytelną dla człowieka nazwę (w Unicode),
- wykorzystuje GUID i atrybuty do określenia zawartości partycji (oznakowanie każdej partycji przy użyciu GUID ułatwia relokację),
- wykorzystuje podstawową i zapasową tablicę dla nadmiarowości.
- podstawowy nagłówek GPT jest zawarty w pierwszym bloku urządzenia logicznego, zapasowy zaś w jego ostatnim bloku. Nagłówek nie może być większy niż jeden blok urządzenia. Mimo, że ten schemat nie wspiera zagnieżdżonych partycji, można posiadać układ oparty na MBR zagnieżdżony wewnątrz partycji GPT, jednakże EFI nie wykona kodu rozruchowego zawartego w MBR.



GPT





Opis nagłówków GPT

Offset	Rozmiar w bajtach	Wartość w przykładzie	Nazwa pola i opis
0x00	8	45 46 49 20 50 41 52 54	Sygnatura. Używana do identyfikacji wszystkich nagłówków GPT. Wartość musi być zawsze równa 45 46 49 20 50 41 52 54.
0x08	4	00 00 01 00	Rewizja. Numer rewizji specyfikacji EFI, z którą jest zgodny nagłówek. Dla wersji 1.0 wartość jest równa 00 00 01 00.
0x0C	4	5C 00 00 00	Rozmiar nagłówka. Rozmiar nagłówka GPT wyrażony w bajtach. Rozmiar jest zawsze równy 92 bajty (0x5C). Pozostałe bajty w LBA 1 są zarezerwowane
0x10	4	27 6D 9F C9	Suma kontrolna CRC32. Wykorzystana do weryfikacji integralności nagłówka GPT.
0x14	4	00 00 00 00	Zarezerwowane. Musi być ustawione na 0.
0x18	8	01 00 00 00 00 00 00 00	Podstawowe LBA. Numer LBA, który zawiera podstawowy nagłówek GPT (zawsze 1).
0x20	8	37 C8 11 01 00 00 00 00	Zapasowe LBA. Adres LBA zawierającego zapasowy nagłówek GPT. Wartość jest zawsze równa ostatniemu LBA na dysku.
0x28	8	22 00 00 00 00 00 00 00	Numer pierwszego LBA do wykorzystania. Numer LBA, które może zawierać wpis z partycją GUID. Innymi słowy, pod tym LBA zaczyna się pierwsza partycja(zazwyczaj LBA34).
0x30	8	17 C8 11 01 00 00 00 00	Numer ostatniego LBA do wykorzystania.
0x38	16	00 A2 DA 98 9F 79 C0 01 A1 F4 04 62 2F D5 EC 6D	GUID Dysku. Unikalny numer, który pozwala na identyfikację nagłówka tablicy partycji i samego dysku
0x48	8	02 00 00 00 00 00 00 00	Numer pierwszego LBA dla wpisów w tablicy partycji. Ten numer zawsze jest równy LBA 2.
0x50	4	80 00 00 00	Liczba wpisów w tablicy partycji. Maksymalna liczba wpisów, które mogą istnieć w tablicy partycji GUID. W systemach 64 bitowych Windows jest to 128.
0x54	4	80 00 00 00	Rozmiar pojedynczego wpisu w tablicy partycji. Rozmiar wyrażony w bajtach. Każdy wpis zawiera 128 bajtów.
0x58	4	27 C3 F3 85	Suma kontrolna CRC32 wpisów tablicy partycji. Używana do weryfikacji integracji wpisów w tablicy partycji.
0x5C	420		Zarezerwowane. Musi być ustawione na 0.



Przykład nagłówka GPT

00000000:	45 46 49 20 50 41 52 54 - 00 00 01 00 5C 00 00 00	EFI PART....\...
00000010:	27 6D 9F C9 00 00 00 00 - 01 00 00 00 00 00 00 00	'm.....
00000020:	37 C8 11 01 00 00 00 00 - 22 00 00 00 00 00 00 00	7....."
00000030:	17 C8 11 01 00 00 00 00 - 00 A2 DA 98 9F 79 C0 01	y..
00000040:	A1 F4 04 62 2F D5 EC 6D - 02 00 00 00 00 00 00 00	...b/..m.....
00000050:	80 00 00 00 80 00 00 00 - 27 C3 F3 85 00 00 00 00	'
00000060:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	



Kody partycji w GPT

- C12A7328-F81F-11D2-BA4B-00A0C93EC93B – partycja EFI
- 5808C8AA-7E8F-42E0-85D2-E1E90434CFB3 – partycja na informacje o partycjach danych systemu Microsoft Windows
- AF9B60A0-1431-4F62-BC68-3311714A69AD – partycja danych w systemie Microsoft Windows



Wpis w tablicy partycji GPT

Offset	Rozmiar w bajtach	Przykładowa wartość	Opis pola
0x00	16	28 73 2A C1 1F F8 D2 11 BA 4B 00 A0 C9 3E C9 3B	GUID typu partycji
0x 10	16	C0 94 77 FC 43 86 C0 01 92 E0 3C 77 2E 43 AC 40	Unikalny identyfikator partycji (GUID)
0x 20	8	3F 00 00 00 00 00 00 00	Startowe LBA.
0x 28	8	CC 2F 03 00 00 00 00 00	Końcowe LBA.
0x 30	8	00 00 00 00 00 00 00 00	Bity atrybutów.
0x 38	72	EFI system partition	Nazwa partycji. (36-znakowy łańcuch Unicode).



Tablica partycji GPT

00000000:	28 73 2A C1 1F F8 D2 11 - BA 4B 00 A0 C9 3E C9 3B	(s*.....K...>;
00000010:	C0 94 77 FC 43 86 C0 01 - 92 E0 3C 77 2E 43 AC 40	..w.C.....<w.C.@
00000020:	3F 00 00 00 00 00 00 00 - CC 2F 03 00 00 00 00 00	?...../.....
00000030:	00 00 00 00 00 00 00 00 - 45 00 46 00 49 00 20 00	E.F.I. .
00000040:	73 00 79 00 73 00 74 00 - 65 00 6D 00 20 00 70 00	s.y.s.t.e.m. .p.
00000050:	61 00 72 00 74 00 69 00 - 74 00 69 00 6F 00 6E 00	a.r.t.i.t.i.o.n.
00000060:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	
00000070:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	
00000080:	16 E3 C9 E3 5C 0B B8 4D - 81 7D F9 2D F0 02 15 AE	\..M.}.-....
00000090:	80 BC 80 FC 43 86 C0 01 - 50 7B 9E 5F 80 78 F5 31	C...P{._.x.l
000000A0:	CD 2F 03 00 00 00 00 00 - D0 2A 04 00 00 00 00 00	./.....*.....
000000B0:	00 00 00 00 00 00 00 00 - 4D 00 69 00 63 00 72 00	M.i.c.r.
000000C0:	6F 00 73 00 6F 00 66 00 - 74 00 20 00 72 00 65 00	o.s.o.f.t. .r.e.
000000D0:	73 00 65 00 72 00 76 00 - 65 00 64 00 20 00 70 00	s.e.r.v.e.d. .p.
000000E0:	61 00 72 00 74 00 69 00 - 74 00 69 00 6F 00 6E 00	a.r.t.i.t.i.o.n.
000000F0:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	
00000100:	A2 A0 D0 EB E5 B9 33 44 - 87 C0 68 B6 B7 26 99 C7	3D..h..&..
00000110:	C0 1B 0B 00 44 86 C0 01 - F1 B3 12 71 4F 75 88 21	D.....qOu.!
00000120:	D1 2A 04 00 00 00 00 00 - 4E 2F 81 00 00 00 00 00	.*.....N/.....
00000130:	00 00 00 00 00 00 00 00 - 42 00 61 00 73 00 69 00	B.a.s.i.
00000140:	63 00 20 00 64 00 61 00 - 74 00 61 00 20 00 70 00	c. .d.a.t.a. .p.
00000150:	61 00 72 00 74 00 69 00 - 74 00 69 00 6F 00 6E 00	a.r.t.i.t.i.o.n.
00000160:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	
00000170:	00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00	



Systemy plików (File systems)

Tomasz Dziubich
dziubich@eti.pg.edu.pl



POLITECHNIKA
GDAŃSKA

Agenda

1. File Allocation Table - FAT
2. Extfs
3. NTFS



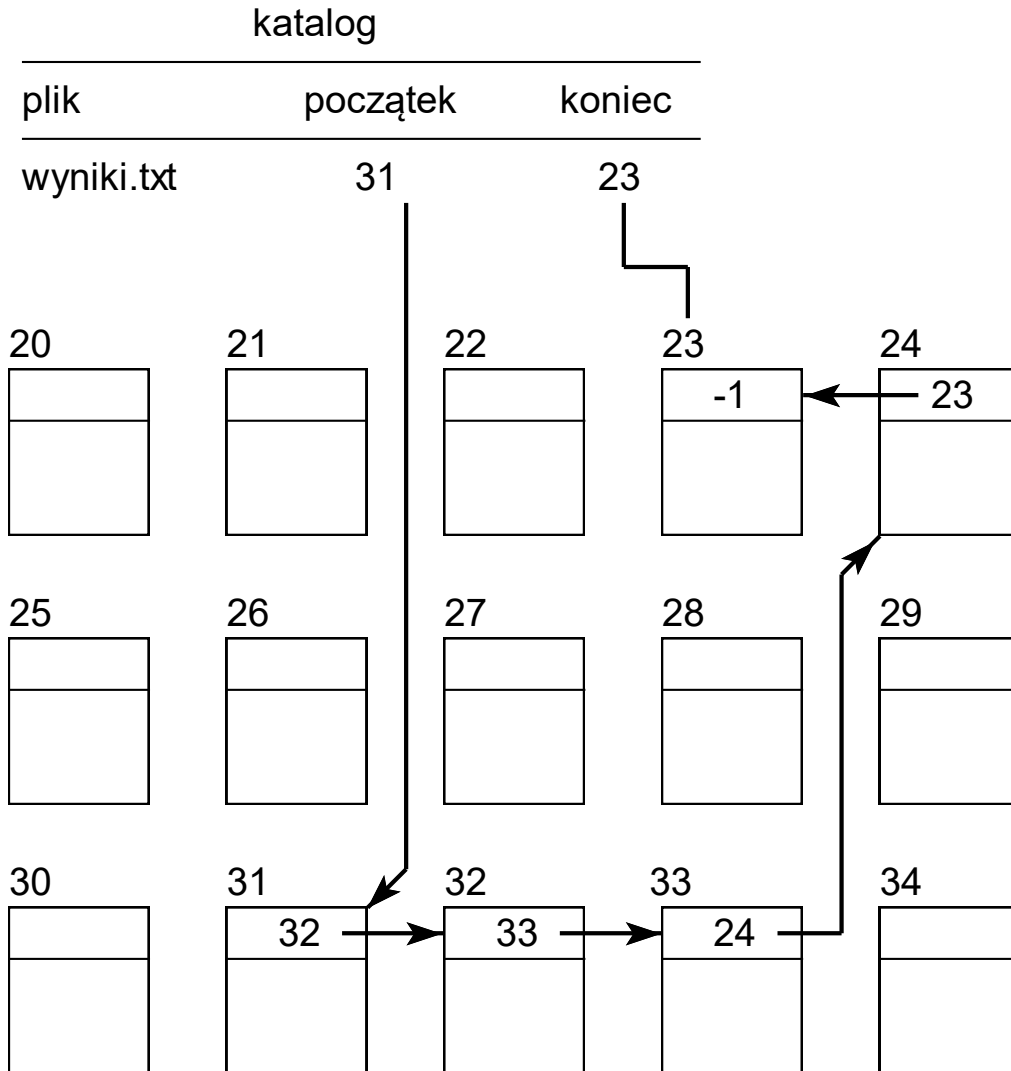
Rozmieszczenie plików na dysku

- ◆ używa się trzech metod przydziału miejsca na dysku:
 - ◆ ciągłej,
 - ◆ listowej,
 - ◆ indeksowej;

- ◆ w metodzie przydziału ciągłego każdy plik musi zajmować ciąg kolejnych bloków na dysku; opis pliku w katalogu plików zawiera informacje o położeniu pierwszego bloku na dysku oraz długość obszaru przydzielonego danemu plikowi;



Listowa metoda przydziału





Układ systemu plików FAT

FAT 16

Sektory zarezerwowane

- Sektor rozruchowy FAT 16 - BPB
 - Extended BPB

Tablica FAT #1

Tablica FAT #2

Katalog główny

Obszar z danymi

FAT 32

Sektory zarezerwowane

- Sektor rozruchowy FAT 32 - BPB
 - FAT 32 Extended boot sektor

Tablica FAT #1

Tablica FAT #2

Obszar z danymi



Field	Field Length	Hex offset	Description
BytesPerSector	WORD	0x000B	Bajtów na sektor
SectorsPerCluster	BYTE	0x000D	Sektorów na klaster
ReservedSectors	WORD	0x000E	Zarezerwowanych sek
FatCopies	BYTE	0x0010	Liczba kopii FAT
RootDirEntries	WORD	0x0011	Liczba wpisów w root
NumSectors	WORD	0x0013	Small Sectors
MediaType	BYTE	0x0015	Opis deskrytpora
SectorsPerFAT	WORD	0x0016	Sektorów na FAT
SectorsPerTrack	WORD	0x0018	Sektorów na ścieżkę
NumberOfHeads	WORD	0x001A	Liczba głowic
HiddenSectors	DWORD	0x001C	Ukryte sektory
SectorsBig	DWORD	0x0020	Large Sectors



Nazwa pola	Offset	Rozmiar
Nazwa pliku (krótka)	0	11
Atrybuty pliku	11	1
bajt zarezerwowany	12	1
pole dodatkowe precyzujące czas utworzenia pliku (następne pole) w postaci liczby jednostek 10 ms	13	1
Czas utworzenia pliku	14	2
Data utworzenia pliku	16	2
Data ostatniego dostępu do pliku	18	2
Nr początkowej pozycji w tablicy FAT32 (dwa starsze bajty)	20	2
Czas ostatniej modyfikacji pliku	22	2
Data ostatniej modyfikacji pliku	24	2
Nr początkowej pozycji w tablicy FAT32 (dwa młodsze bajty)	26	2
Rozmiar pliku	28	4



01 53 00 65 00 6D 00 69
00 6E 00 0F 00 14 61 00

72 00 69 00 75 00 6D 00
5F 00 00 00 31 00 39 00

53 45 4D 49 4E 41 7E 31
50 50 54 20 00 87 50 66

73 37 BB 3A 00 00 CC 64
73 37 C7 74 00 28 81 00

.S.e.m.i.n....a.r.i.u.m._...1.9.SEMINA
~1PPT .‡Pfs7»:...Ěds7Çt.(..

Sektor 511



Atrybuty plików

- bajt atrybutu określa własności pliku lub podkatalogu:
 - 01H tylko do odczytu
 - 02H plik ukryty
 - 04H plik systemowy
 - 08H etykieta dysku
 - 10H katalog
 - 20H znacznik archiwizacji
 - wartość 0FH oznacza, że pozycja katalogu zawiera część (dłuższej) nazwy pliku;



Nazwa pliku w FAT

- 00H — pozycja dotychczas w ogóle nie używana;
- E5H — opis pliku skasowanego (kasowanie pliku polega na wpisaniu kodu E5H do pierwszego bajtu);
- 05H — pierwszym znakiem nazwy pliku jest E5H (znak E5H jest dopuszczalnym znakiem w zestawach narodowych)
- 2EH — opis podkatalogu; jeśli następny bajt zawiera także kod 2EH (tj. '..'), to analizowana pozycja opisuje katalog nadrzędny w stosunku do bieżącego — w tym przypadku pole 'Nr początkowej pozycji ...' wskazuje na numer pozycji w FAT, zawierającej katalog nadrzędny; zero oznacza, że katalogiem nadrzędnym jest katalog główny;



Nazwy plików w FAT

- pozycje opisujące długie nazwy plików (ang. LFN – long file name) przechowywane są w dodatkowych rekordach 32-bajtowych przed rekordem opisującym plik
- w tym przypadku rekord opisujący plik zawiera nazwę pliku obciętą do sześciu pierwszych znaków, po których następuje znak tyldy ~ i kolejny numer porządkowy (w razie wystąpienia krótkiej tej samej nazwy pliku);
- charakterystycznym elementem rekordów dodatkowych jest kod 0FH umieszczony w polu atrybutu — identyfikuje on rekord zawierający długą nazwę pliku (lub jej część); ponieważ właściwe informacje o pliku przechowywane są w rekordzie podstawowym, więc w każdym rekordzie dodatkowym można umieścić 26 bajtów nazwy pliku — przy nazwach dłuższych stosowany jest łańcuch rekordów;
- nazwa kodowana jest w 16-bitowym kodzie Unicode; często jest to znak ASCII, po którym występuje bajt zerowy



Długie nazwy plików w FAT

Nazwa pola	Offset	Rozmiar
Nr rekordu (w sekwencji rekordów zawierających nazwę)	0	1
Pole nazwy pliku (5 znaków Unicode)	1	10
Atrybuty pliku — zawsze kod 0FH	11	1
(pole zarezerwowane)	12	1
Suma kontrolna rekordu podstawowego *)	13	1
Pole nazwy pliku (6 znaków Unicode)	14	12
Nr początk. pozycji w tablicy FAT (= 0000H)	26	2
Pole nazwy pliku (2 znaki Unicode)	28	4



- tablica rozmieszczenia plików FAT (ang. file allocation table) zawiera informacje o stanie wszystkich klasterów obszaru danych dysku; kolejne wiązki numerowane są: 2, 3, ...;
- struktura informacji w tablicy FAT:
 - pierwsze dwie pozycje są rezerwowane dla specjalnych informacji;
 - trzecia pozycja tablicy FAT zawiera informacje o pierwszym klasterze obszaru danych (tj. klasterze oznaczonym numerem 2);
 - następne pozycje tablicy FAT zawierają informacje o kolejnych klasterach;
 - każda pozycja w tablicy FAT używana przez plik wskazuje następną pozycję w FAT dla tego pliku — sekwencja numerów kolejnych pozycji FAT jest taka sama jak numery wiązek składających się na plik, a zatem opisane jest położenie pliku na dysku;



- stosuje się trzy metody kodowania elementów tablicy: 12-bitową dla dyskietek (omawianą w podanym dalej przykładzie) i 32-bitową dla dysków twardych (FAT32);
- kodowanie 16-bitowe nie jest obecnie używane;
- rozmiar klastra zależy zwykle od rozmiaru dysku; w systemie Windows XP dla FAT32 stosuje się klaster 4 kB dla dysków < 8 GB, 8 kB dla dysków < 1 GB i 16 kB dla dysków < 32 GB;



FAT - przykład

ANALIZA	FAT	0	1.12.16	14:11 pm	0	Vol
VGAPL	PAS	4561	2.10.14	8:36 am	2	
OS3	C	2339	3.12.16	13:49 pm	11	
CONFIG	SYS	200	3.12.16	13:51 pm	16	
DMA	WPG	3387	1.2.17	16:26 pm	17	

		(2) 3	4	5	6	7	8
(8) 9	10	EOF	12	13	14	15	EOF
(16) EOF	18	19	20	21	22	23	EOF
(24) 0	0	0	0	0	0	0	0
(32) 0	0	0	0	BAD	BAD	BAD	0



FAT - przykład

ANALIZA	FAT	0	12.96.16	14:11 pm	0	Vol
VGAPL	PAS	4561	10.91.14	8:36 am	2	
OS3	C	3231	12.96.16	13:49 pm	11	
CONFIG	SYS	200	12.96.16	13:51 pm	16	
DMA	WPG	3387	1.96.21	16:26 pm	17	

		(2) 3	4	5	6	7	8
(8) 9	10	EOF	12	13	14	15	24
(16) EOF	18	19	20	21	22	23	EOF
(24) 25	EOF	0	0	0	0	0	0
(32) 0	0	0	0	BAD	BAD	BAD	0



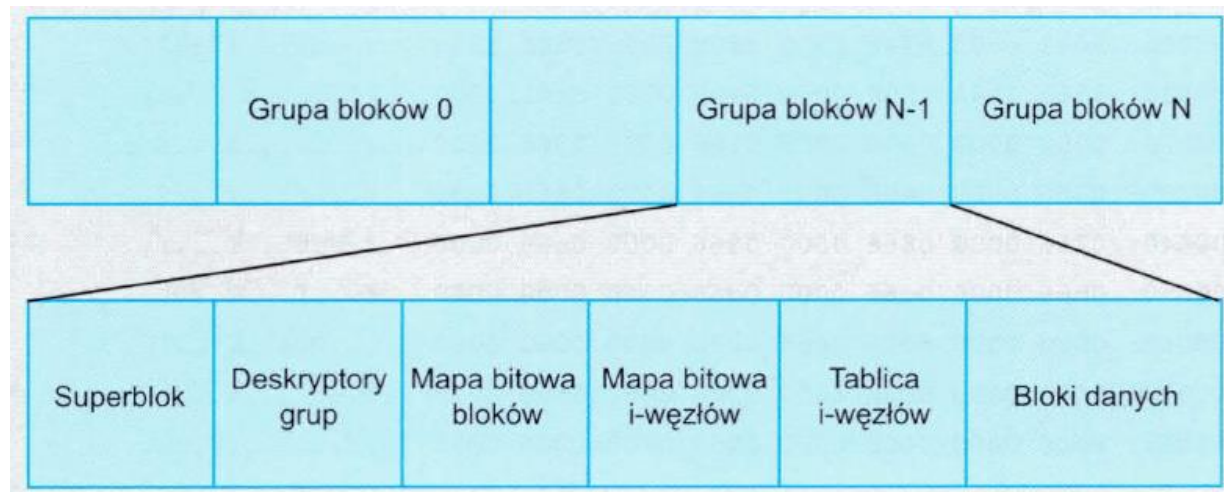
Ext – extended file system

- Bazuje na swoich poprzednikach – Minix (1980), ext (1992), ext2 (1993).
- Ext3 różni się od ext2 w zasadzie tylko elementem mechanizmu księgowania (journaling)
- Pozostałe elementy to synchroniczny zapis i mechanizm indeksowania katalogów
- Ext4 – istnienie od 12.2008 w jądrze 2.6.28 – powiększenie rozmiaru partycji 1 EB (Eksabajtów 10 do 18) i możliwość rezerwacji obszarów dla plików



Ext - metoda indeksowa dostępu do plików

- Partycja ext podzielona jest na bloki o stałym rozmiarze (4 kB).
- Bloki grupowane są w grupy bloków.
- Superblok – ilość wolnych bloków, i węzłów, rozmiar bloku, ilość montowań FS, itd..
- Deskryptor grupy – zbiór rekordów opisujących bloki należące do tej grupy





Ext - metoda indeksowa dostępu do plików

- w systemie Unix (Linux) podstawowe informacje o pliku przechowywane są w strukturze zwanej *i-węzłem* (ang. i-node);
- i-węzły przechowywane są w *tablicy i-węzłów*;
- każdemu otwartemu plikowi przyporządkowany jest dokładnie jeden i-węzeł;
- każdy i-węzeł zawiera:
 - opis położenia pliku na dysku i rozmiar pliku;
 - identyfikator właściciela pliku;
 - typ pliku (pliki zwykłe, katalogi, . . .);
 - prawa dostępu do pliku (właściciela pliku, grupowego właściciela pliku, pozostałych użytkowników);
 - czasy dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku);
 - liczba dowiązań do pliku;
 - liczba procesów, które otworzyły ten plik;
 - inne;
 - można zauważyć, że i-węzeł nie zawiera nazwy pliku !;



Ext - metoda indeksowa dostępu do plików

- zwykle i-węzły przechowywane są w dużej tablicy na początku partycji dyskowej (lub też są rozproszone wewnątrz partycji)
- w miarę otwierania plików są odczytywane i umieszczane w tablicy i-węzłów, która jest przechowywana w pamięci operacyjnej;
- system plików odwołuje się do i-węzłów poprzez podanie indeksu w tablicy i-węzłów;
- specjalną grupę plików stanowią katalogi (ang. directories) — na dysku są one przechowywane tak jak zwykłe pliki, m.in. z każdym katalogiem związany jest i-węzeł; z kolei dane zapisane w katalogu zawierają nazwy plików wraz z indeksami przyporządkowanych im i-węzłów;
- taka organizacja plików umożliwia przyporządkowanie każdemu plikowi więcej niż jednej nazwy — w katalogach plików mogą być umieszczone różne nazwy wskazujące na ten sam i-węzeł; każdą nazwę określa się jako *dowiązanie* (ang. link);

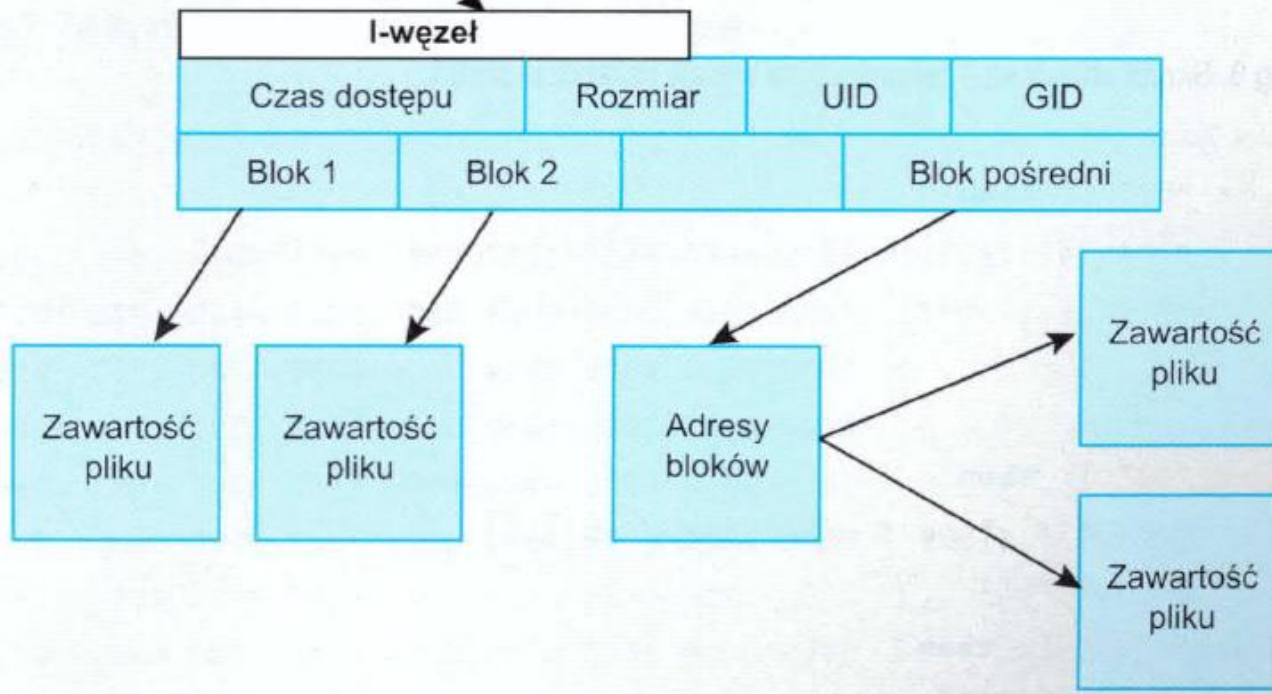


Ext - metoda indeksowa dostępu do plików

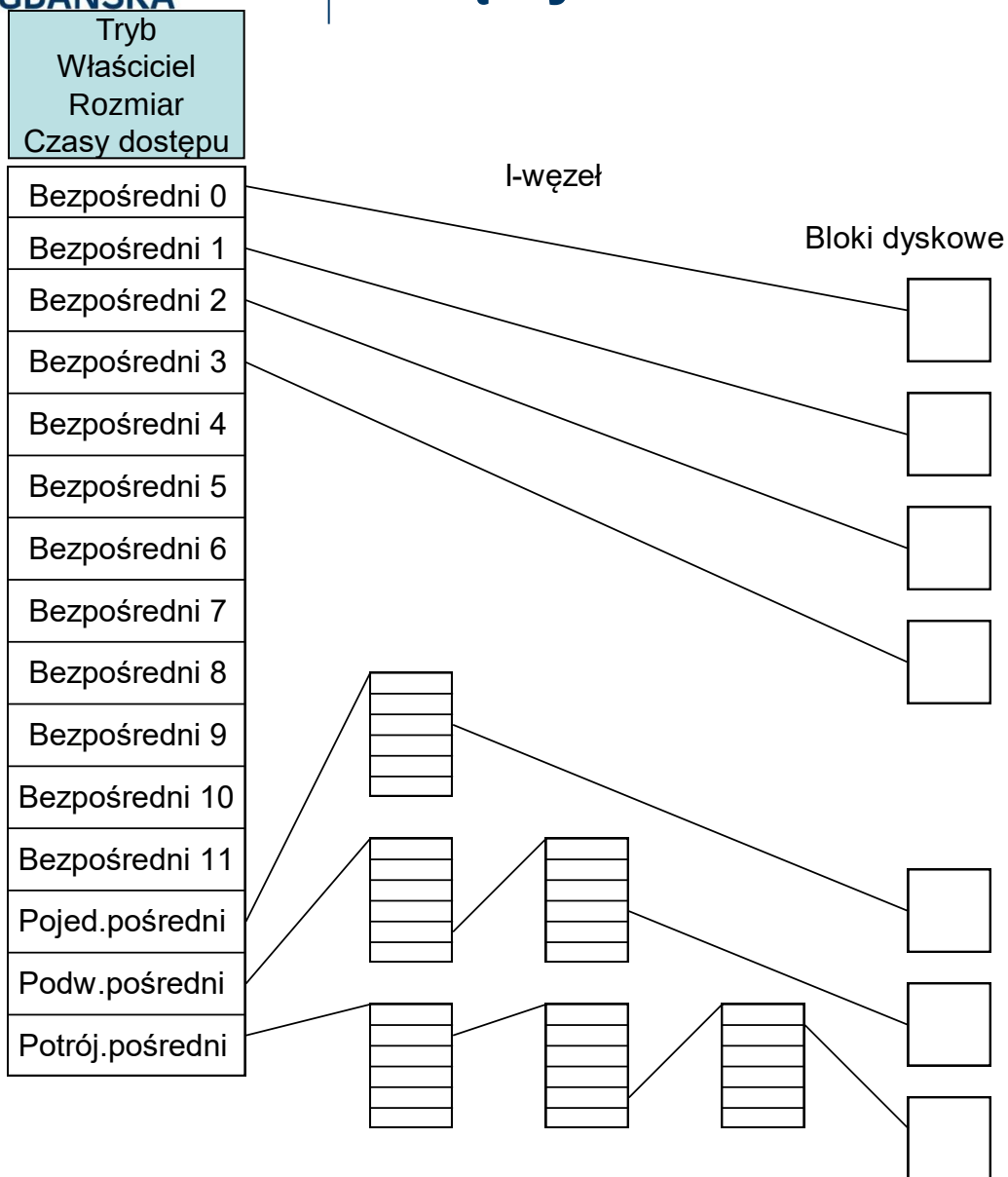
- pamięć na dysku podzielona jest na bloki, które są adresowane za pomocą numerów; rozmiar bloku wynosi zazwyczaj 1-8 KB
- i-węzeł zawiera tablicę adresów, za pomocą której można zlokalizować części pliku na dysku;
- proste odwzorowanie elementów tablicy adresów w bloki dyskowe powodowałoby konieczność tworzenia tablic o znacznych rozmiarach w przypadku dużych plików; ponadto byłoby korzystnie, gdyby rozmiar i-węzła nie zależał od rozmiaru pliku;
- biorąc pod uwagę podane postulaty w systemie Unix przyjęto, że tablica adresów bloków (oznaczona *i_block*) zawiera ustaloną liczbę pozycji, np. 15 w systemie Ext2, a każda pozycja zajmuje 4 bajty;
 - początkowe 12 pozycji omawianej tablicy zawiera adresy bezpośrednie, tj. wskazujące konkretne bloki — dzięki temu dostęp do plików o niewielkich rozmiarach odbywa się bardzo szybko; dalsze pozycje zawierają numery bloków: *pojedynczego pośredniego* (ang. indirect block), *podwójnego pośredniego* (ang. double indirect block), *potrójnego pośredniego* (ang. tripple indirect block);
 - dostęp do danych przez blok pośredni wymaga wczytania tego bloku, i znalezienia odpowiedniej pozycji z adresem bloku bezpośredniego;



I-Węzły



Zakres bajtów (offset)	Zawartość
04 – 07	Rozmiar pliku
40 – 87	Adresy bloków bezpośrednich
88 – 91	Adres pojedynczego bloku pośredniego
92 – 95	Adres podwójnego bloku pośredniego
96 – 99	Adres potrójnego bloku pośredniego





- Jeśli przyjąć, że w systemie stosowane są 4-kilobajtowe bloki dyskowe, a numer bloku zapisywany jest na 32 bitach, to maksymalny rozmiar pliku przekracza 4 TB:
 - 12 bloków bezpośrednich po 4 KB = 48 KB
 - 1 blok pośredni i 1024 bloki bezpośrednie = 4 MB
 - 1 blok podwójnie pośredni i 1024 pośrednie = 4 GB
 - 1 blok potrójnie pośredni i 1024 bloki podwójnie pośrednie = 4 TB



I-Węzły – przykład dostępu

- bajt 30000, to $30000/4096 = 7$ (reszta 1328),
 - potrzebny bajt znajduje się w bloku, którego numer podany jest na pozycji 7 tablicy adresów bloków;
 - wewnątrz bloku żądany bajt jest odległy o 1328 bajtów od początku bloku;
- bajt 5 000 000
 - wymaga użycia bloku podwójnie pośredniego;
 - pierwszy bajt dostępny poprzez blok podwójnie pośredni ma numer 48 KB + 4 MB = $49152 + 4\,194\,304 = 4\,243\,456$;
 - bajt o numerze 5 000 000 jest więc bajtem o numerze 756 544 względem początku bloku podwójnie pośredniego;
 - każdy blok pojedynczo pośredni daje dostęp do 4 MB, więc bajt o numerze 5 000 000 będzie się wskazywany pośrednio przez pozycję zerową bloku podwójnie pośredniego — po odczytaniu wyznaczonego w ten sposób bloku pośredniego poszukuje się w nim pozycji o indeksie $756\,544 / 4096 = 184$ (reszta 2880);
 - zawartość pozycji o indeksie 184 wskazuje na blok dyskowy zawierający żądany bajt — jest on odległy 2880 bajtów od początku bloku.



NTFS

New Technology File System

- system NTFS jest w pewnym stopniu podobny do FAT, m.in. opisy plików znajdują się w pojedynczej (dalej omawianej) *nadrzędnej tablicy plików* (Master File Table);
- w przeciwieństwie do FAT, NTFS pozwala na ograniczenie dostępu do plików i katalogów dla poszczególnych użytkowników;
 - poniżej wymieniono główne własności systemu NTFS:
 - możliwość określania praw dostępu do plików i katalogów,
 - prowadzenie dziennika transakcji, co pozwala na odtworzenie stanu systemu w przypadku uszkodzeń;
 - możliwość szyfrowania danych,
 - możliwość kompresji danych,
 - możliwość przydzielania przestrzeni na dysku indywidualnie dla każdego użytkownika;
 - możliwość obsługi bardzo dużych partycji (264 bajtów) i plików (16 Terabajtów);

http://technet.microsoft.com/en-us/library/cc781134%28WS.10%29.aspx#w2k3tr_ntfs_how_dhao



- w partycji NTFS wyróżnia się cztery obszary:
 - sektor ładujący,
 - nadrzędna tablica plików (ang. Master File Table),
 - pliki systemowe,
 - obszar danych;
- sektor ładujący ma strukturę podobną do używanej w FAT32:
- typowy rozmiar klastra dla woluminów o pojemności do 2 TB wynosi 4 kiB, jedynie dla woluminów o pojemności mniejszej od 2 GiB stosuje się mniejsze klastry;
- system NTFS posługuje się 64-bitowymi numerami klastrów nazywanymi LCN — logical cluster number — logiczny numer klastra;
- w szczególności klaster LCN 0 zawiera sektor ładujący; biorąc pod uwagę rozmiar klastra 4 kiB można zauważyć, że na poziomie współczesnej techniki rozmiar partycji jest praktycznie nieograniczony.



MFT - nadrzędna tablica plików

Plik 0	MFT
Plik 1	Fragment MFT (kopia)
Plik 2	Pliki systemowe (NTFS metadata files)
Plik 16	
	Pliki i katalogi użytkownika

Rekord plikowy

nagłówek
atrybut
atrybut
atrybut
znacznik końca rekordu plikowego:FFFFFFFFH



MFT - nadrzędna tablica plików

Nr	Nazwa pliku	Opis
0	\$MFT	nadrzędna tablica plików (spis wszystkich plików)
1	\$MFTMirr	kopia zapasowa fragmentu MFT
2	\$LogFile	dziennik transakcji
3	\$Volume	informacje o partycji (woluminie)
4	\$AttrDef	tabela nazw atrybutów, ich numerów i opisów
5	*)	katalog główny
6	\$Bitmap	mapa bitowa wszystkich klastrów dysku (wolny/zajęty)
7	\$Boot	sektor ładujący
8	\$BadClus	lista uszkodzonych klastrów
9	\$Secure	lista zabezpieczeń nałożonych na pliki na dysku
10	\$UpCase	tablica zamiany małych liter na duże (W standardzie Unicode)
11	\$Extend	rożne zastosowania
12		



- każdy plik opisany jest w tablicy MFT w postaci *rekordu plikowego* (1024 ÷ 4096 bajtów); rekord plikowy składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary (atrybut, wartość);
- charakterystycznym elementem *nagłówka rekordu plikowego* są znaki „FILE” (w kodzie ASCII) umieszczone na początku nagłówka;
- nagłówek zawiera kilkanaście pól o różnym przeznaczeniu, m.in. podany jest faktyczny rozmiar rekordu plikowego (4-bajtowe pole o offsecie 18H) oraz zaalokowany rozmiar rekordu plikowego (4-bajtowe pole o offsecie 1CH) — w omawianym dalej przykładzie rekordu plikowego pola te zawierają liczby 168H i 400H; tak więc przykładowy rekord plikowy zajmuje 1024 bajty



Atrybuty w rekordzie plikowym

- każdy atrybut składa się z nagłówka i właściwych danych (nazywanych również strumieniem); wyróżnia się cztery formaty nagłówka atrybutu, m.in. nagłówek atrybutu rezydentnego bez nazwy zawiera 24 bajty; we wszystkich formatach dwa pierwsze pola mają jednakowe znaczenie:
 - typ atrybutu — 4-bajtowe pole o offsecie 00H (np. 00000030H oznacza atrybut nazwy pliku);
 - długość (obejmująca nagłówek i dane) — 4-bajtowe pole o offsecie 04H;



Atrybuty w rekordzie plikowym

Typ atrybutu	Nazwa	Nr	Opis
Informacje standardowe	\$STANDARD_INFORMATION	10H	zawiera znaczniki czasu, liczba łączy, prawa dostępu.
Nazwa pliku	\$FILE_NAME	30H	zawiera nazwę pliku lub katalogu
Informacje o woluminie	\$VOLUME_INFORMATION	70H	nazwa (wcześniej: wersja) woluminu
Dane	\$DATA	80H	Dane pliku
Indeks główny	\$INDEX_ROOT	90H	Używany do implementowania katalogów i indeksów
Mapa bitowa	\$BITMAP	B0H	dane tego atrybutu zawierają pole bitowe

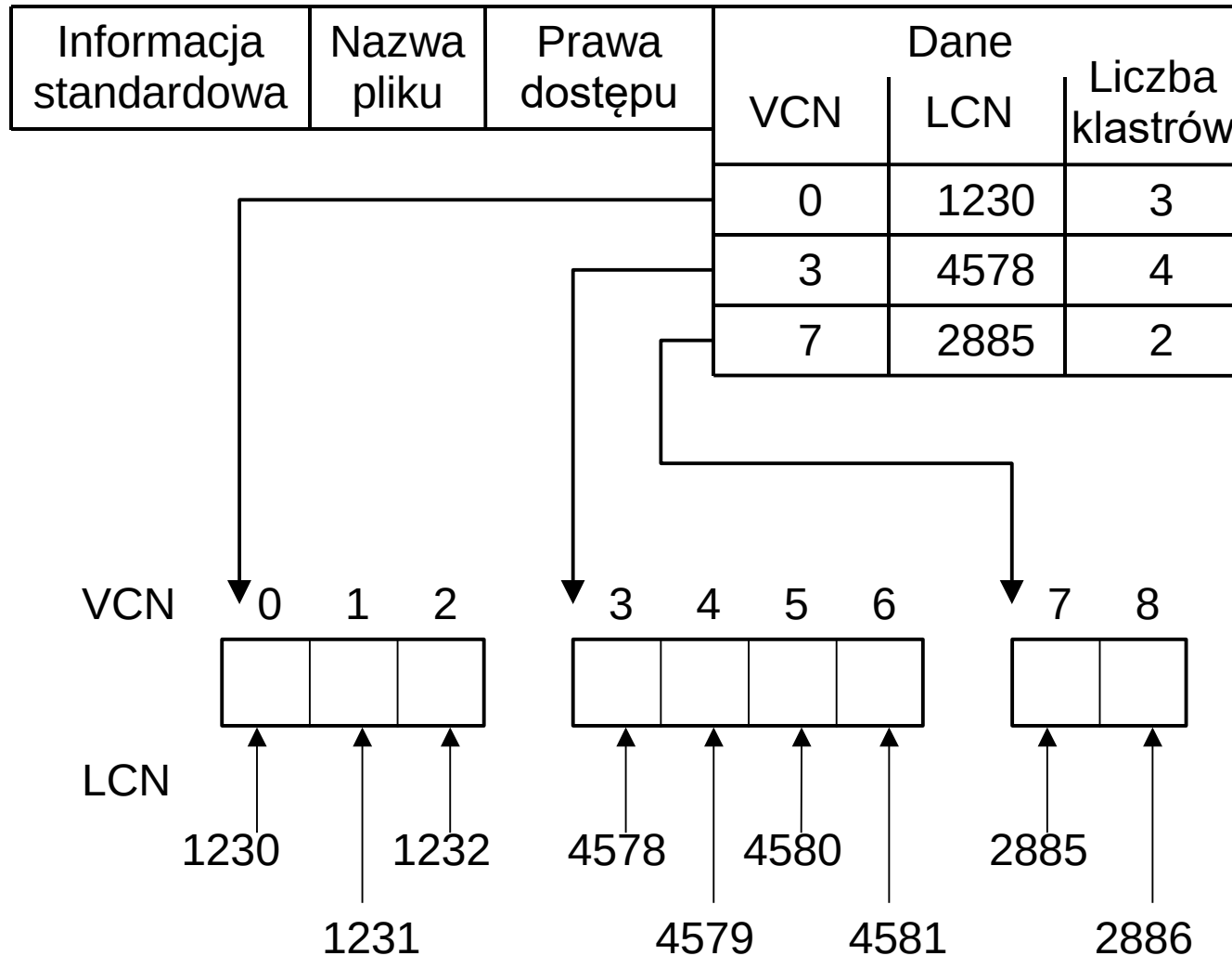


Atrybuty rezydentne i nierezydentne

- atrybuty dzieli się na dwie klasy:
 - rezydentne, których rozmiar nie przekracza wielkości rekordu plikowego, np. nazwa pliku oraz znaczniki czasu są zawsze rezydentne;
 - nierezydentne, które nie mieszczą się w jednym rekordzie i muszą być przechowywane poza tablicą MFT; w tym przypadku w miejscu, gdzie powinny się znajdować dane atrybutu, umieszczony jest ciąg wskaźników do poszczególnych części danych zwany *listą odwołań* (ang. runlist), która informuje o umiejscowieniu danych na dysku;



Atrybuty rezydentne i nierezzydentne





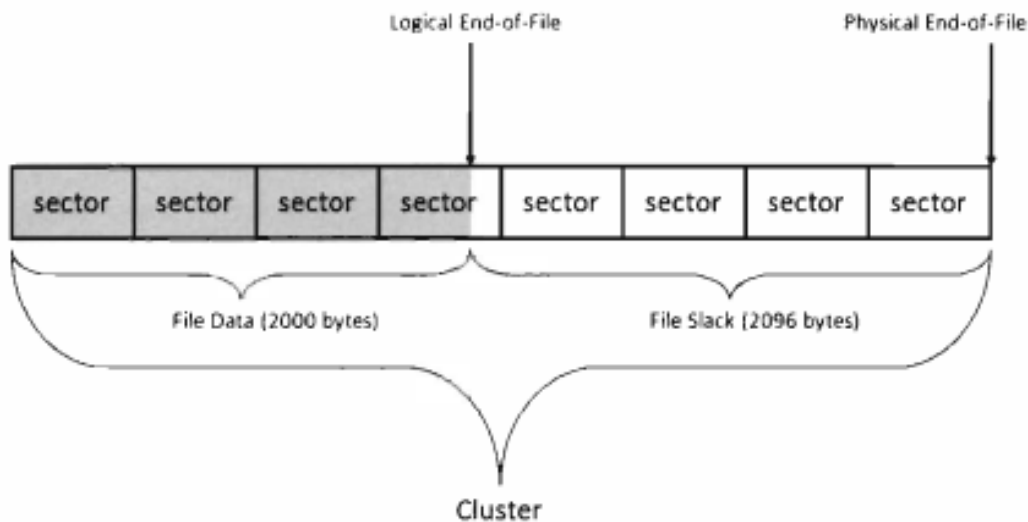
NTFS - dziennik transakcji

- kolejny rekord zawiera rejestr (log) operacji dyskowych — przechowywane są w nim informacje o wszystkich operacjach wykonywanych na plikach i katalogach (kopiowanie, kasowanie, przesunięcie do innego katalogu, itp.); prowadzenie tego rejestru umożliwia odtwarzalność danych;
- dziennik transakcji (kronikowanie – ang. *journaling*) jest prowadzony w celu zwiększeniu bezpieczeństwa danych dyskowych; gdy dowolny plik jest tworzony, modyfikowany lub usuwany, system NTFS dodaje odpowiedni rekord do dziennika transakcji (dziennika zmian); każdy rekord w dzienniku zmian zajmuje w przybliżeniu 80-100 bajtów; faktyczny zapis danych następuje po rejestracji w dzienniku; w przypadku niepowodzenia w zapisie (wskutek błędów transmisji lub uszkodzeń) transakcje są wycofywane z dysku;
- między innymi omawiany dziennik może być przydatny do tworzenia przyrostowych kopii zapasowych; używa się do tego celu odpowiednich funkcji API, które są udostępniane przez system.



Ukrywanie danych

- Out-of-band
- In-band (ASD)
- Ukrywanie w warstwie aplikacyjnej





Odwzorowywanie plików na pamięć liniową procesu - nmap

```
HANDLE          hPlik, hMap;
char * p;
DWORD rozmiar;
hPlik = CreateFile ("d:\\Builder\\teksty1.txt",
                   GENERIC_READ | GENERIC_WRITE,
                   0, NULL, OPEN_ALWAYS,
                   FILE_ATTRIBUTE_NORMAL, 0);

rozmiar = GetFileSize (hPlik, NULL);
hMap = CreateFileMapping( hPlik, NULL, PAGE_READWRITE, 0, 4096 , NULL);
p = (char *) MapViewOfFile (hMap, FILE_MAP_ALL_ACCESS, 0, 0, 0);
*(p+10) = '#';
*(p+40) = '#';
*(p+rozmiar-1) = '\\0';
FlushViewOfFile(p, rozmiar);
UnmapViewOfFile(p);
CloseHandle(hMap);
CloseHandle (hPlik);
```

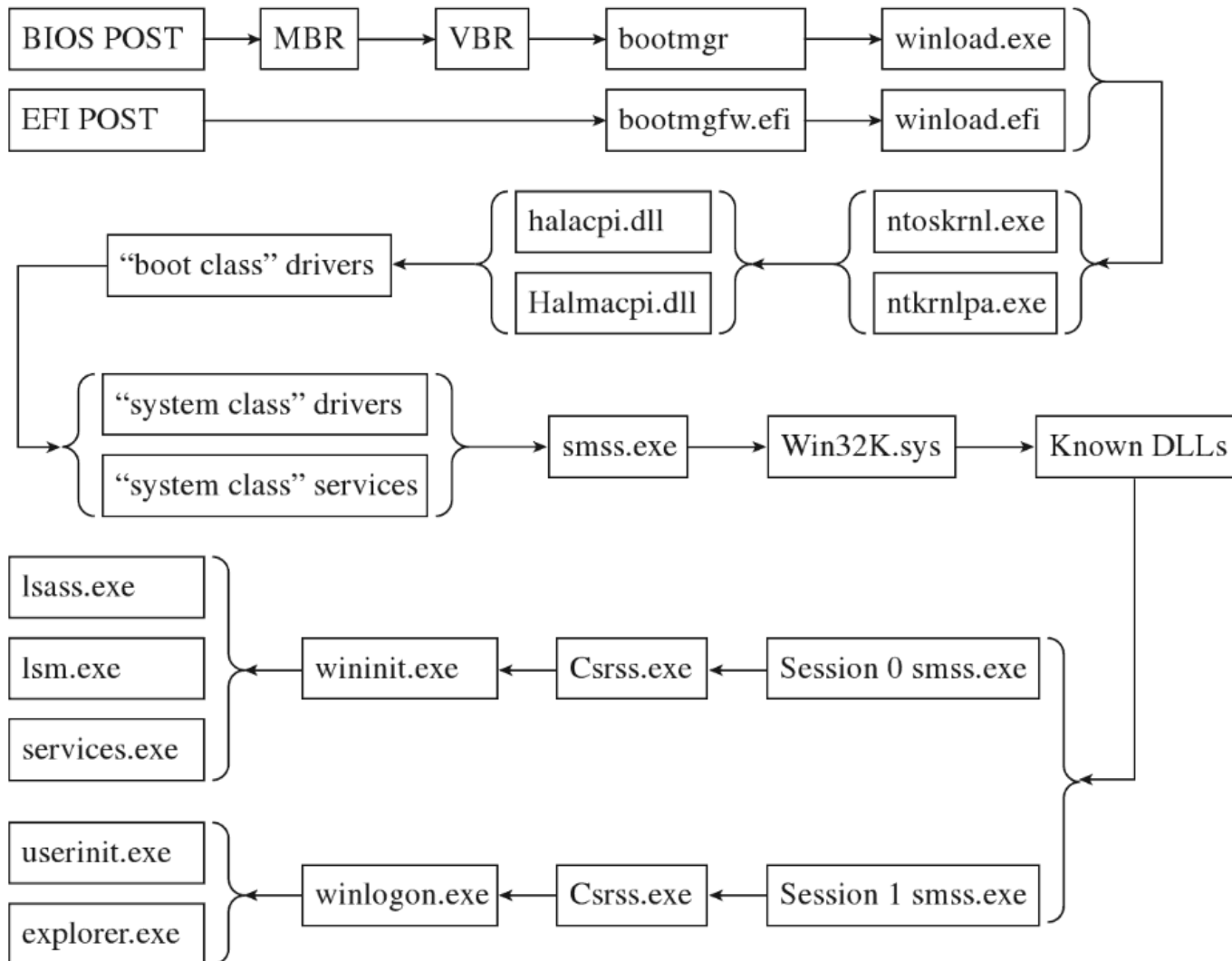


Formaty plików wykonywalnych (Executable file formats)

Tomasz Dziubich
dziubich@eti.pg.edu.pl



Krótkie przypomnienie





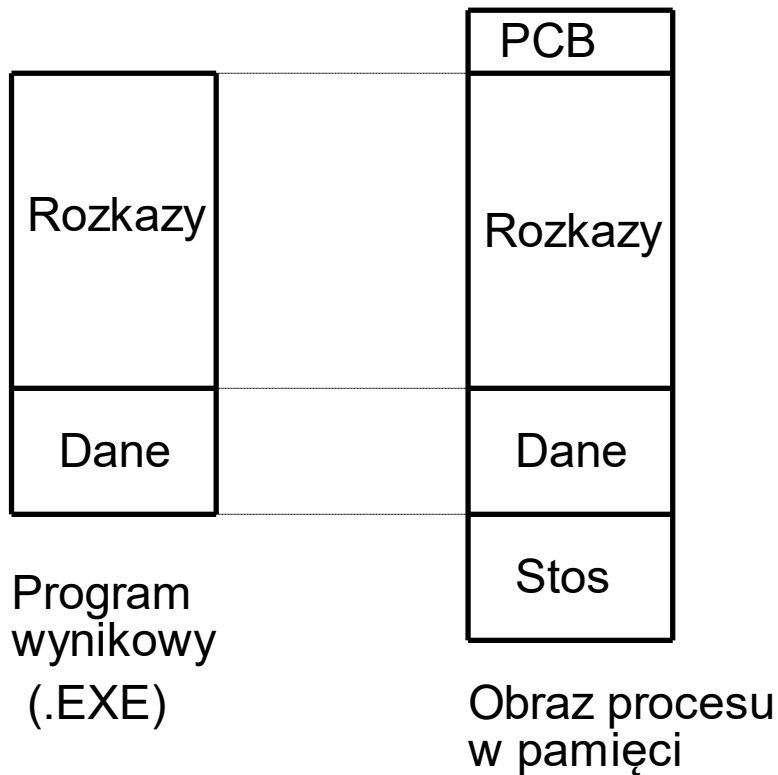
POLITECHNIKA
GDAŃSKA

Agenda

1. Obrazy
2. COM
3. a.out
4. DOS EXE
5. PE
6. ELF
7. Biblioteki dynamiczne



Ładowanie programów do pamięci



- ładowanie bezwzględne,
- ładowanie relokowalne,
- ładowanie dynamiczne.



Pliki zawierające kod i dane programu, zakodowane w sposób zrozumiały dla procesora to tzw. pliki obiektowe (*object files*, *.obj)

Plik obiektowy może być:

- linkowalny (*linkable*),
- wykonywalny (*executable*),
- ładowalny (*loadable*),
- mogą też występować kombinacje trzech ww. możliwości;

W systemie Windows pliki obiektowe są kodowane w formacie OMF (wcześniejsze wersje), COFF (aktualnie). W Linux COFF (wcześniej), aktualnie ELF;

OMF – Object Module Format

COFF – Common Object File Format

ELF – Executable and Linkable Format



Składowe struktury plików obiektowych

- nagłówek (ang. *header*) — zawiera ogólne informacje o pliku, data tworzenia, rozmiar kodu, itp.;
- sekcje — obszary w pliku zawierające bajty o tym samym znaczeniu,
- kod wykonywalny (ang. *object code*) — instrukcje i dane w postaci binarnej, wygenerowane przez kompilator lub assembler;
- opis relokacji — lista miejsc w kodzie wykonywalnym, które muszą być skorygowane;
- symbole — symbole globalne zdefiniowane w danym module, symbole importowane z innych modułów lub symbole definiowane przez linker;
- informacje dla debuggera — dodatkowe informacje potrzebne dla debuggera, jak np. symbole lokalne, numery linii programu źródłowego, opisy struktur danych, itp.

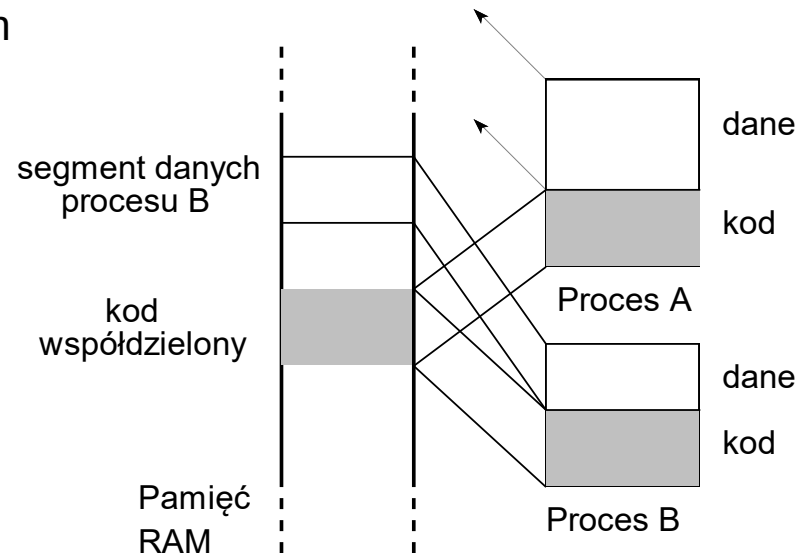


- bezpośrednio przed rozpoczęciem wykonywania programu system operacyjny ładuje plik do wolnego obszaru pamięci
- początkowe 256 bajtów tego obszaru zajmuje blok PCB, tu nazywany przedrostkiem segmentu programu (ang. PSP — *program segment prefix*)
- zawartość pliku wpisywana jest do obszaru począwszy od offsetu 100H
- rejestry segmentowe są ustawiane tak, że wskazują początek bloku PSP, rejestr SP wskazuje koniec segmentu, po czym następuje skok do początku programu;



Format a.out

```
int a_magic;           // magic number
int a_text;            // rozmiar segmentu kodu (text segment)
int a_data;            // rozmiar segmentu danych inicjalizowanych
                        // (initialized data)
int a_bss;             // rozmiar segmentu danych nieinicjalizowanych
                        // (uninitialized data)
int a_syms;            // rozmiar tablicy symboli (symbol table)
int a_entry;           // punkt wejścia
int a_trsize;          // rozmiar relokacji kodu (text relocation size)
int a_drsize;          // rozmiar tablicy relokacji danych
                        // (data relocation size)
```





DOS EXE

```
char signature[2] = "MZ"; // identyfikator formatu (magic number)
short lastsize;           // długość pliku mod 512
short nblocks;            // liczba bloków 512 zajmowanych przez plik
short nreloc;             // liczba pozycji w tablicy modyfikacji obrazu programu
short hdrsize;            // rozmiar nagłówek w (16-bajtowych) paragrafach
short minalloc;           // minimalna liczba przydzielanych dodatkowych paragrafów
short maxalloc;           // maksymalna liczba przydzielanych dodatkowych
                           // paragrafów

void far * sp;            // zawartość początkowa rejestrów SS:SP
short checksum;           // suma kontrolna
void far * ip;            // zawartość początkowa rejestrów CS:IP
short relocpos;           // położenie tablicy modyfikacji obrazu
                           // programu (ang. location of relocation
                           // fixup table)

short noverlay;           // liczba nakładek programu
char extra[ ];            // dodatkowy obszar dla nakładek, itp.
void far * relocs[ ];     // kolejne pozycje tablicy relokacji
```



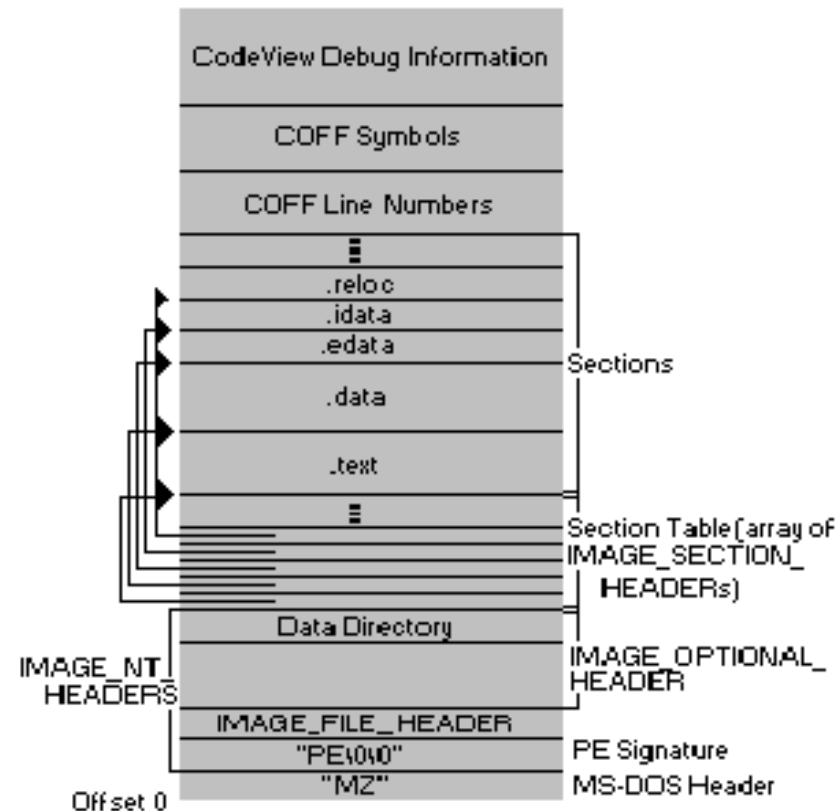
DOS EXE (2)

		0000H	4D	5A	31	00	02	00	01	00
			20	00	29	00	29	00	04	00
rozказы	SEGMENT	0010H	80	02	50	8C	00	00	00	00
	assume cs:rozказы		1E	00	00	00	01	00	04	00
pocz:	nop	0020H	00	00	00	00	00	00	00	00
	mov ah, 09H ; wyświetlanie łańcucha		00	00	00	00	00	00	00	00
	mov dx, SEG znaki ; znaków		00	00	00	00	00	00	00	00
	mov ds, dx		00	00	00	00	00	00	00	00
	mov dx, OFFSET znaki		---	---	---	---	---	---	---	---
	int 21H	01F0H	00	00	00	00	00	00	00	00
	mov ax, 4C00h		00	00	00	00	00	00	00	00
	int 21H	0200H	90	B4	09	BA	00	00	8E	DA
znaki	db 'Testowanie .EXE', 0DH, 0AH, '\$'		BA	11	00	CD	21	B4	4C	CD
rozказы	ENDS	0210H	21	54	65	73	74	6F	77	61
stos_pr	SEGMENT stack		6E	69	65	20	70	72	6F	67
	dw 320 dup (?)	0220H	72	61	6D	75	20	74	79	70
stos_pr	ENDS		75	20	2E	45	58	45	0D	0A
	END pocz	0230H	24							



Format PE

- DOS Stub + Sygnatura
 - Sygnatura PE
 - Wykonywalny program DOS
- IMAGE_NT_SIGNATURE
(0x00004550)
- Nagłówek pliku (COFF)
- Opcjonalny nagłówek (PE Header)
- Katalogi danych
 - Ulokowane w statycznych offsetach
 - Wskazują na specyficzne struktury
 - Imports, Exports, etc
- Nagłówki sekcji
- Sekcje





Format PE – nagłówek PE

```
char signature[4] = "PE\0\0";    // identyfikator (magic number)
                                   // określa także porządek bajtów

// nagłówek formatu COFF
unsigned short Machine;           // typ procesora:
                                   // 0x14C for 80386, itd.

unsigned short NumberOfSections;  // liczba sekcji
unsigned long  TimeDateStamp;     // czas utworzenia lub zero
unsigned long  PointerToSymbolTable; // offset tablicy
                                   // symboli (COFF) lub zero

unsigned long  NumberOfSymbols;   // liczba pozycji
                                   // w tablicy symboli

unsigned short SizeOfOptionalHeader; // rozmiar
                                   // nagłówka opcjonalnego

unsigned short Characteristics;   // 02 = executable,
                                   // 0x200=nonrelocatable,
                                   // 0x2000 = DLL
```



Opcjonalny nagłówek PE

```
typedef struct _OPTHEADERS{  
    WORD    Magic;  
    BYTE    MajorLinkerVersion;  
    BYTE    MinorLinkerVersion;  
    DWORD    SizeOfCode;           // code segment size  
    DWORD    SizeOfInitializedData; // data segment size  
    DWORD    SizeofUninitializedData; // data segment size  
    DWORD    AddressOfEntryPoint; // entry point  
    DWORD    BaseOfCode;  
    DWORD    BaseOfData;  
    DWORD    ImageBase;  
    DWORD    SectionAlignment;  
    DWORD    FileAlignment;  
    WORD    MajorOperatingSystemVersion;  
    WORD    MinorOperatingSystemVersion;  
    WORD    MajorSubsystemVersion;  
    WORD    MinorSubsystemVersion;  
    DWORD    Reserved;  
    DWORD    SizeOfImage;  
    DWORD    SizeOfHeaders;  
    DWORD    CheckSum;  
    WORD    Subsystem;  
    DWORD    DllCharacteristics;  
    DWORD    SizeOfStackReserve;  
    DWORD    SizeOfStackCommit;  
    DWORD    SizeOfHeapReserve;  
    DWORD    SizeOfHeapCommit;  
    DWORD    LoaderFlags;  
    DWORD    NumberOfRvaAndSizes; // data directories  
}OPTHEADERS, *POPTHEADERS;
```



- Wpisy sekcji

- Wyliczanie Adresu:

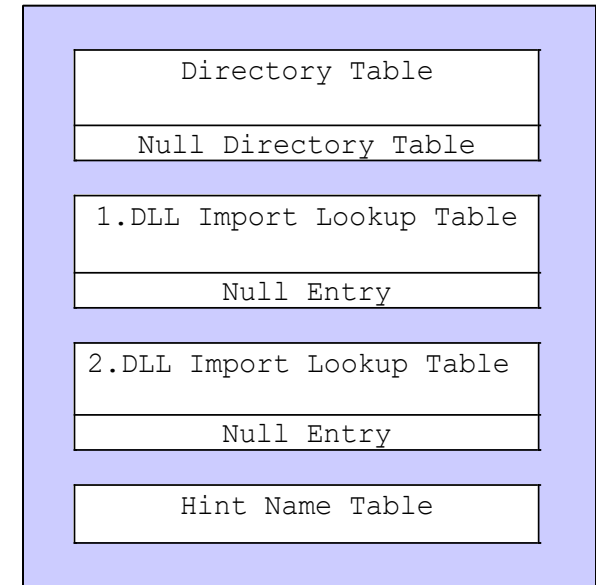
```
base_addr + *(uint32)(base_addr + 0x3c)
+ sizeof(COFF) + PCOFF->SizeOfOptionalHeader
```

- Wpisy z relokacjami są obecne tylko w pliku object

```
typedef struct _SECTIONTABLES {
    BYTE  Name[8];           // Section name
    DWORD VirtualSize;       // Size of section in memory
    DWORD VirtualAddress;    // Address of mapped section
    DWORD SizeOfRawData;     // Size of section on disk
    DWORD PointerToRawData;  // Section file offset
    DWORD PointerToRelocations; // Relocation entries file offset
    DWORD PointerToLineNumbers; // Line-number entries file offset
    WORD  NumberOfRelocations; // Number of relocation entries
    WORD  NumberOfLineNumbers; // Number of line-number entries
    DWORD Characteristics;   // Characteristics flags
}SECTIONTABLES, *PSECTIONTABLES;
```



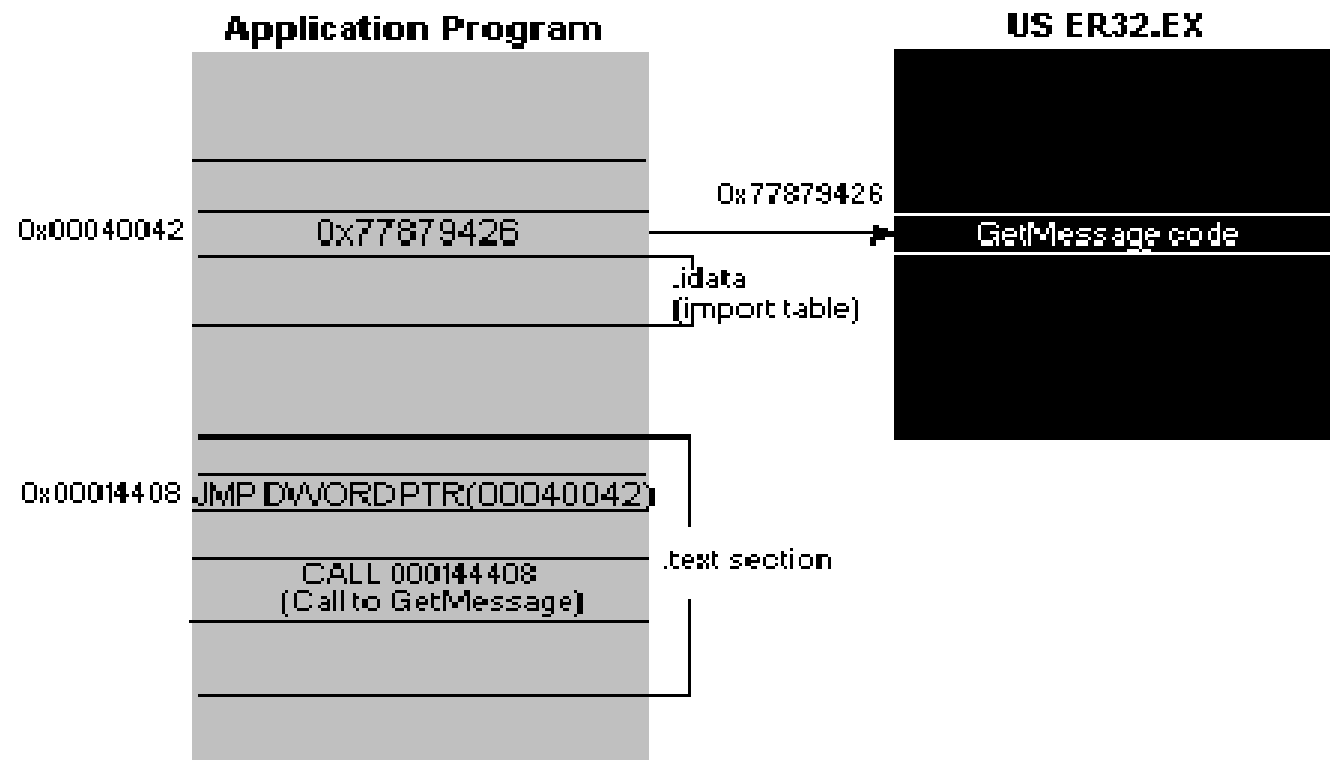
- Symbole PE
 - Data_Directory[1] – importowane katalogi
 - .idata section
 - Importowane encje z opisanych bibliotek dynamicznych
 - DLL Name
 - RVA of Import Lookup Table
 - RVA of Import Address Table



```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
        DWORD Characteristics;
        PIMAGE_THUNK_DATA OriginalFirstThunk;
    } DUMMYUNIONNAME;
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    PIMAGE_THUNK_DATA FirstThunk;
} IMAGE_IMPORT_DESCRIPTOR,*PIMAGE_IMPORT_DESCRIPTOR;
```

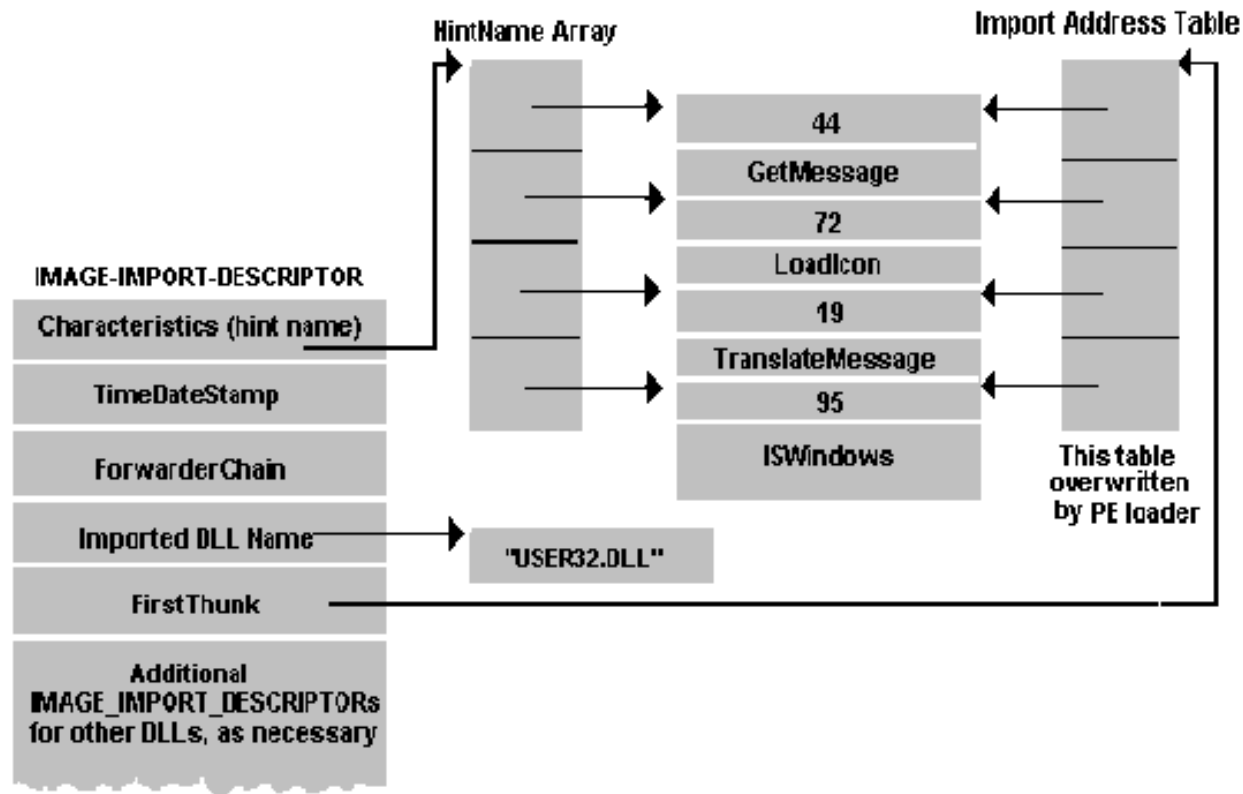


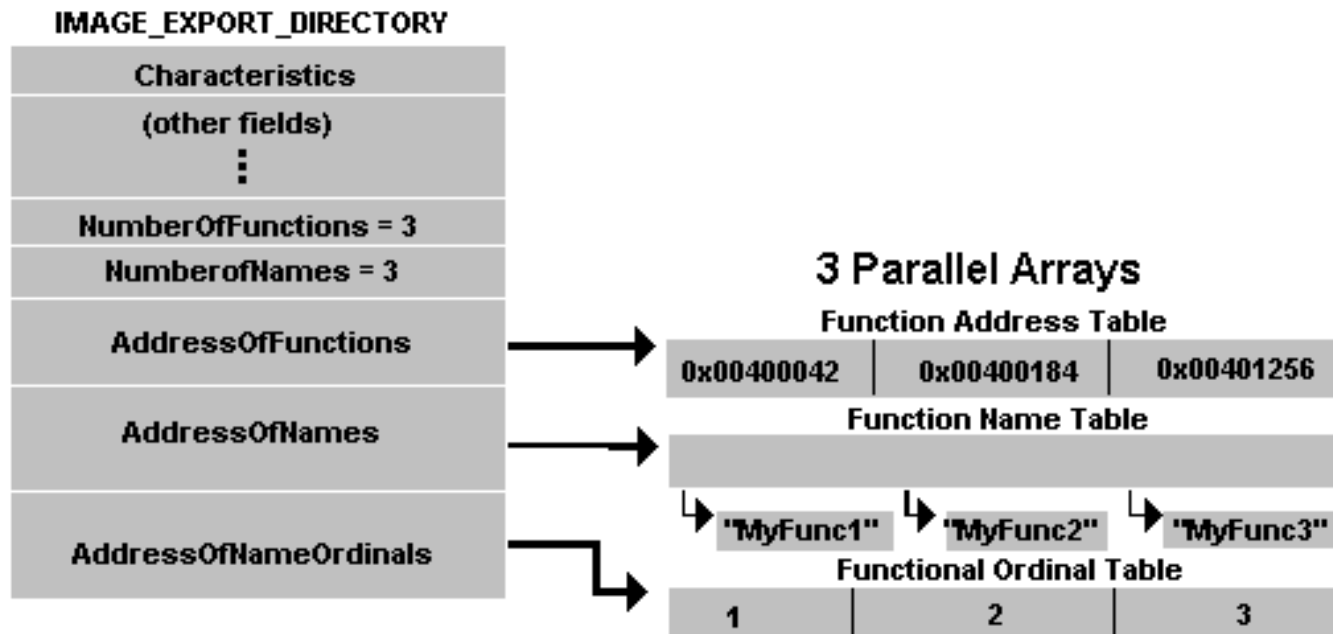
Wywołanie funkcji z innej biblioteki





Tablica importu

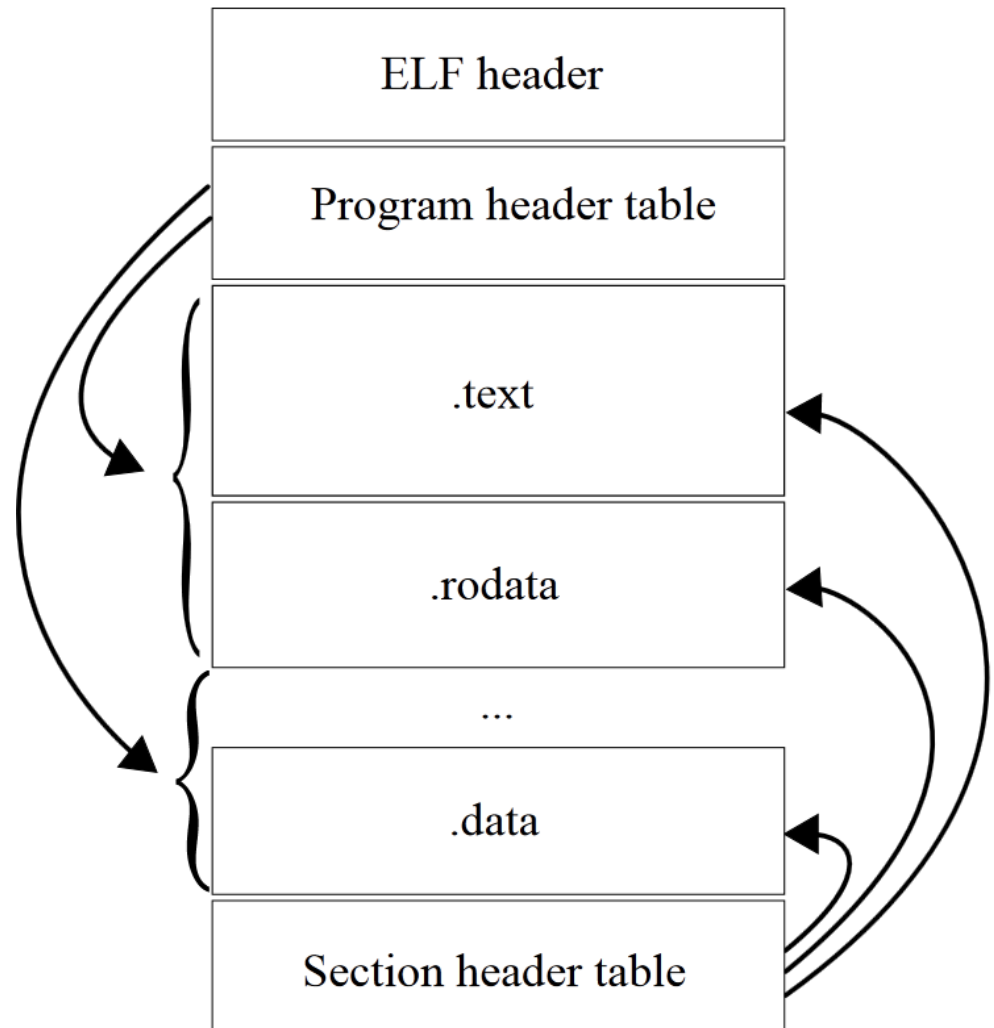






Format ELF

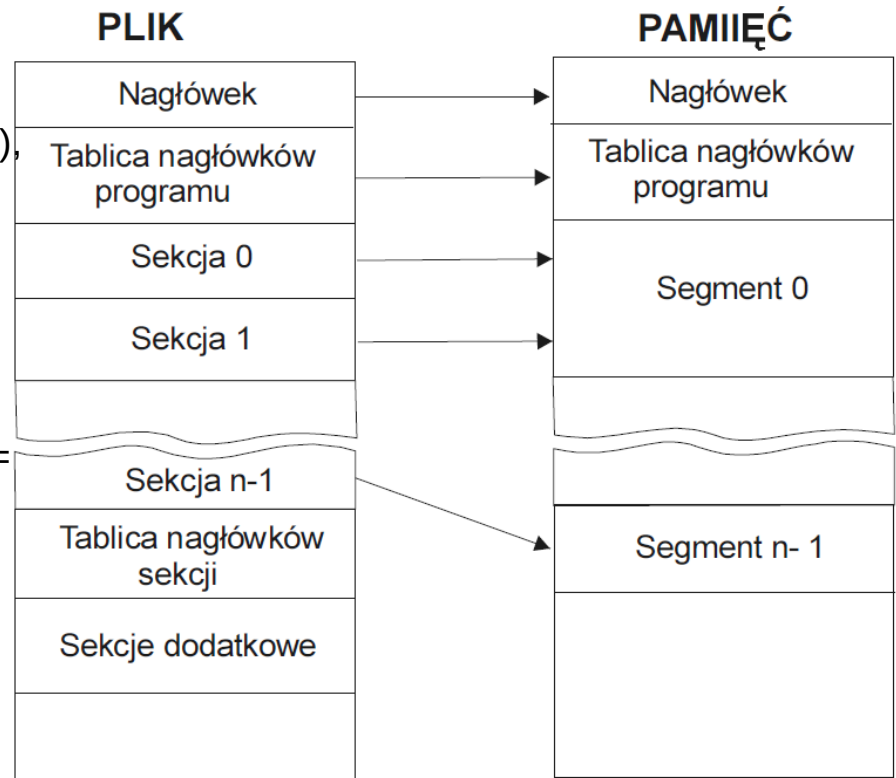
- Następca formatu a.out
- Umożliwia kompilację skrośną
- Występuje w trzech odmianach: pliki wykonywalne, pliki linkowalne i biblioteki (shared objects)





Format ELF

```
char magic[4] = "\177ELF"; // i(magic number)
char class; // rozmiar adresów: 1 = 32 bity, 2 = 64 bity
char byteorder; // porządek bajtów: 1 = mniejsze niżej (LE),
                // 2 = mniejsze wyżej (BE)
char hversion; // wersja nagłówka (zawsze 1)
char pad[9];
short filetype; // typ pliku: 1 = linkowalny (relokowalny),
                // 2 = wykonywalny, 3 = shared object,
                // 4 = obraz pamięci (core image)
short archtype; // typ procesora: 2 = SPARC, 3 = x86, 4 = 68K,
int fversion; // wersja pliku (zawsze 1)
int entry; // punkt wejścia (entry point) jeśli plik
           // wykonywalny
int phdrpos; // offset nagłówka pliku lub 0
int shdrpos; // offset nagłówka sekcji lub 0
int flags; // flaga charakterystyczna dla
           // architektury, zazwyczaj 0
short hdrsize; // rozmiar nagłówka pliku ELF
short phdrent; // rozmiar wpisu w nagłówku programu
short phdrcnt; // liczba wpisów w nagłówku programu lub 0
short shdrent; // rozmiar wpisu w nagłówku sekcji
short shdrcnt; // liczba wpisów w nagłówku sekcji lub 0
short strsec; // numer sekcji, która zawiera nazwy sekcji
```





Tablice GOT i PLT w plikach ELF

Tablica .plt

```
PLT0:
    pushl    *.GOT1
    jmp1     *.GOT2
    nop; nop
    nop; nop
    ...

.PLT11:
    jmp1     *.GOT3_printf
    pushl    $reloffset1
    jmp1     .PLT0

.PLT12:
    jmp1     *.GOT4_fopen
    pushl    $reloffset2
    jmp1     .PLT0
    ...
```

Tablica .got:

```
.GOT0: 0x804bf98

.GOT1: [flagi]

.GOT2: [adres konsolidatora]

.GOT3_printf:
    .PLT11 + 5

.GOT4_fopen:
    .PLT12 + 5

...
```



Biblioteki DLL (PE)

```
#include <windows.h>
#include <stdio.h>
BOOL __stdcall DllMain
(   HINSTANCE hinstDLL           // handle to DLL module
    DWORD fdwReason              // reason for calling function
    LPVOID lpReserved)
FILE* fptr;
fptr=NULL;
fptr = fopen("C:\\skelog.txt", "a");
switch(fdwReason)
{
    case DLL_PROCESS_ATTACH:
        fprintf(fptr,"Process pid=(%d) loading DLL\\n",GetCurrentProcessId());
        break;
    case DLL_THREAD_ATTACH :
        break;
    case DLL_THREAD_DETACH :
        break;
    case DLL_PROCESS_DETACH :
        break;
}
fclose(fptr);
return(TRUE);
__declspec(dllexport) void printMsg(char *str)
{
    printf("%s", str);
}
```



POLITECHNIKA
GDAŃSKA

Łączenie kodu biblioteki

- Biblioteki statyczne
- Konsolidacja dynamiczna w czasie ładowania (LTL)
- Konsolidacja dynamiczna w czasie wykonania (RTL)



POLITECHNIKA
GDAŃSKA

LTL

```
void printMsg(char *str);
```

```
void main()
```

```
{
```

```
    printMsg("using a DLL routine via Load-Time Linking!\n");
```

```
    return;
```

```
}
```



```
#include "windows.h"
typedef void (*printMsgptr)(char *str); //deklaracja wskaźnika
void main()
{
    HINSTANCE hinstLib;
    printMsgptr printMsg;
    hinstLib = LoadLibraryA( "Bibl.dll");
    if (hinstLib != NULL)
    {
        printMsg = (printMsgptr)GetProcAddress(hinstLib,"printMsg");
        if (printMsg != NULL)
        {
            printMsg("using a DLL via Run-Time Linking\n");
        }
        FreeLibrary(hinstLib);
    }
    return;
}
```



- Celem jest zmodyfikowanie tablicy IAT (Import Address Table) - pozyskanie adresu w liniowej przestrzeni adresowej procesu
- Można tego dokonać:
 - Zmiana wpisu w rejestrze
 - Ustawienie haka
 - Użycie zdalnych wątków



Zmiana wpisu w rejestrze

- HKLM\Software\Microsoft\Windows
NT\CurrentVersion\Windows
- ApplInit_DLLs = 0x01
- LoadAppInit_DLLs+ lista ładowanych bibliotek razem z
user32.dll



Haki (hooks)

```
HOOK SetWindowsHookEx  
(  
    int hookType, //event that will invoke hook routine  
  
    HOOKPROC procptr, //exported Dll routine to call when  
        event occurs  
  
    HINSTANCE dllHandle, //handle to DLL containing hook  
        procedure  
  
    DWORD dwThreadId //specific thread, or (0) all threads  
        on the desktop  
);
```



Hooks - przykład

```
HOOKPROC procPointer;  
static HMODULE dllHandle;  
static HHOOK procHandle;  
dllHandle = LoadLibraryA(" c:\\windows\\testDll.dll");  
if (dllHandle=NULL)  
    {return;}  
procPointer = (HOOKPROC)GetProcAddress(dllHandle,  
    "?MouseProc@1'!!lVFDSfdZ");  
if (procPointer=NULL) {return;}  
procHandle=SetWindowsHookEx(WH_MOUSE,  
    procPointer,dllHandle,0);  
if (procHandle=NULL) {return;}
```

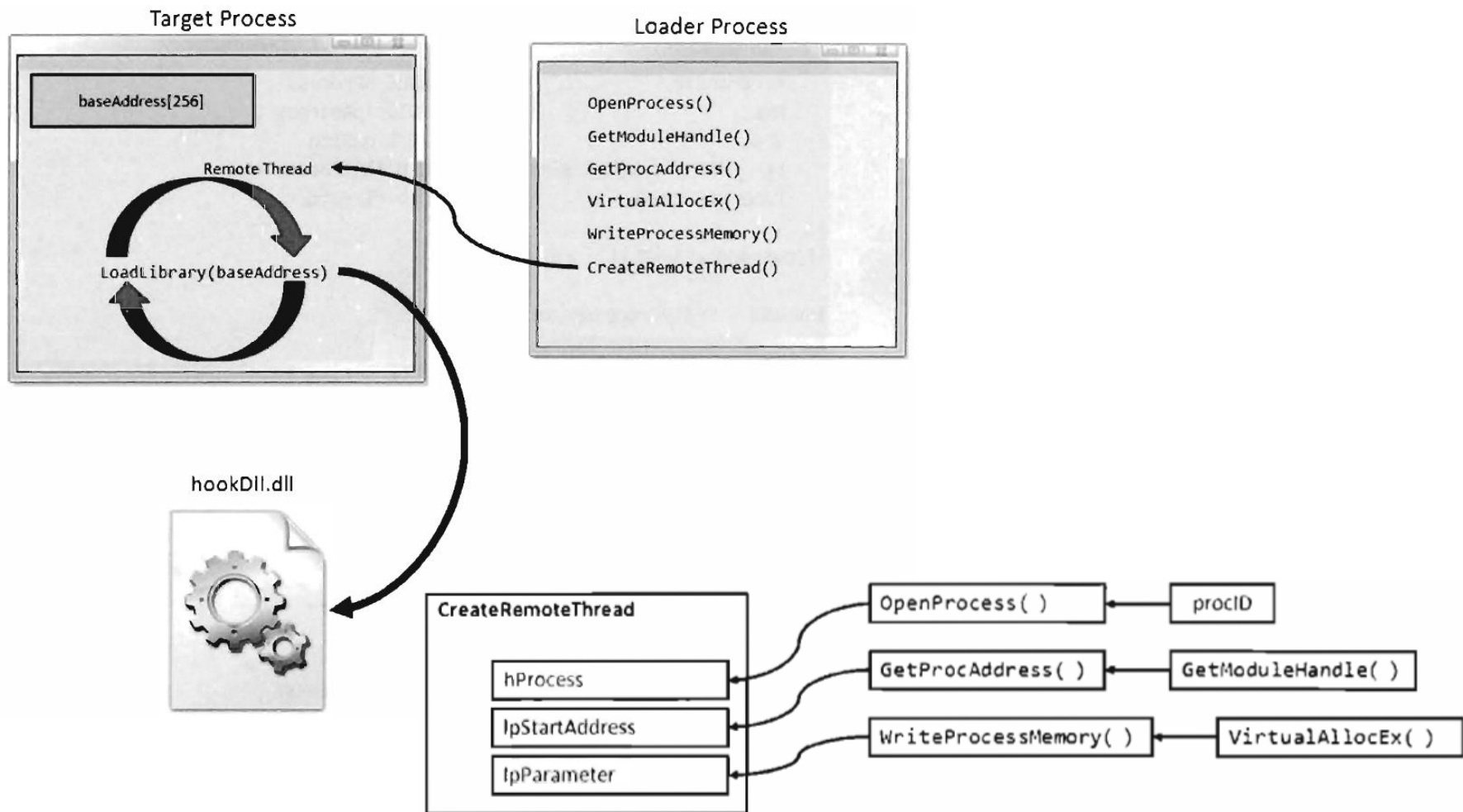


Hooks - przykład

```
_declspec(dllexport) __LRESULT CALLBACK MouseProc  
(int code,  
WPARAM wParam,  
LPARAM lParam)  
{  
/*  
Tutaj kod  
*/  
  
return(CallNextHookEx(NULL, nCode, wParam, lParam));  
}
```



Zdalny wątek (1)





Zdalny wątek (2)

```
procHandle= OpenProcess (PROCESS_ALL_ACCESS, FALSE, procID);  
  
if (procHandle==NULL){ return;}  
  
dllHandle=GetModuleHandleA("kernel32");  
if(dllHandle==NULL){ return; }  
  
loadLibraryAddress= GetProcAddress(dllHandle, "LoadLibraryA");  
if (loadLibraryAddress==NULL){ return; }  
  
baseAddress= VirtualAllocEx(procHandle, NULL,256, MEM_COMMIT |  
    MEM_RESERVE, PAGE_READWRITE);  
If (baseAddress==NULL){ return;}
```



Zdalny wątek (3)

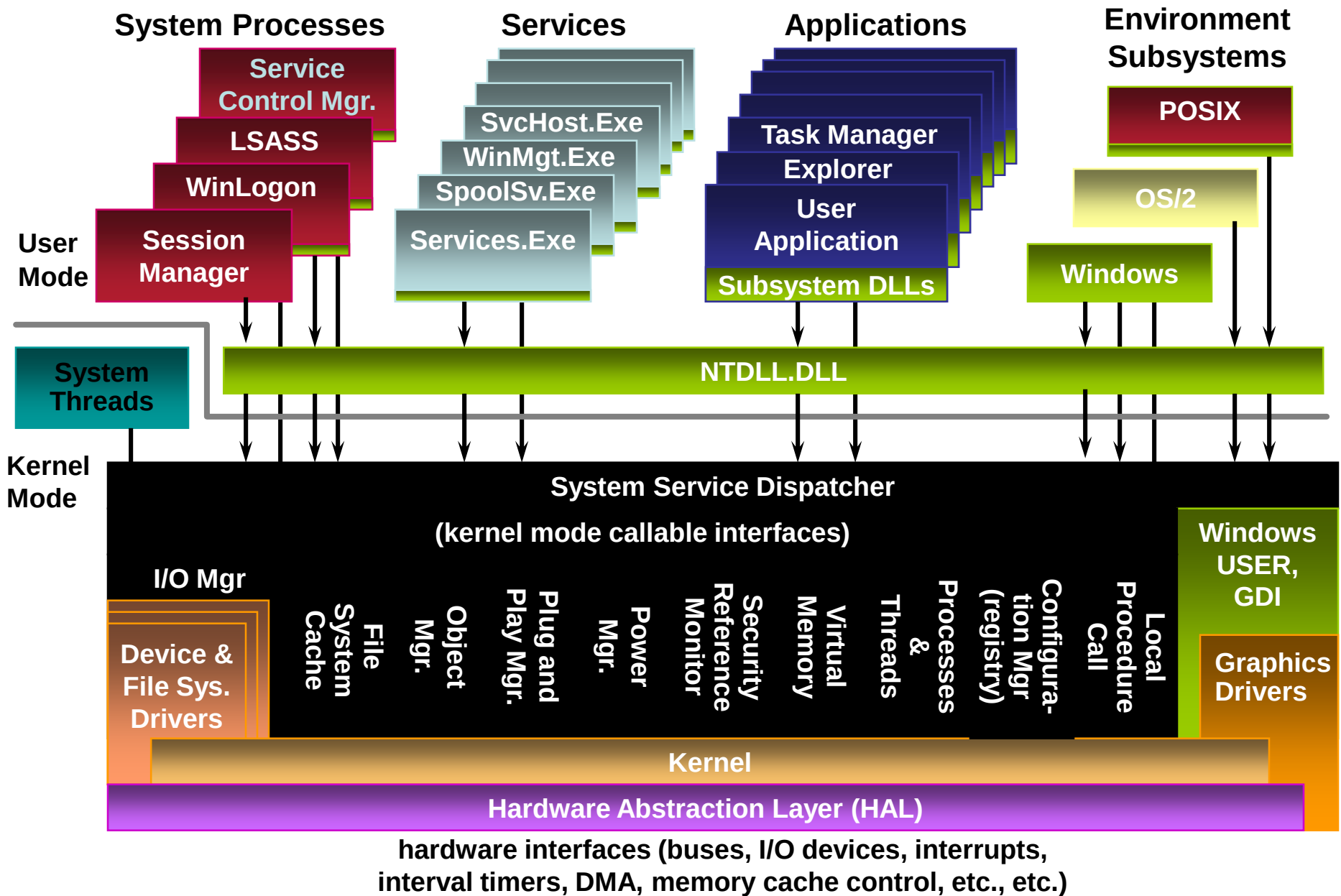
```
isValid = WriteProcessMemory (procHandle, baseAddress,  
    argumentBuffer, sizeof( argument Buffer )+1,NULL)  
if(isValid==0){ return; }
```

```
threadHandle= CreateRemoteThread(  
    procHandle,NULL,0,loadLibraryAddress,baseAddress,0,NULL);
```

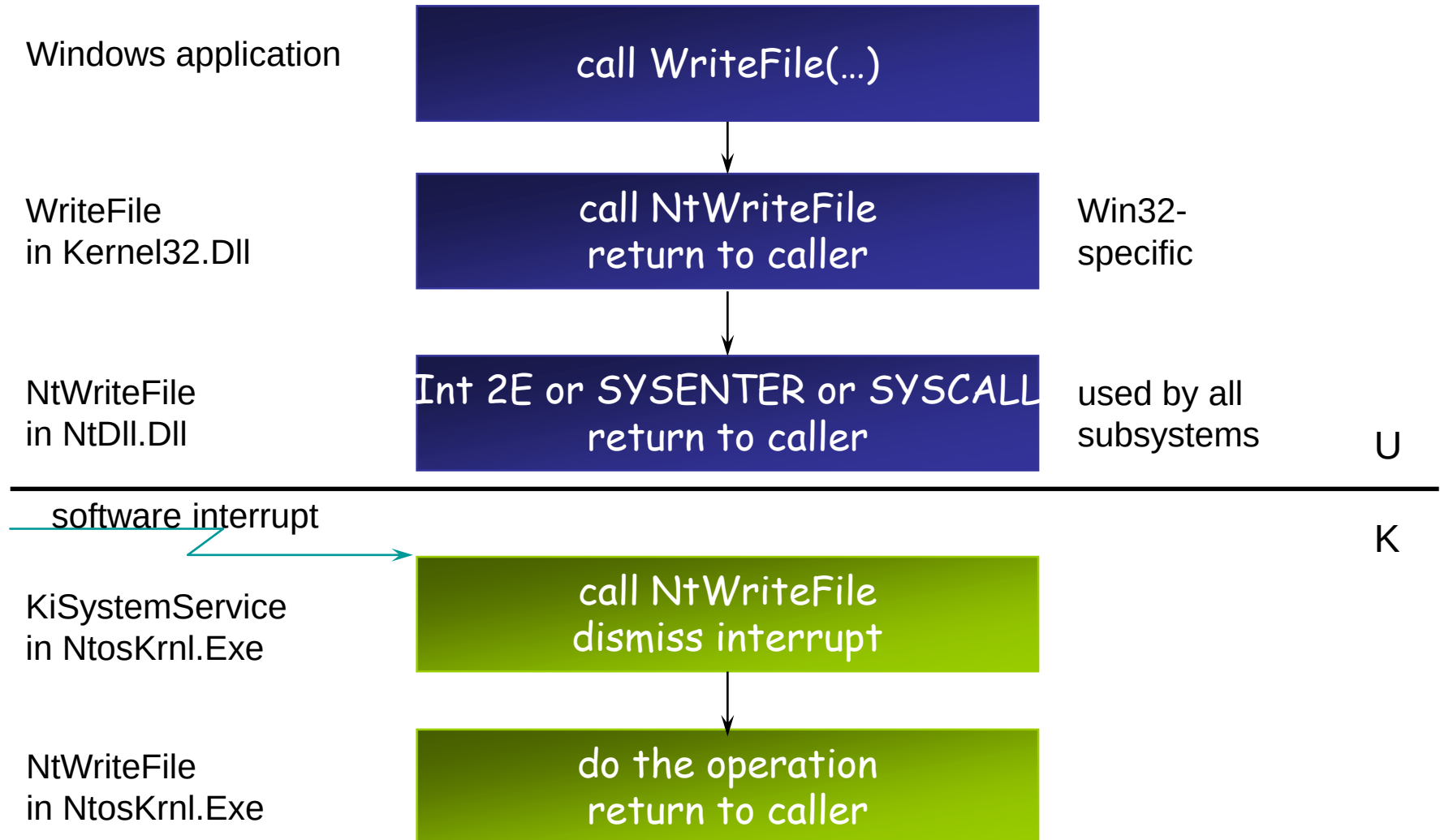
Oprogramowanie Systemowe

Procesy i wątki w Windows

Architektura Windows XP



Wywołanie funkcji systemowych kernela



Przykładowy kod

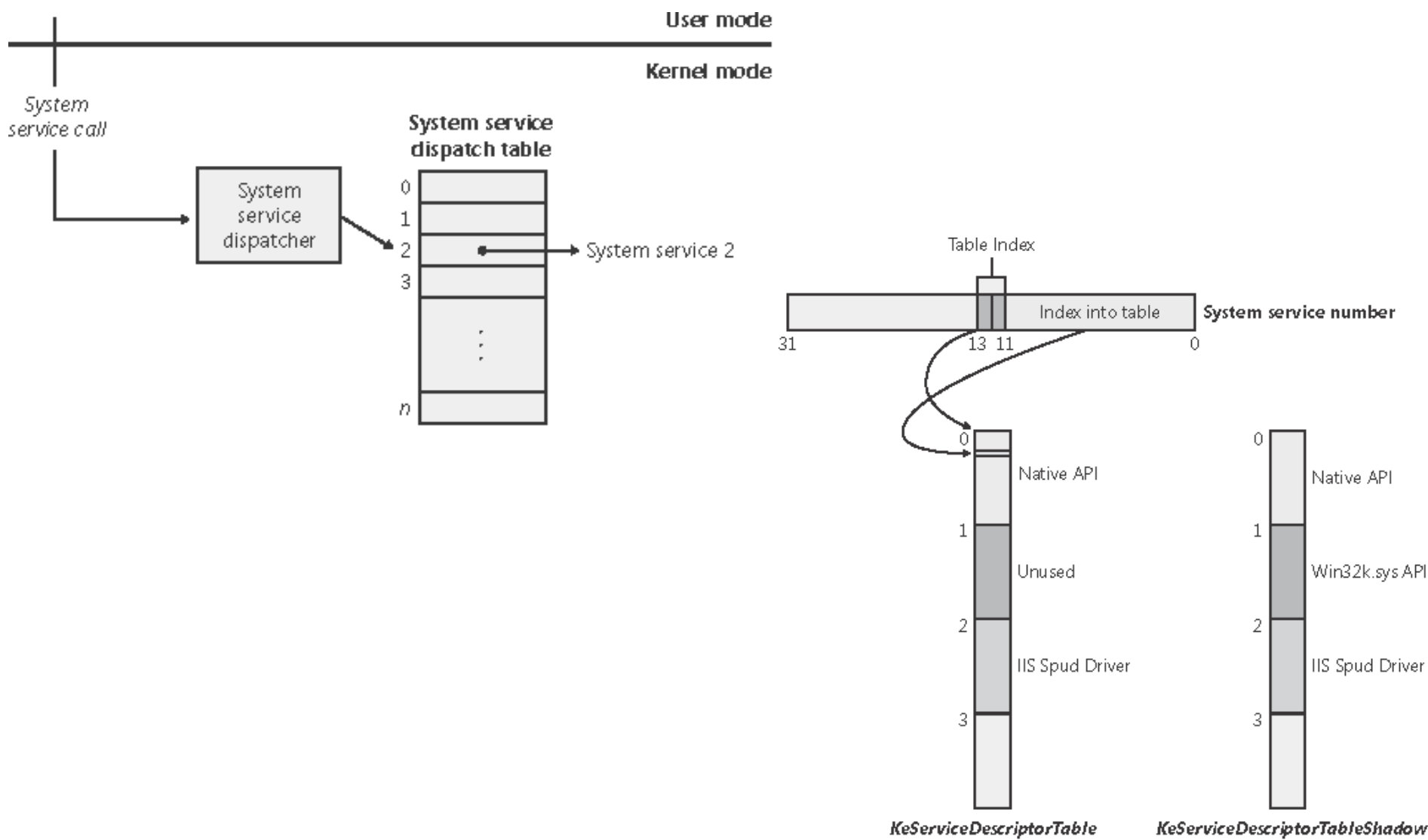
ntdll!NtReadFile:

77f5bfa8 b8b7000000	mov eax,0xb7
77f5bfad ba0003fe7f	mov edx,0x7ffe0300
77f5bfb2 ffd2	call edx
77f5bfb4 c22400	ret 0x24

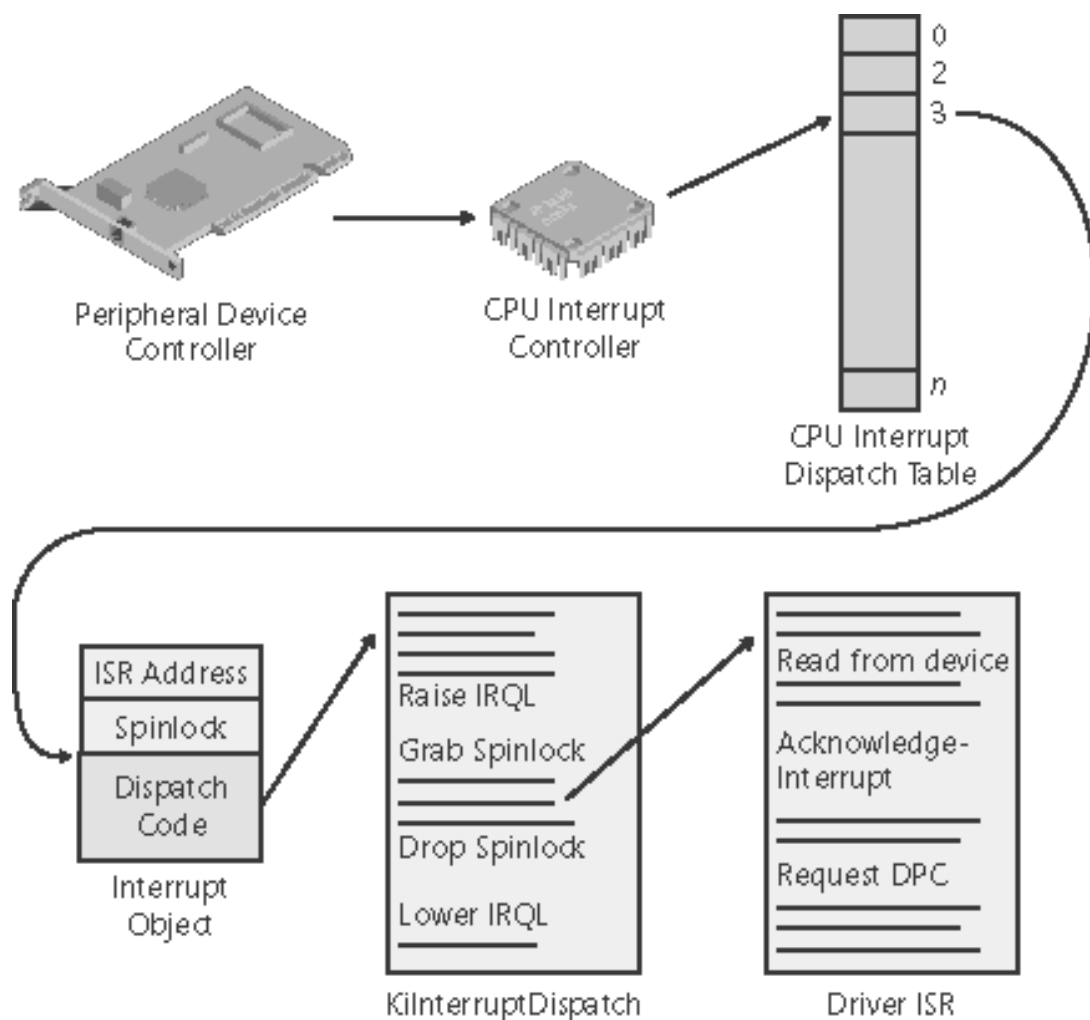
SharedUserData!SystemCallStub:

7ffe0300 8bd4	mov edx,esp
7ffe0302 0f34	sysenter
7ffe0304 c3	ret

Dalsza część przetwarzania



Przerwania



Wirująca blokada (Spinlock)

synchronizacja wątków na poziomie oprogramowania sprowadza się zwykle do cyklicznego testowania zawartości pewnej zmiennej współdzielonej przez kilka wątków, co określane jest terminem *wirująca blokada*;

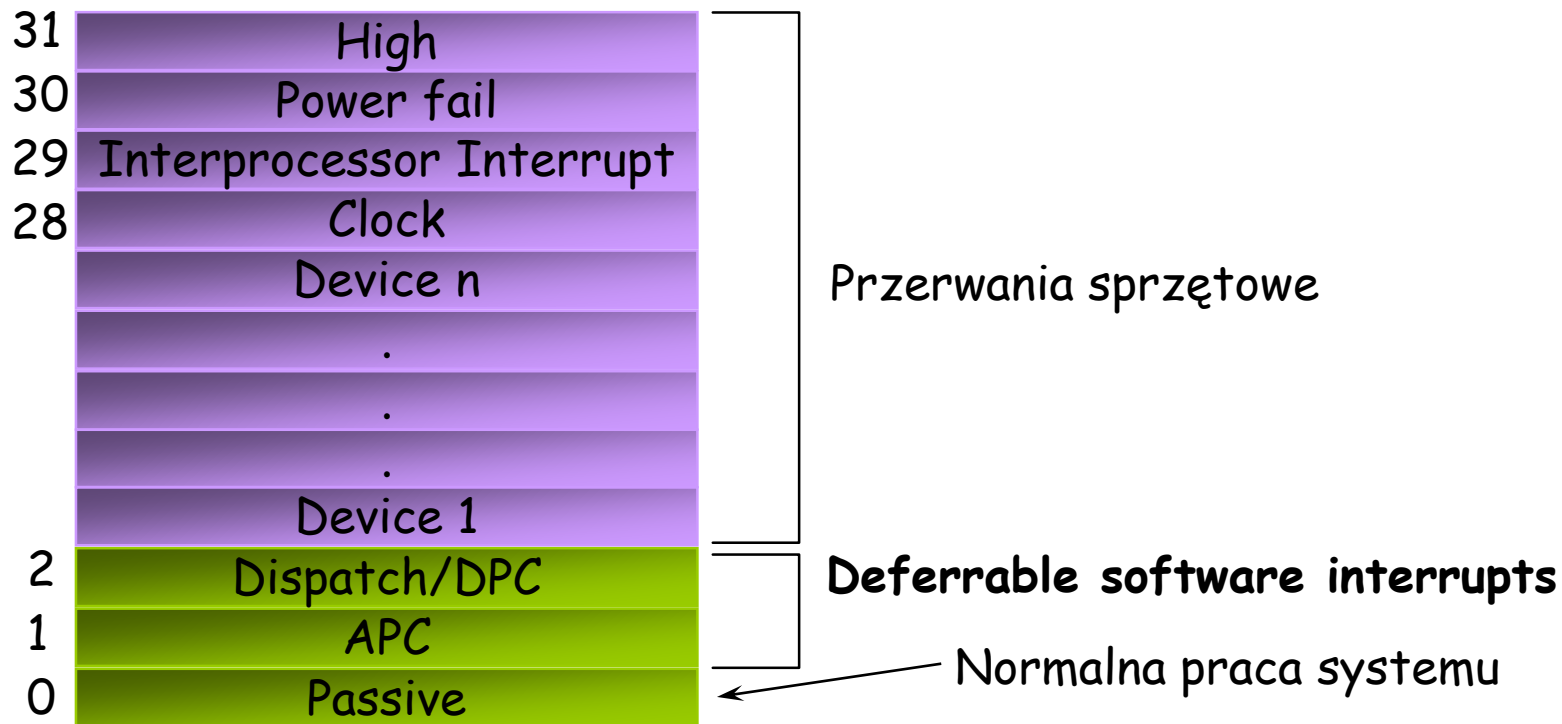
w architekturze IA-32 dostępny jest rozkaz XCHG, który zamienia zawartości podanych operandów; przypuśćmy, że zmienna zamek dostępna jest dla kilku wątków wykonywanych przez różne procesory;

```
zamek      db      0
_ _ _ _ _
powtorz:    mov     al, 1
            xchg    zamek, al
            cmp     al, 1
            je      powtorz
_ _ _ _ _ ; sekcja krytyczna (tylko jeden wątek
_ _ _ _ _ ; może znajdować się w tej sekcji)
            mov     zamek, 0
```

- warunkiem poprawnej realizacji podanego kodu jest niepodzielne wykonywanie rozkazu XCHG;

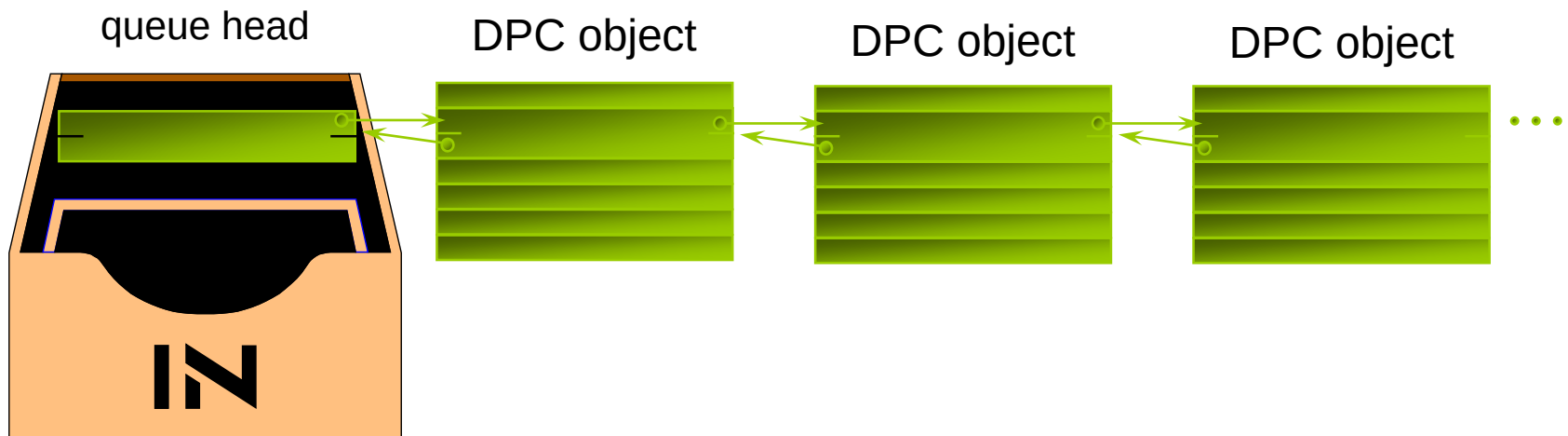
Poziomy uprzywilejowania przerwań IRQL

- IRQL = Interrupt Request Level
 - To nie to samo co IRQ (27-IRQ)
- IRQL jest stanem procesora (nie jako wirtualnym)
- Nie wolno oczekiwać na obiekt oraz nie może wystąpić wymiana stron (tzn. $IRQL \geq DISPATCH_LEVEL$)



Deferred Procedure Calls (DPCs)

- Odroczone wywołanie procedury (DPC) jest wykorzystane do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy
 - Sterownik (proces jądra) kolejkuje żądania
 - Wykonanie zadań z kolejki następuje w momencie zakończenia pracy wszystkich przerwań o wyższym IRQL
- Wykorzystane jest do generowania timeout'ów oraz przełączania wątków





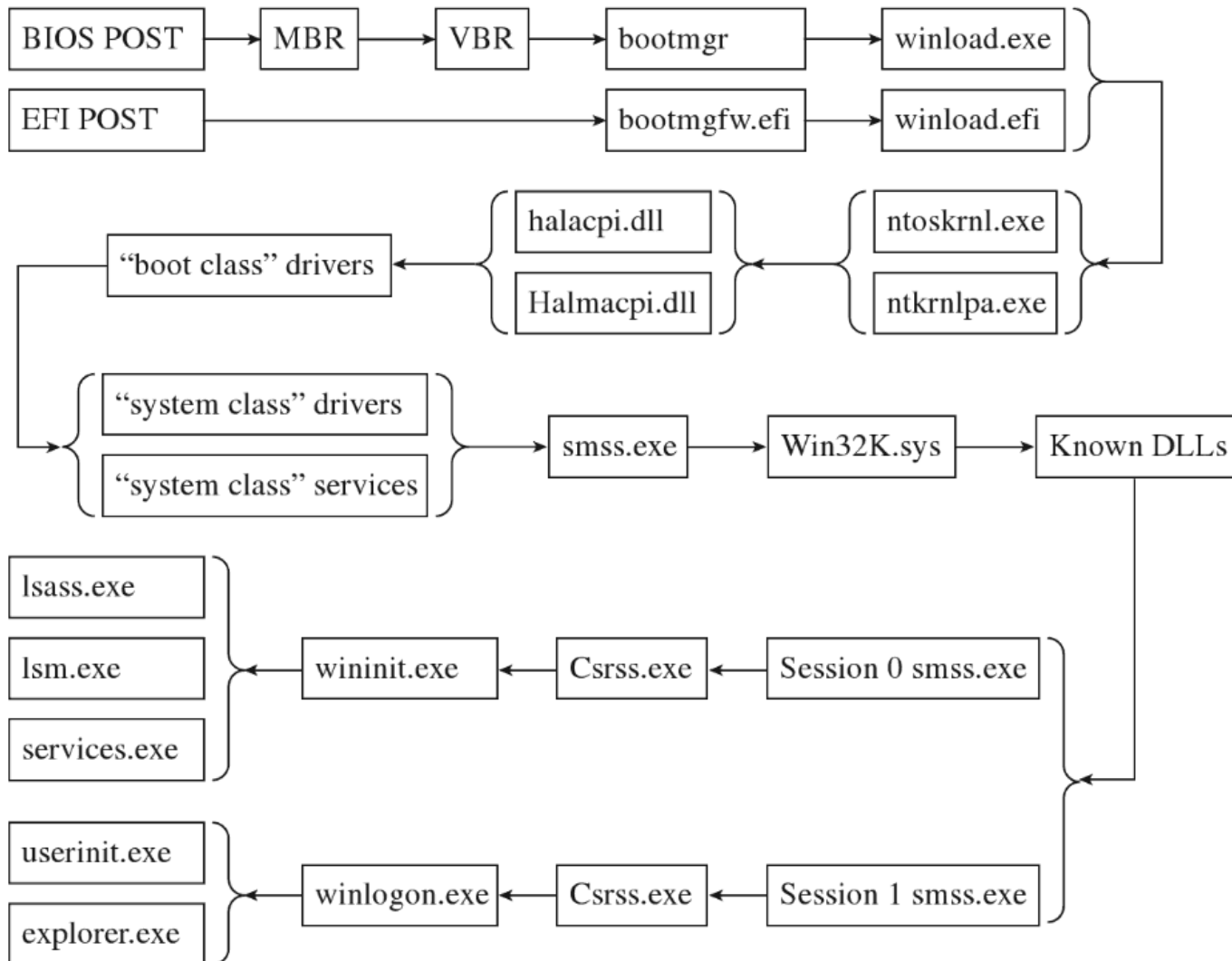
**POLITECHNIKA
GDAŃSKA**

Zarządzanie procesami (Process management)

Tomasz Dziubich
dziubich@eti.pg.edu.pl

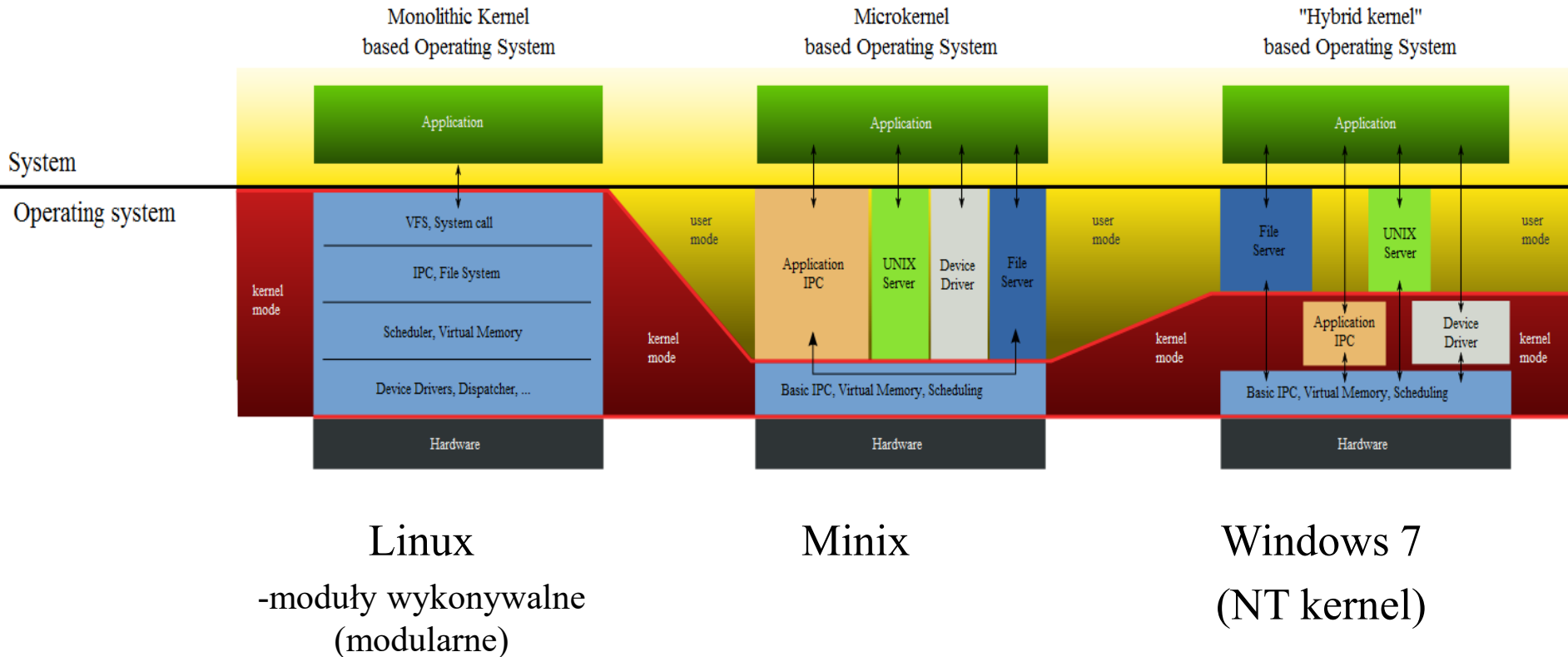


Krótkie przypomnienie



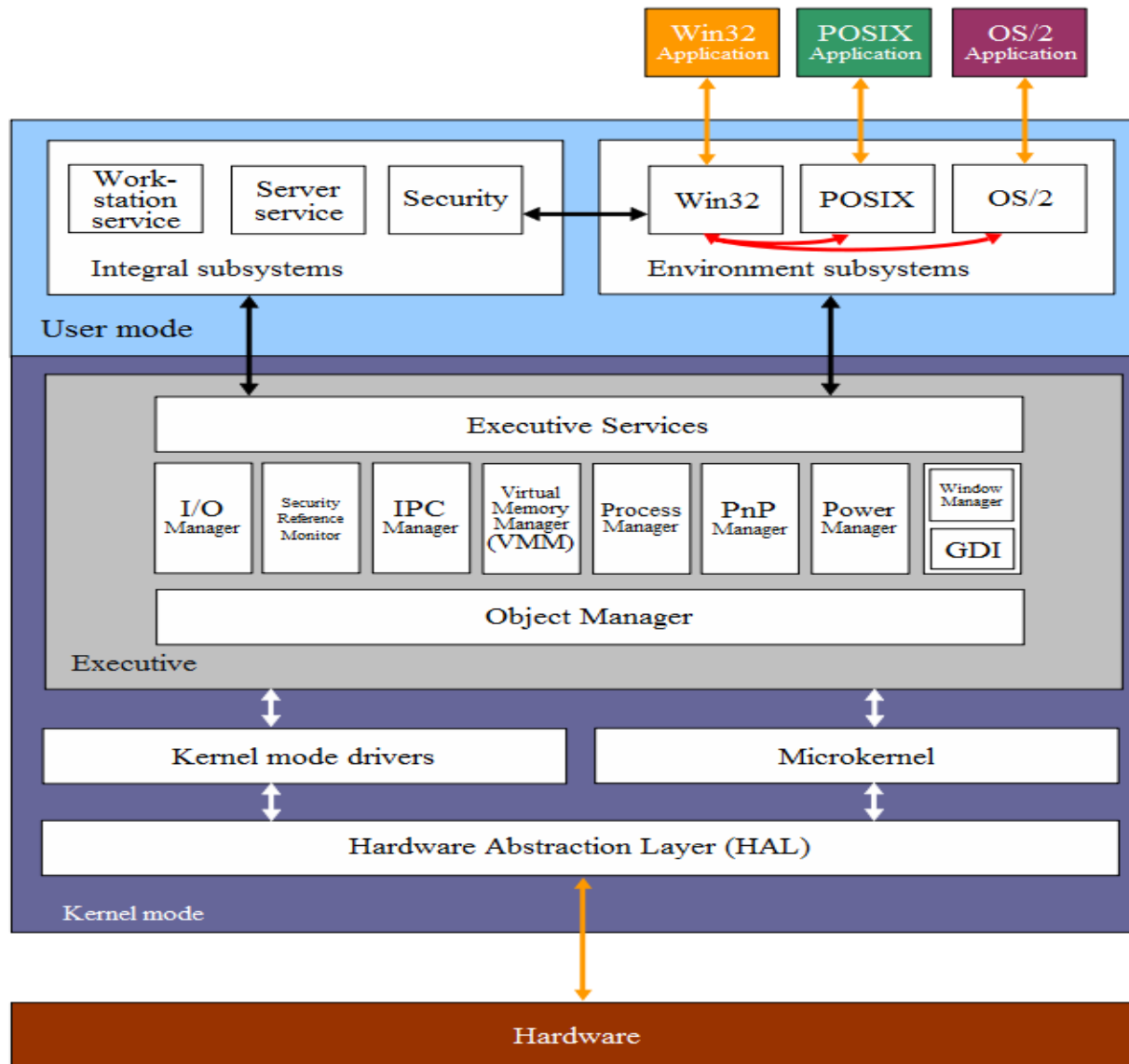


Podstawowe architektury SO



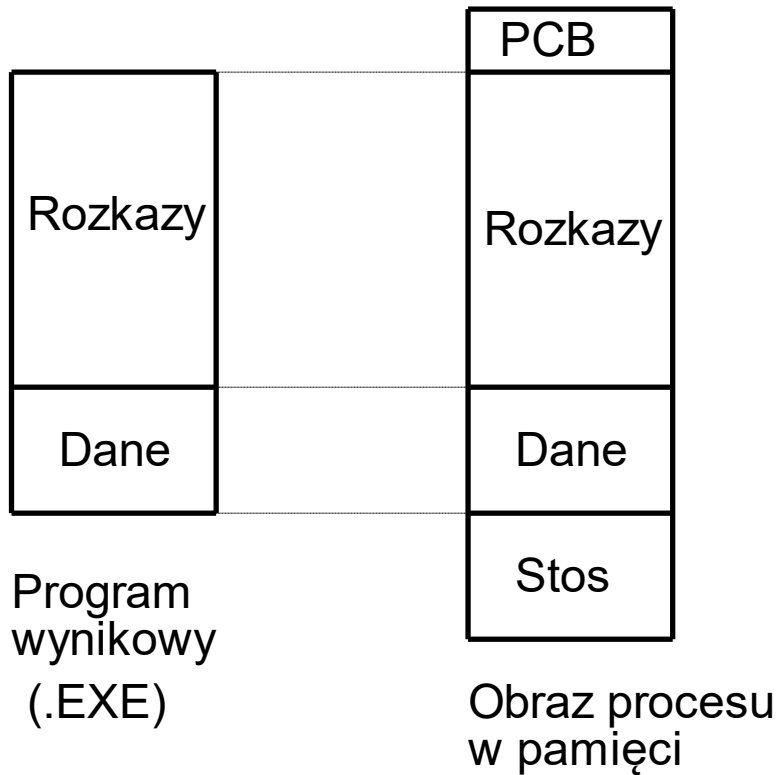


Architektura Windows

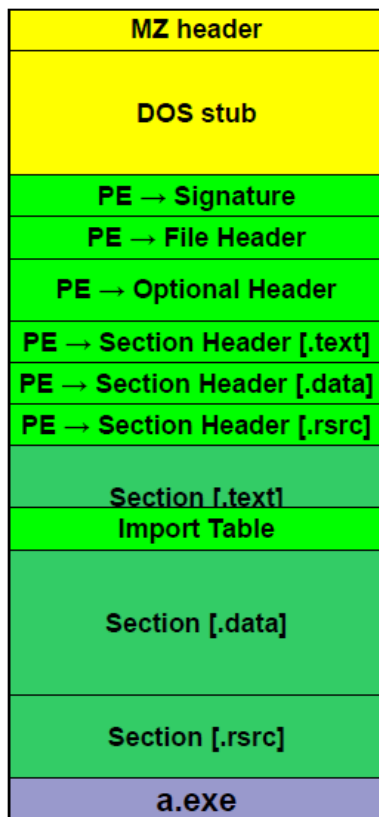




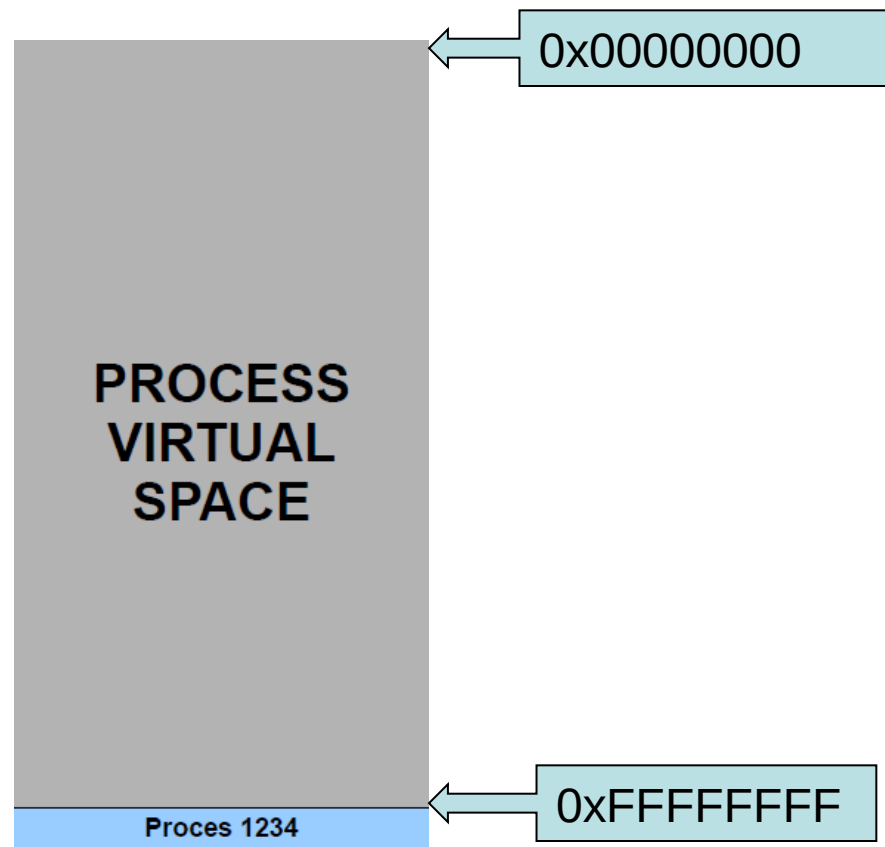
Ładowanie programów do pamięci -> tworzenie procesu



Proces to kontener z zasobami
używanymi podczas
wykonywania instancji
programu



Plik



Pamięć wirtualna
przydzielona procesowi

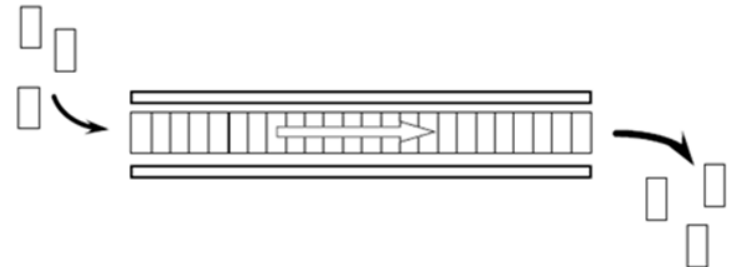
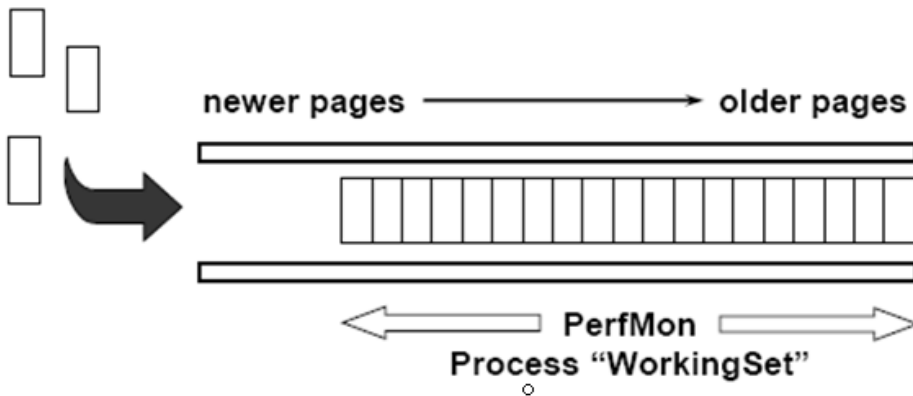


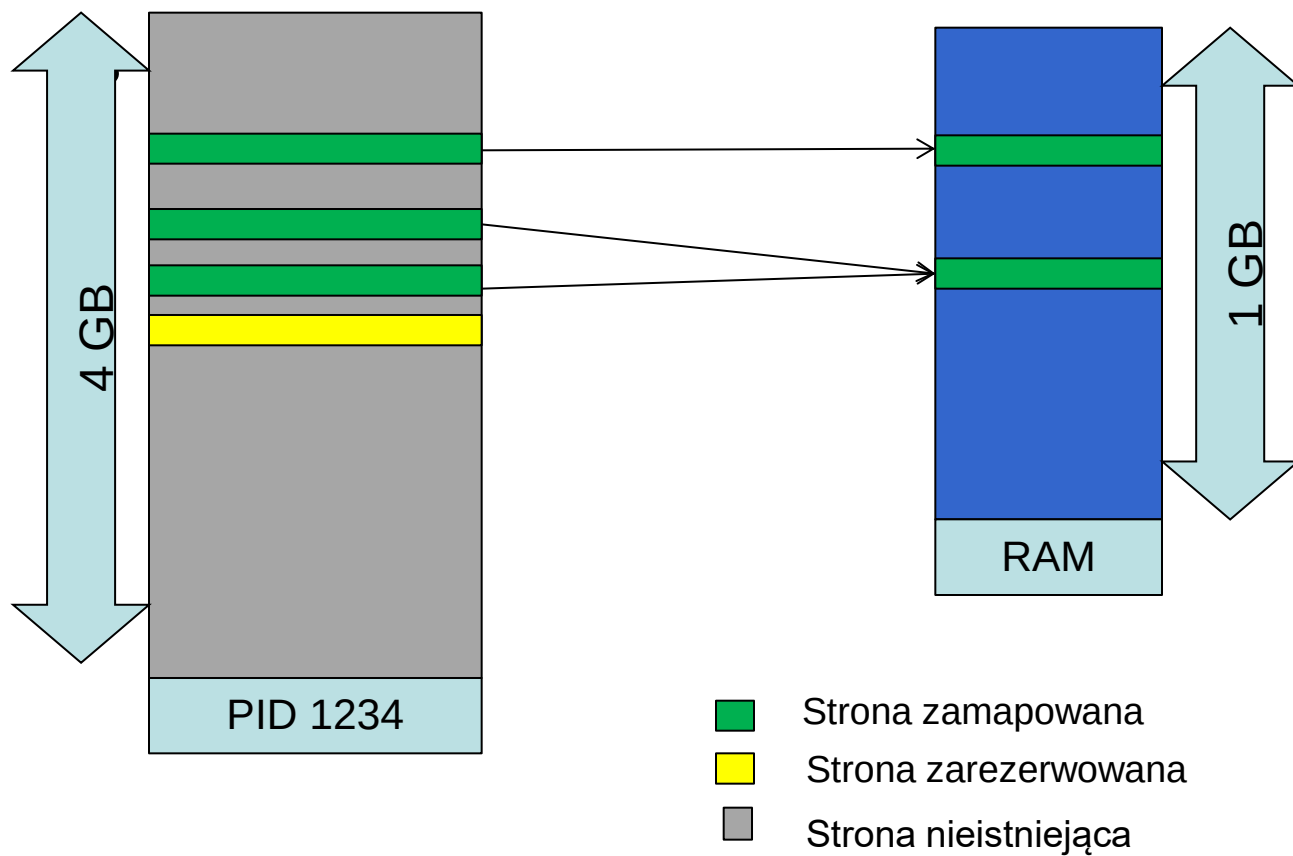
Struktury systemowe procesu w Windows

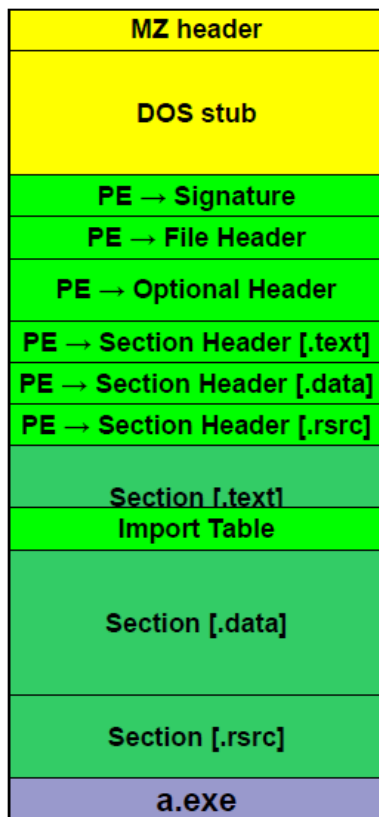
- Prywatna wirtualna przestrzeń adresowa - zbiór adresów pamięci wirtualnej do dyspozycji procesu.
- Program wykonywalny - zmapowany w pamięci kod i dane początkowe.
- Lista wskaźników itp. do różnych zasobów takich jak semaforey, porty komunikacyjne, pliki, dostępne dla wszystkich wątków w procesie.
- Znacznik dostępu - struktura zawierająca security ID, listę grup, jakich członkiem jest dany użytkownik, przywileje. Znacznik dostępu jest dziedziczony po procesie-rodzicu.
- Unikatowy identyfikatora procesu.
- Przynajmniej jeden wątku wykonawczego.
- Wskaźnika do procesu nadrzędnego



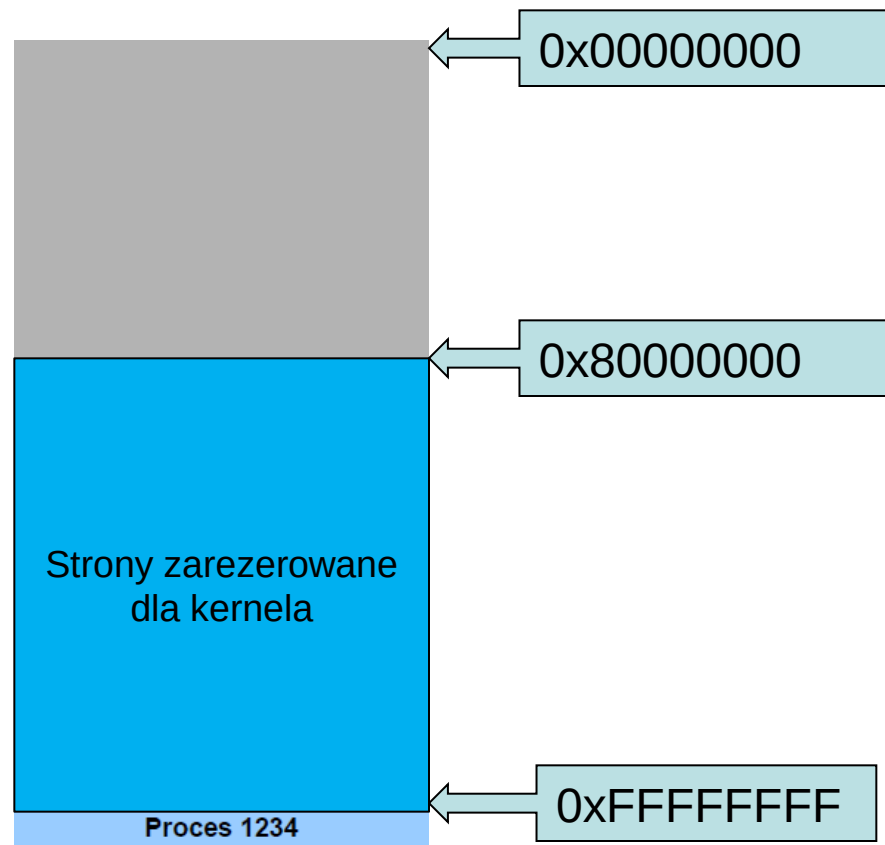
- Virtual Address Descriptors (VADs)
 - Zbiór struktur do śledzenia który z adresów wirtualnych w przestrzeni adresowej procesu jest zarezerwowany a który nie.
 - Opisuje stan przestrzeni adresowej procesu
- Working Set
 - Working set jest podzbiorem stron, które rezydują w pamięci fizycznej.



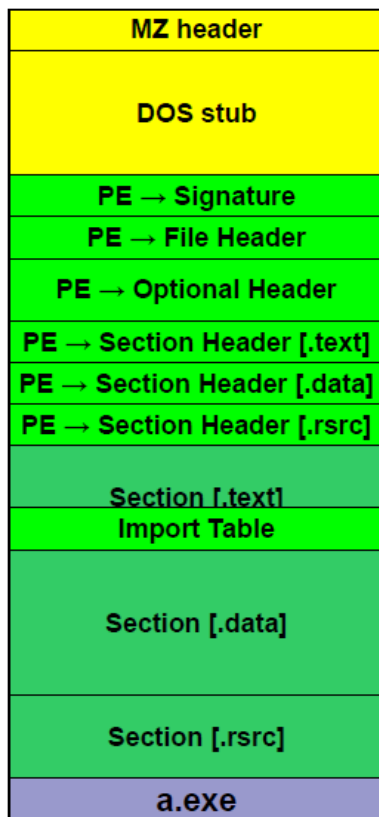




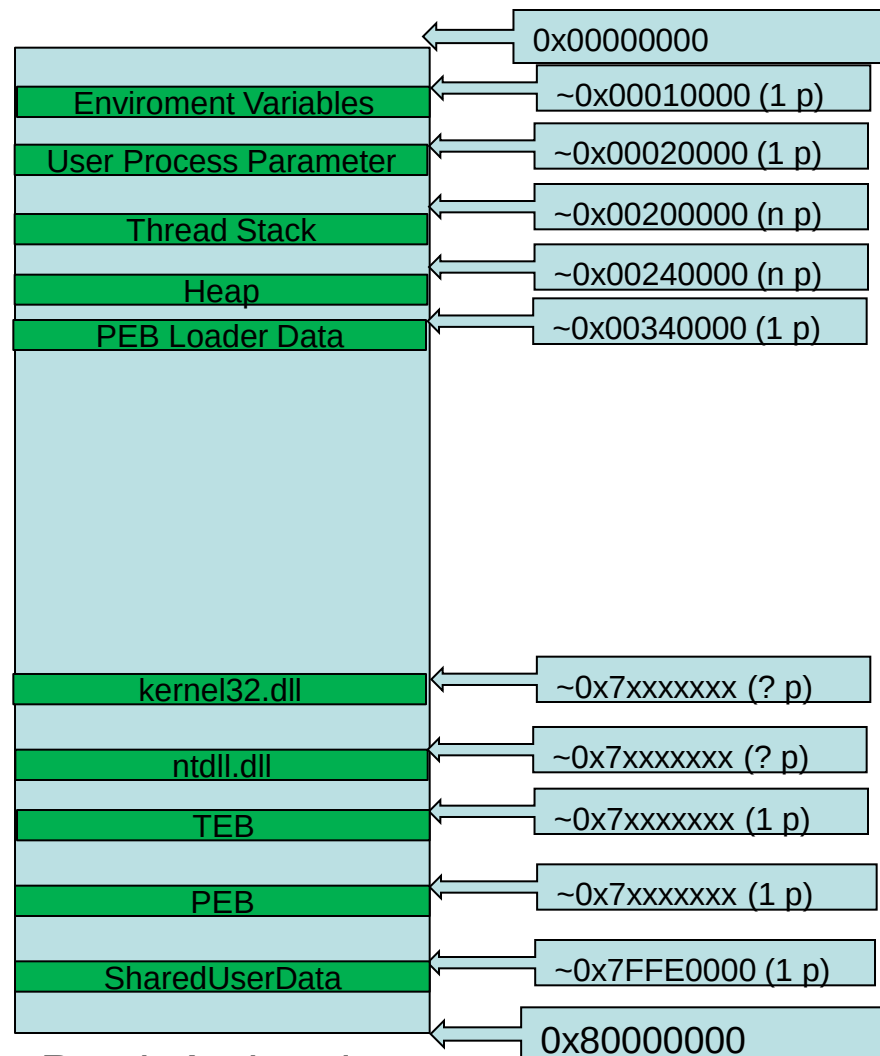
Plik



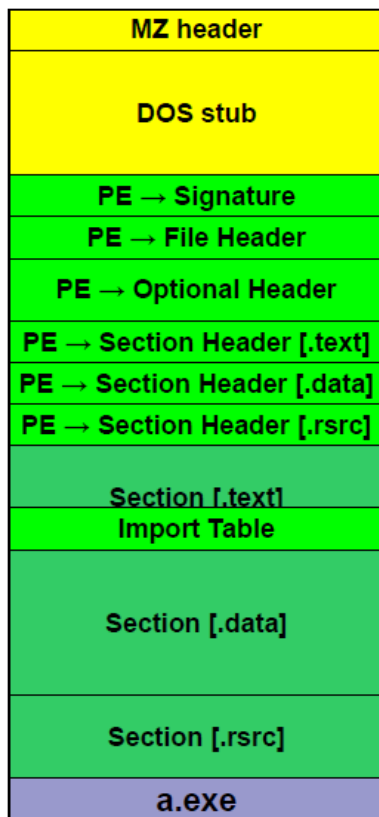
Pamięć wirtualna
przydzielona procesowi



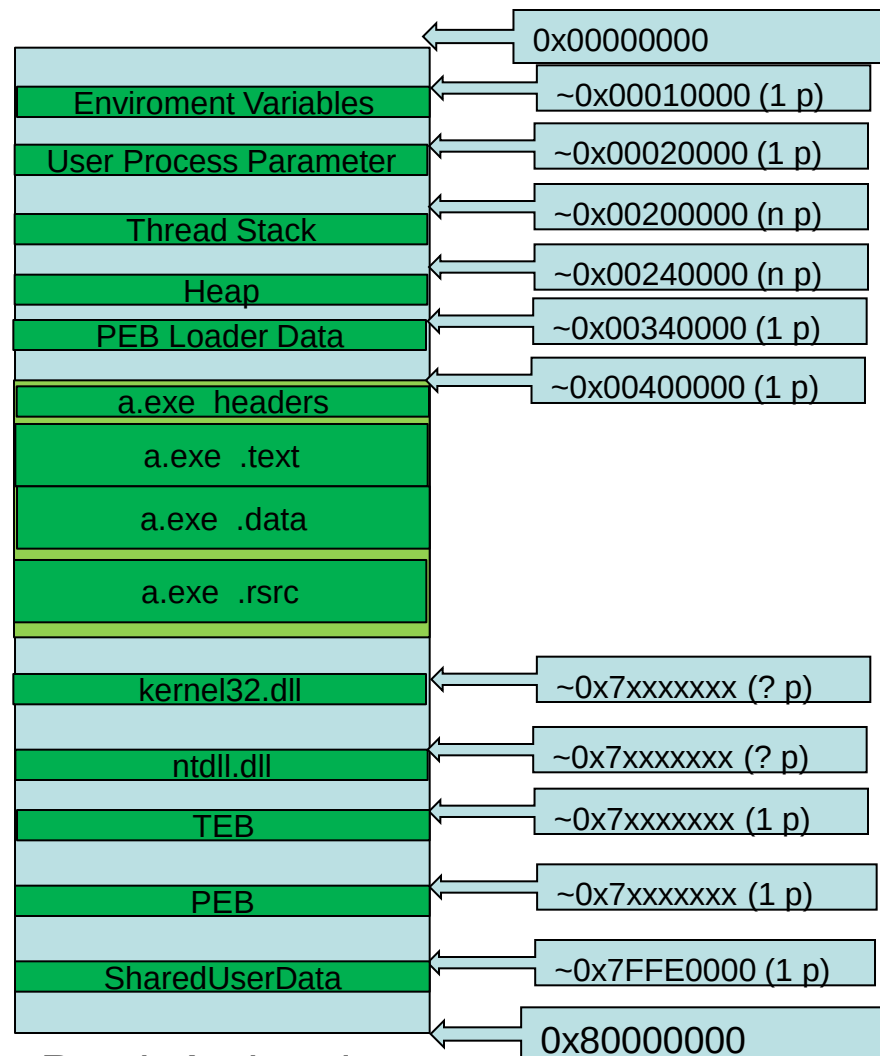
Plik



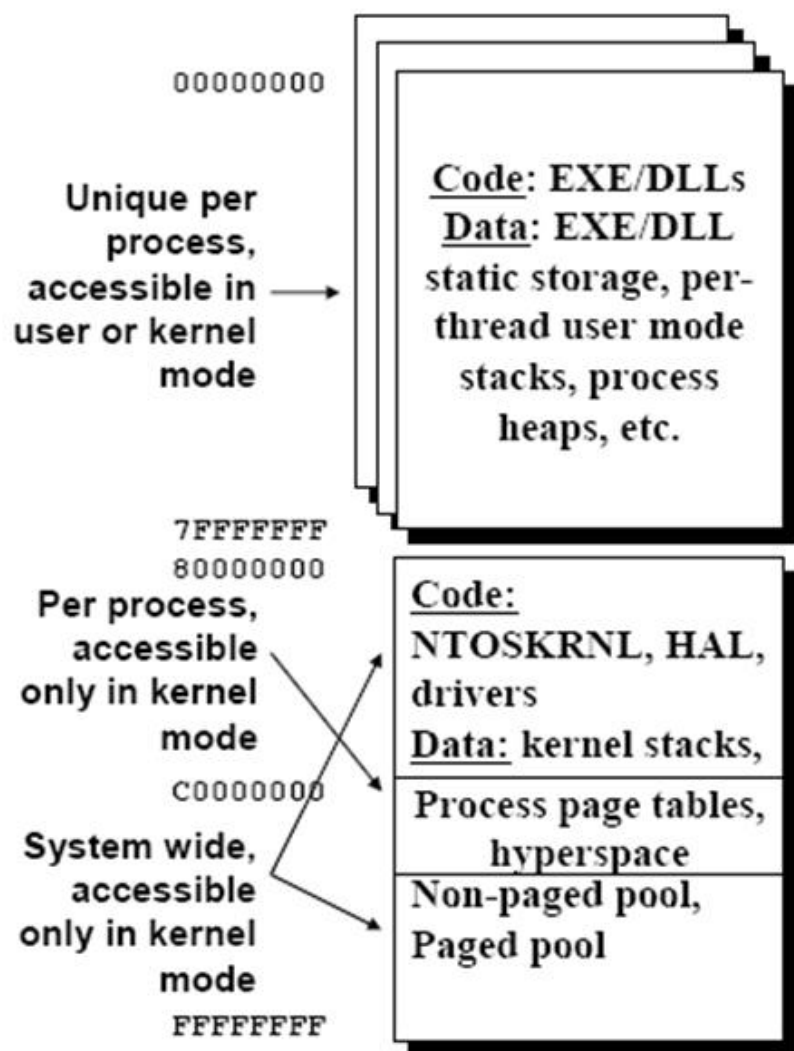
Pamięć wirtualna
przydzielona procesowi



Plik



Pamięć wirtualna
przydzielona procesowi



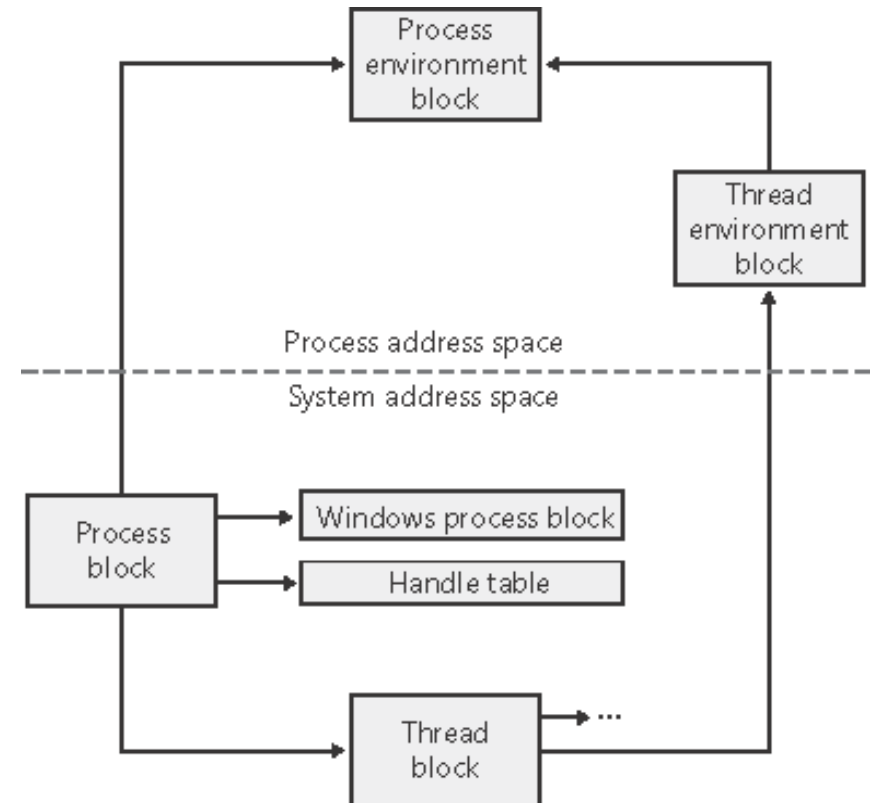
- **2 GB per-process**
 - Address space of one process is not directly reachable from other processes
- **2 GB system-wide**
 - The operating system is loaded here, and appears in every process's address space
 - The operating system is not a process (though there are processes that do things for the OS, more or less in "background")
- **3 GB user space and Address Windowing Extensions (AWE) t.b.d.**



Struktury systemowe procesu w Windows

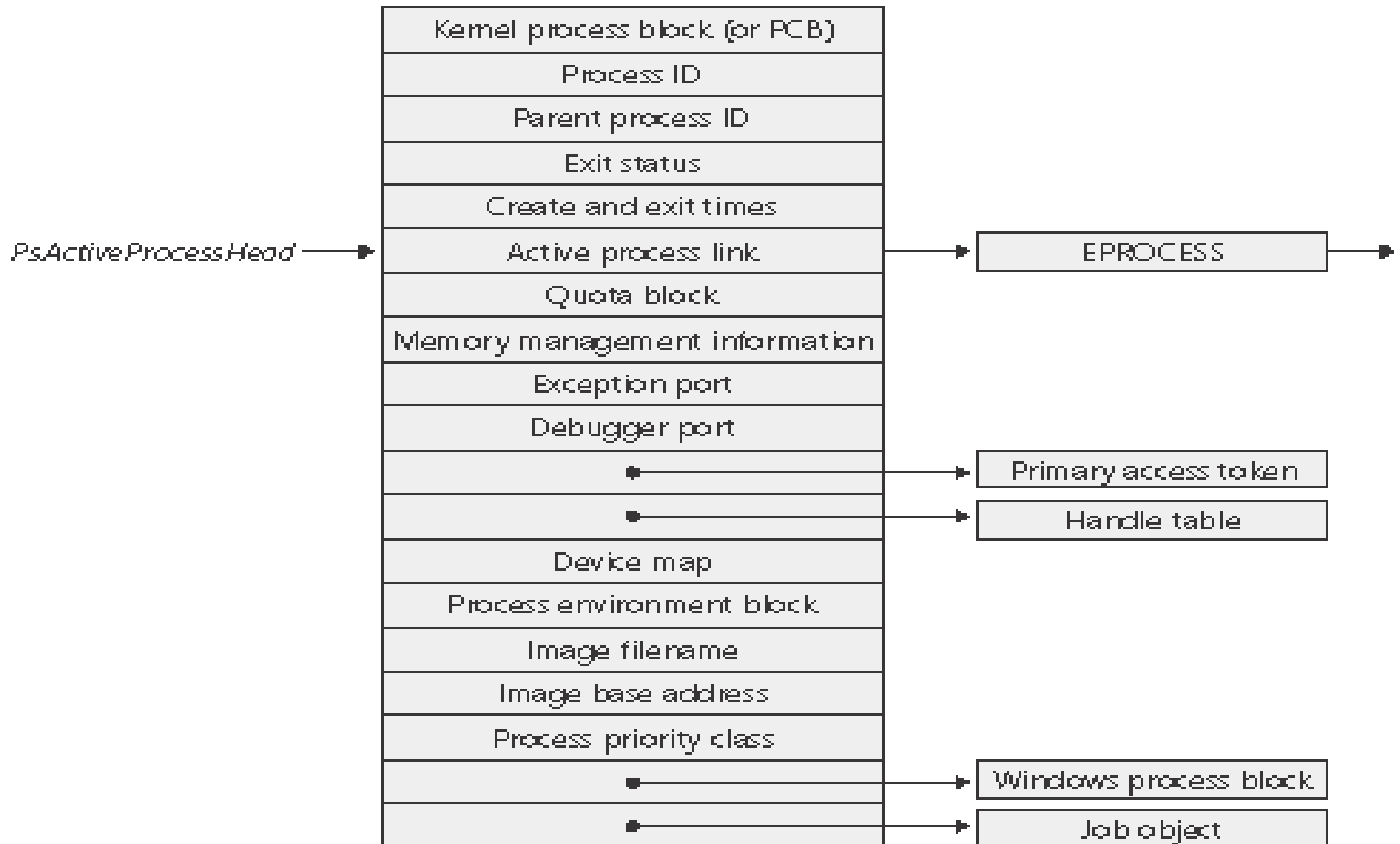
Z procesem powiazane są

- Blok wykonawczy procesu (executive process block EPROCESS)
- Bloki wykonawcze wątków (executive thread blocks ETHREAD)
- Blok środowiska procesu



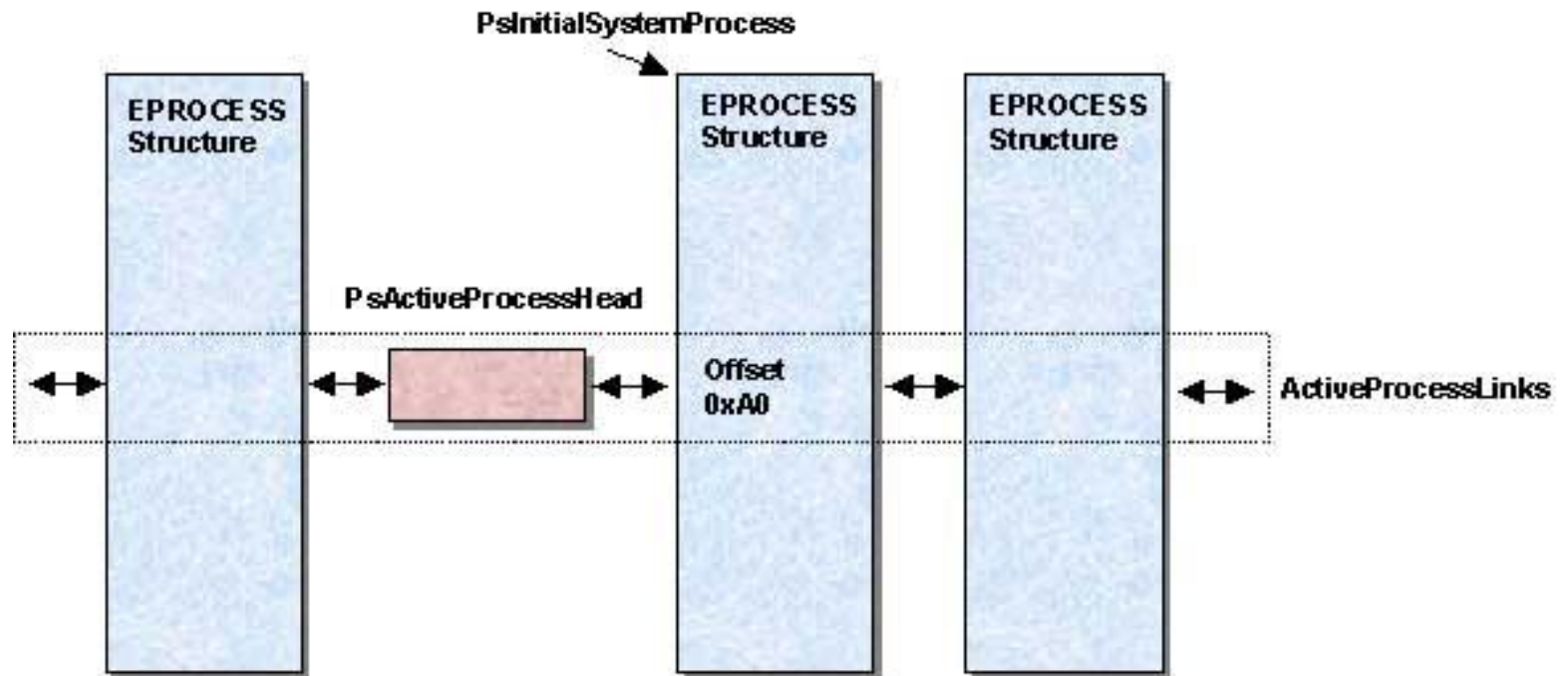


EPROCESS



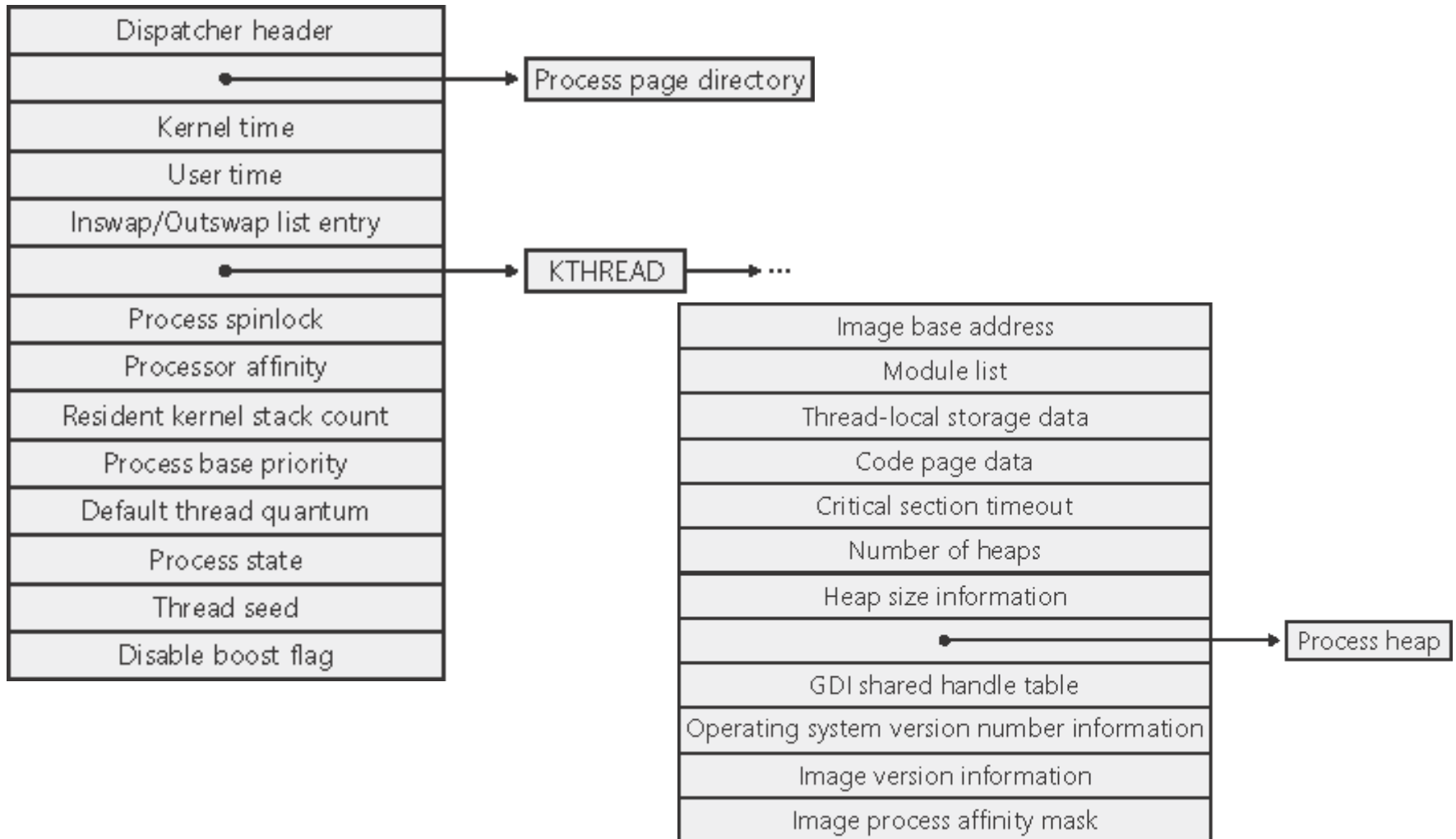


Kolejka procesów





PCB/KPROCESS i PEB





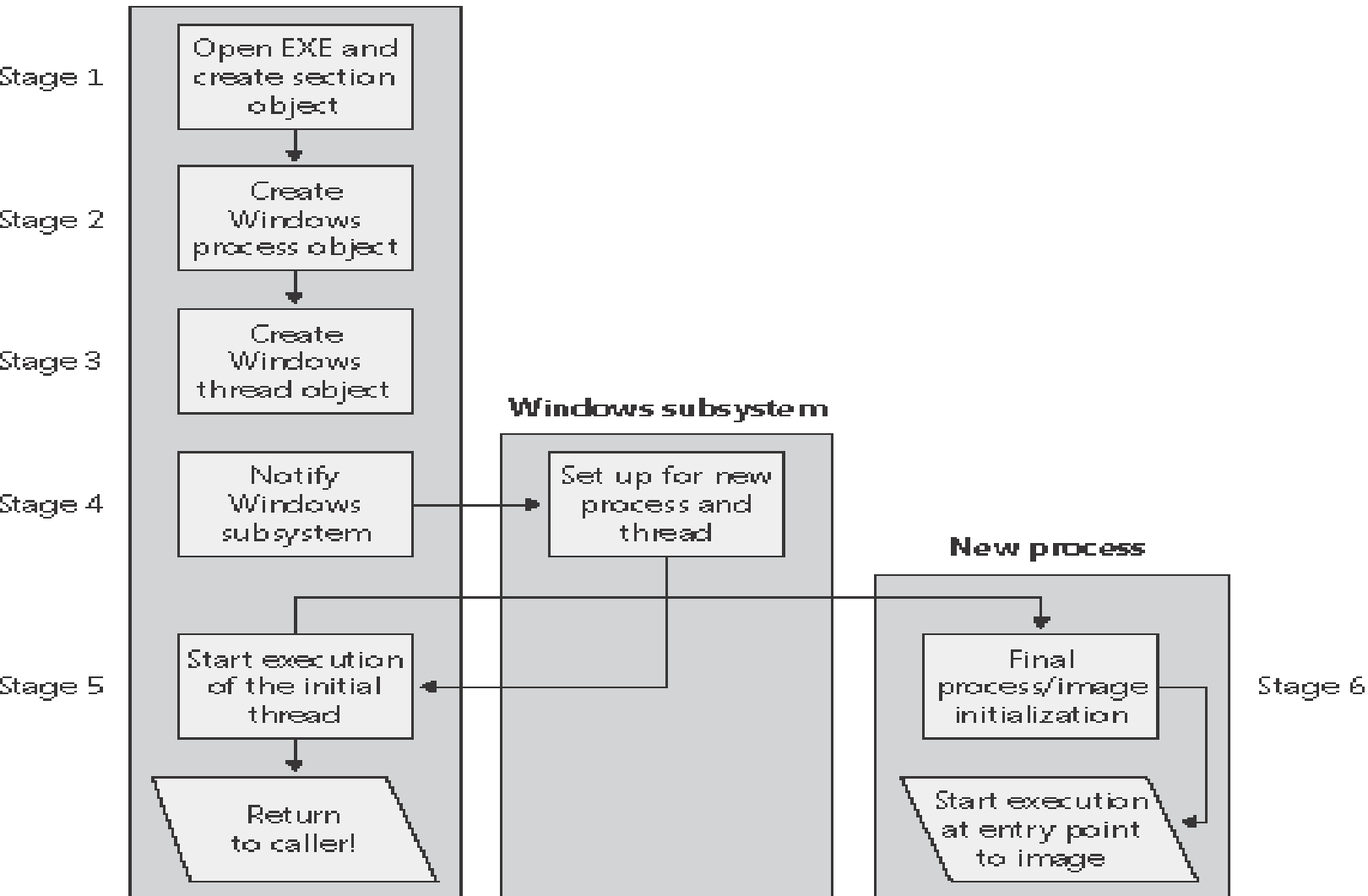
Przepływ sterowania podczas tworzenia procesu - podsumowanie

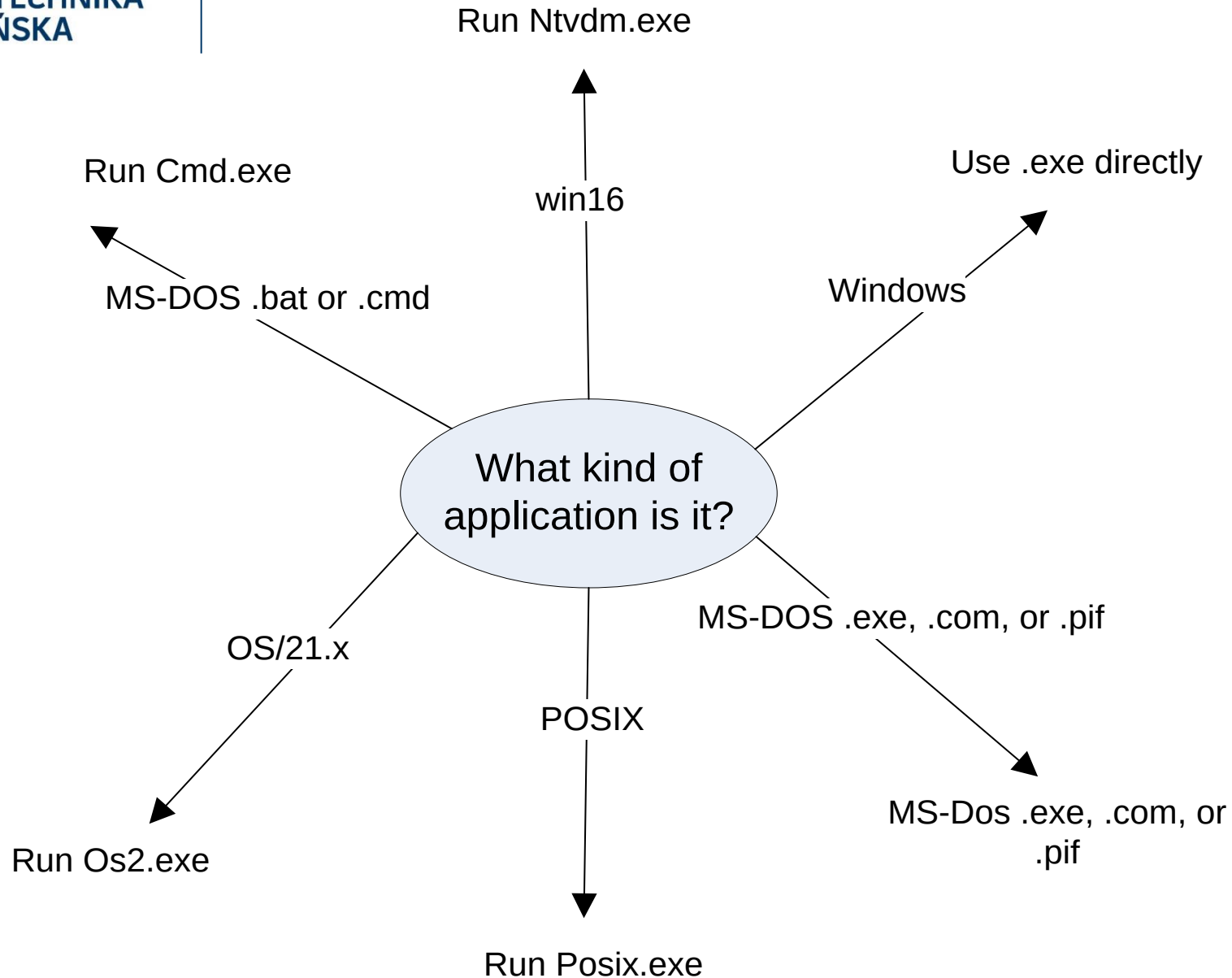
1. Otwarcie pliku.exe i utworzenie obiektów sekcji
2. Utworzenie obiektów procesu
 1. Ustawienie bloku EPROCESS
 2. Utworzenie inicjalnej przestrzeni adresowej
 3. Utworzenie bloku kernela procesu
 4. Ukończenie tworzenia przestrzeni adresowej
 5. Ustawienie PEB
 6. Ukończenie konfiguracji EPROCESS
3. Utworzenie obiektu wątku
4. Powiadomienie podsystemu (csrss) o nowym procesie
5. Utworzenie wątku głównego
6. Utworzenie kontekstu i uruchomienie wątku



Przeływ sterowania podczas tworzenia procesu

Creating process







- **Nowy** — formowanie procesu, czyli gromadzenie zasobów niezbędnych do rozpoczęcia wykonywania procesu. Po zakończeniu formatowania proces przechodzi w stan Gotowy.
- **Wykonywany** — wykonywanie instrukcji programu danego procesu i wynikająca z ich wykonywania zmiana stanu odpowiednich zasobów systemu.
- **Oczekujący** — zatrzymanie wykonywania instrukcji programu danego procesu ze względu na potrzebę przydziału dodatkowych zasobów, konieczność otrzymania danych lub osiągnięcia odpowiedniego stanu przez otoczenie procesu (np. urządzenia zewnętrzne lub inne procesy).
- **Gotowy** — oczekiwanie na przydział kwantu czasu procesora (dostępność wszystkich niezbędnych zasobów z wyjątkiem procesora).
- **Zakończony** — zakończenie wykonywania programu, zwolnienie większości zasobów i oczekiwanie na możliwość przekazania informacji o zakończeniu innym procesom lub jądra systemu operacyjnego.



Zakończenie procesu

Proces jest wykonywany dopóki nie zajdzie sytuacja:

- Dowolny wątek wykona funkcję `ExitProcess`
- Dowolny wątek wykona funkcję `TerminateProcess`
- Główny (podstawowy) wątek zwróci sterowanie do systemu (można zapobiec przez wywołanie funkcji `ExitThread`)
- Ostatni wątek procesu zakończy działanie
- Użytkownik zamyka system lub się wylogowuje



- Kiedy proces się kończy, stan obiektu związanego z procesem staje się ustawiony, zwalniając tym samym wątki, które czekały na zakończenie tego procesu.
- Otwarte handlersy do plików lub inne zasoby są zamykane automatycznie podczas zamykania procesu.
- Jednakowoż, obiekty same w sobie istnieją dopóki wszystkie otwarte handlersy do nich nie zostaną zamknięte.
- Oznacza to, że obiekty mogą pozostać ważne po zamknięciu danego procesu, jeśli inny proces posiada do nich handlersy.
- Jeśli proces jest zakończony przez użycie funkcji **ExitProcess**, system wywołuje punkt wejścia każdej podłączonej biblioteki DLL z wartością informującą, że proces odłącza się od DLL-a. Biblioteki DLL nie są informowane, gdy proces jest zakańczany przez użycie funkcji **TerminateProcess**.
- **Ostrzeżenie:** Funkcja **TerminateProcess** powinna być używana tylko w ekstremalnych przypadkach. Użycie jej nie pozwala wątkom na wyczyszczenie zasobów lub zapis danych oraz nie informuje podłączonych DLL-i o zakończeniu procesu.



Wątek to jednostka w ramach procesu, jaką Windows szereguje wśród zadań wykonawczych. Wątek musi zawierać:

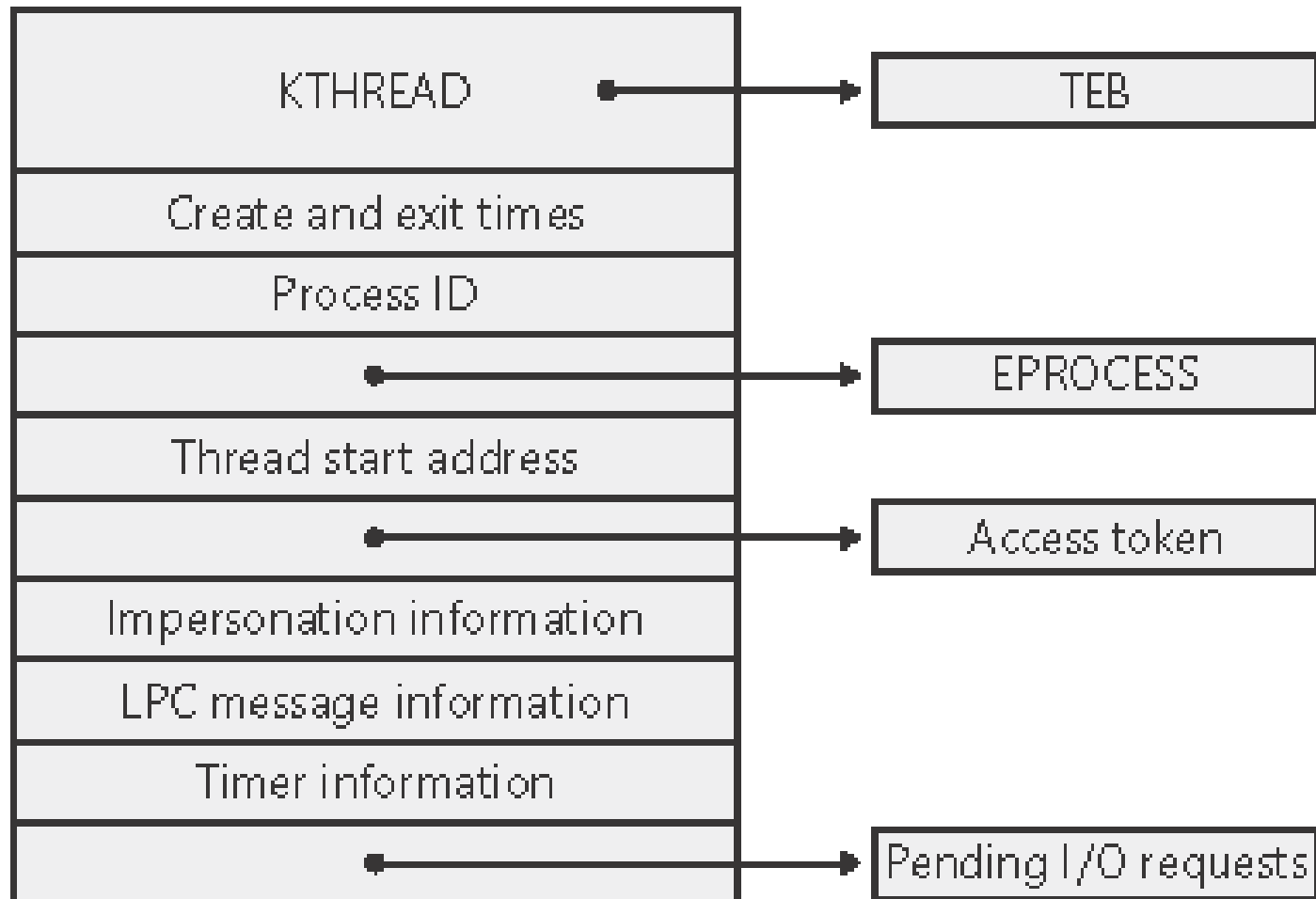
- Stan rejestrów procesora.
- Dwa stosy: jeden wykorzystywany w kernel mode a jeden w user mode.
- Prywatny obszar przechowywania TLS (thread-local storage) do wykorzystania przez różne podsystemy, biblioteki uruchomieniowe, biblioteki dynamiczne.
- Unikatowy identyfikator wątku.
- Czasem dodatkowo kontekst bezpieczeństwa - dodatkowe dane do wykorzystania w wielowątkowych aplikacjach uruchamianych na serwerach.



- Włókno bywa nazywane lekkim wątkiem i pozwala aplikacji na szeregowanie własnych wątków samodzielnie zamiast zdawać się na globalny dyspozytor.
- Włókna są niewidzialne dla kernela, ponieważ działają wyłącznie w trybie użytkownika. W odróżnieniu od wątku włókno nie zaczyna się od razu wykonywać po utworzeniu - musi być uruchomione.

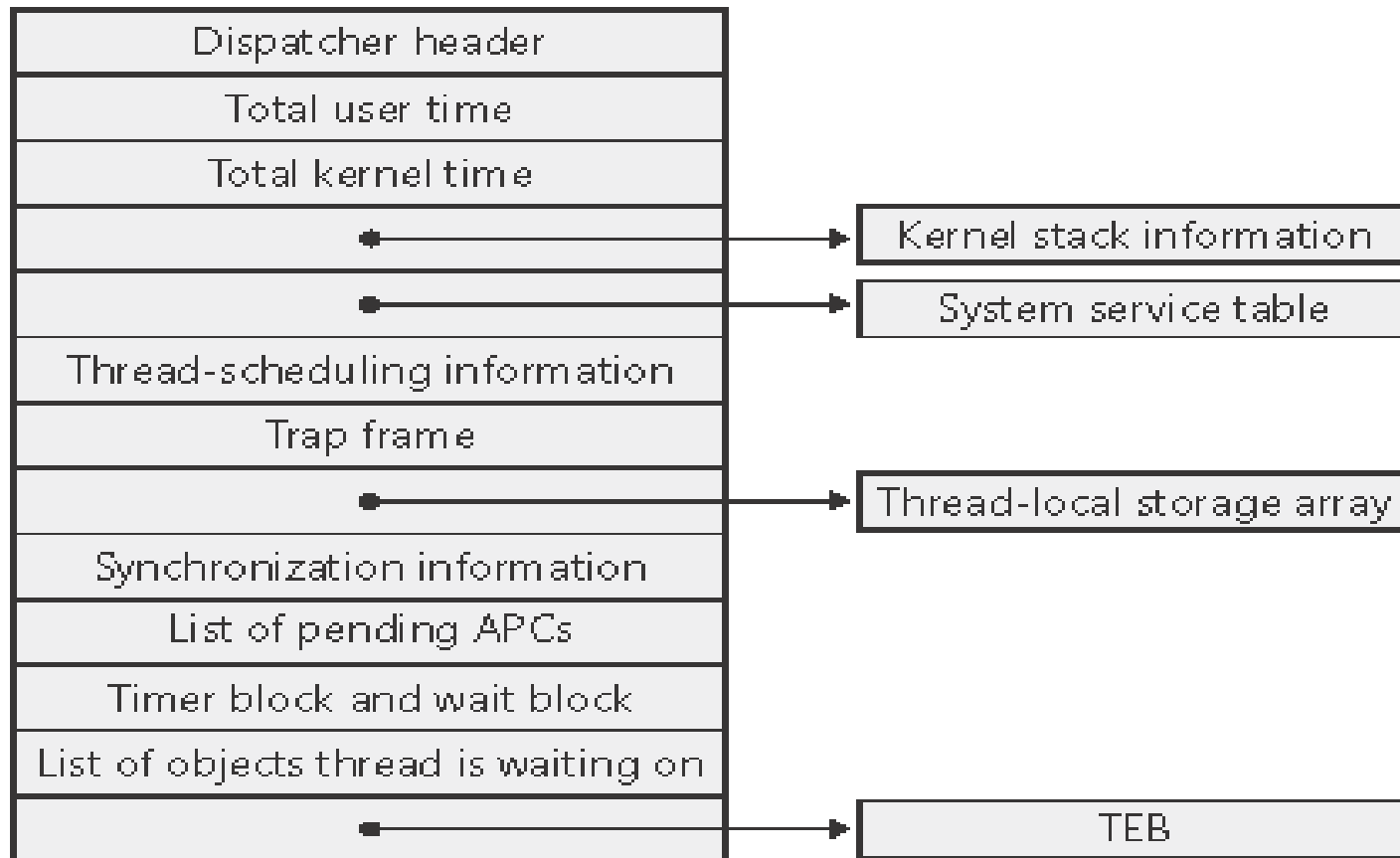


Structure of executive thread block



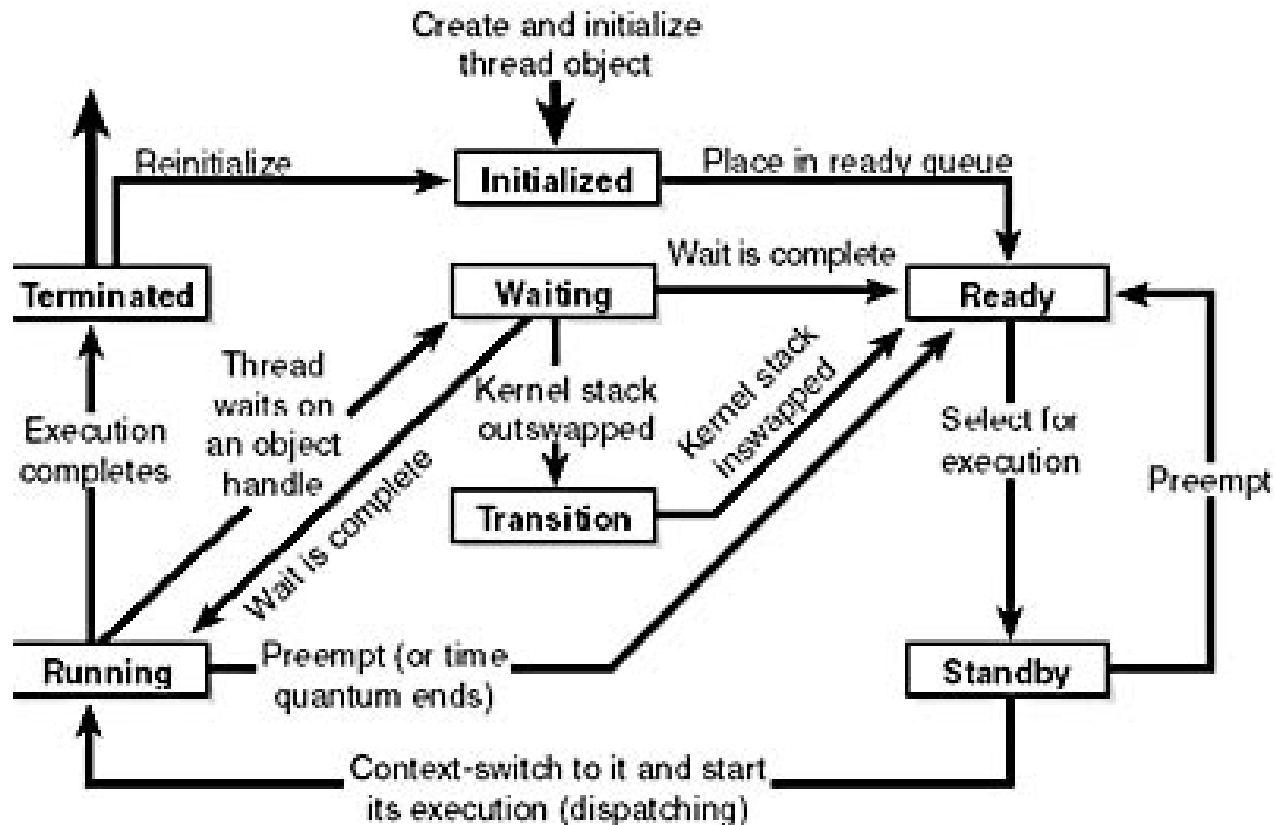


KTHREAD i TEB





Stany wątku





Stany Wątków

- `THREAD_STATE_INITIALIZED` - inicjalizacja wątku
- `THREAD_STATE_READY` - gotowość oznacza czekanie na wykonywanie. Dyspozytor szuka tylko wątków znajdujących się w tym stanie. Wątek w stanie gotowości z najwyższym priorytetem przechodzi w stan pogotowia.
- `THREAD_STATE_RUNNING` - wykonywany - procesor wykonuje jego instrukcje dopóki jądro go nie wyłączy chąc uruchomić wątek o wyższym priorytecie, jego kwant upłynął, zakończył swoje działanie lub dobrowolnie zrezygnował z wykonywania się
- `THREAD_STATE_STANDBY` - stan pogotowia. Wątek który znajduje się w tym stanie będzie wykonywany w pierwszej kolejności. W systemie wieloprocessorowym każdy procesor utrzymuje jeden wątek w stanie pogotowia.
- `THREAD_STATE_TERMINATED` - kiedy proces kończy działanie przechodzi w ten stan.
- `THREAD_STATE_WAIT` - wątek jest w stanie oczekiwania jeśli czeka na sygnał od obiektu dyspozytora. Wątek może się znaleźć w tym stanie w momencie gdy: czeka na obiekt synchronizujący jego działanie, operację wejścia-wyjścia. Po otrzymaniu przechodzi w stan gotowości
- `THREAD_STATE_TRANSITION` - stan przejściowy - gdy jądro oczekuje na zasoby niezbędne do działania systemu np. gdy następuje brak pamięci.
- `THREAD_STATE_UNKNOWN`



Tworzenie nowego wątku

1. *CreateThread* tworzy stos dla nowego wątku w przestrzeni adresowej procesu
2. *NtCreateThread* - jest wywoływane i tworzy nowy wątek w stanie zawieszonym.
3. *CreateThread* informuje podsystem win32 o nowym wątku
4. Uchwyt wątku i ID (generowane w punkcie 2) są zwracane ...
5. Jeżeli wątek nie został stworzony jako zawieszony (z ustawioną flagą `CREATE_SUSPENDED`) to wątek zostaje wznowiony



Wątek jest wykonywany dopóki nie wystąpi jedno z poniższych zdarzeń:

- Wątek wywoła funkcję **ExitThread**.
- Inny wątek procesu wywoła funkcję **ExitThread**.
- Funkcja wątku zakończy działanie i zwróci wartość.
- Dowolny wątek wywoła funkcję **TerminateThread** z parametrem w postaci handlera zamykanego wątku.
- Dowolny wątek wywoła funkcję **TerminateProcess** z parametrem w postaci handlera zamykanego procesu (proces również zostaje zniszczony).



Jeśli wątek jest zakończony przez wywołanie funkcji **ExitThread**, system wywołuje punkt wejścia funkcji każdej z powiązanych bibliotek DLL z wartością wskazującą, że wątek jest odłączany od DLLa (chyba że wywołano funkcję **DisableThreadLibraryCalls**).

Jeśli wątek jest zakończony przez wywołanie funkcji **ExitProcess**, punkty wejścia DLLa są wywoływane raz, dla wykrycia że proces jest zwalniany.

Biblioteki DLL nie są informowane gdy wątek jest zakańczany przez wywołanie funkcji **TerminateThread** lub **TerminateProcess**.

Ostrzeżenie: Funkcje **TerminateThread** i **TerminateProcess** powinny być używane tylko w ekstremalnych okolicznościach , ponieważ one nie pozwalają wątkom na wyczyszczenie się (alokacje, handlery itp.), nie informują podłączonych DLL-i i nie zwalniają stosu.



- System Windows planuje wątki za pomocą wielopoziomowych kolejek ze sprzężeniem zwrotnym - umożliwia to przemieszczanie się procesów między kolejkami.
- Pomysł polega na rozdzieleniu procesów o różnym zapotrzebowaniu na moc procesora. Jeśli proces zużywa za dużo czasu procesora, to zostanie przeniesiony do kolejki o niższym priorytecie.
-
- Planista wielopoziomowych kolejek ze sprzężeniem zwrotnym jest określony za pomocą następujących parametrów:
 - liczby kolejek
 - algorytmu planowania dla każdej kolejki
 - metody użytej do decydowania o awansowaniu procesu do kolejki o wyższym priorytecie
 - metody wyznaczającej kolejkę, do której trafia proces potrzebujący obsługi



Planowanie zadań (scheduling)(2)

- Windows przydziela czas na poziomie wątków (thread granularity)
- Planista (dyzpozytor) bazuje na wywłaszczaniu w oparciu o priorytety
- Wpierw wykonywane są wątki o najwyższych priorytetach – algorytm round robin z priorytetami.
- Windows XP wykorzystuje 32 (31) poziomy priorytetów System level (0), Variable levels (1-15), Real-time levels (16-31)
- Z punktu widzenia Win32 API
 - Procesy - Idle, Below Normal, Normal, Above Normal, High, Real-time
 - Wątki - Idle, Lowest, Below-Normal, Normal, Above Normal, Highest, Time-Critical

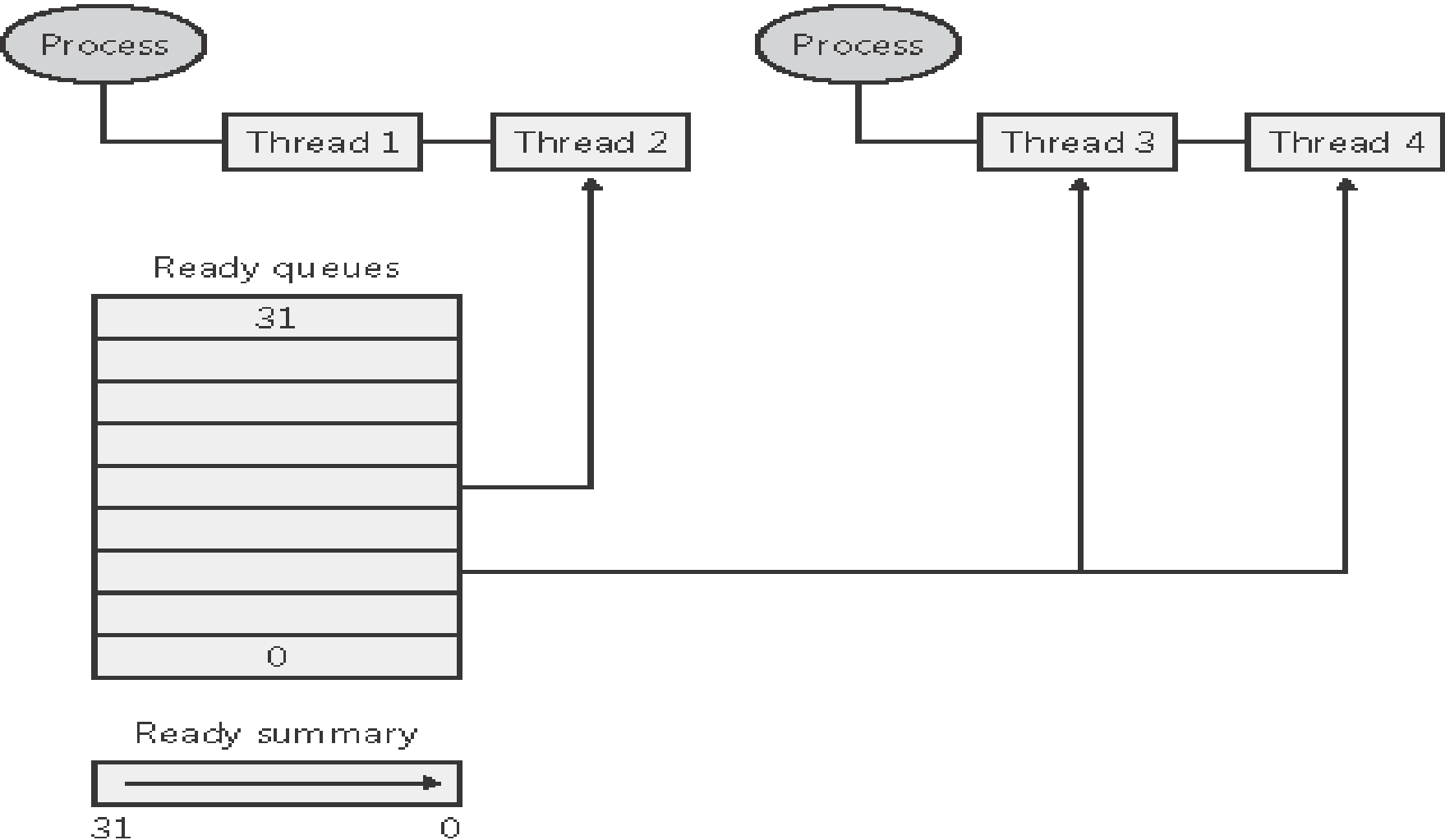


Przydzielanie czasu procesora dla wątku

- System przydzielania czasu z wyłączeniem opartym na priorytetach (wątek o najwyższym priorytecie działa stale)
- Po upływie kwantu (jednostkowego czasu działania wątku) system sprawdza czy inny wątek o tym samym lub wyższym priorytecie nie czeka na uruchomienie lub czy nie należy obniżyć priorytetu wątku
- Wątek może zostać wyłączony przed upływem wyznaczonego czasu przez wątek o wyższym priorytecie.
- Kod przydzielania czasu procesora jest zaimplementowany w różnych modułach jądra. Procedury odpowiedzialne za to zadanie nazywane są wspólnym terminem **dyspozytor**



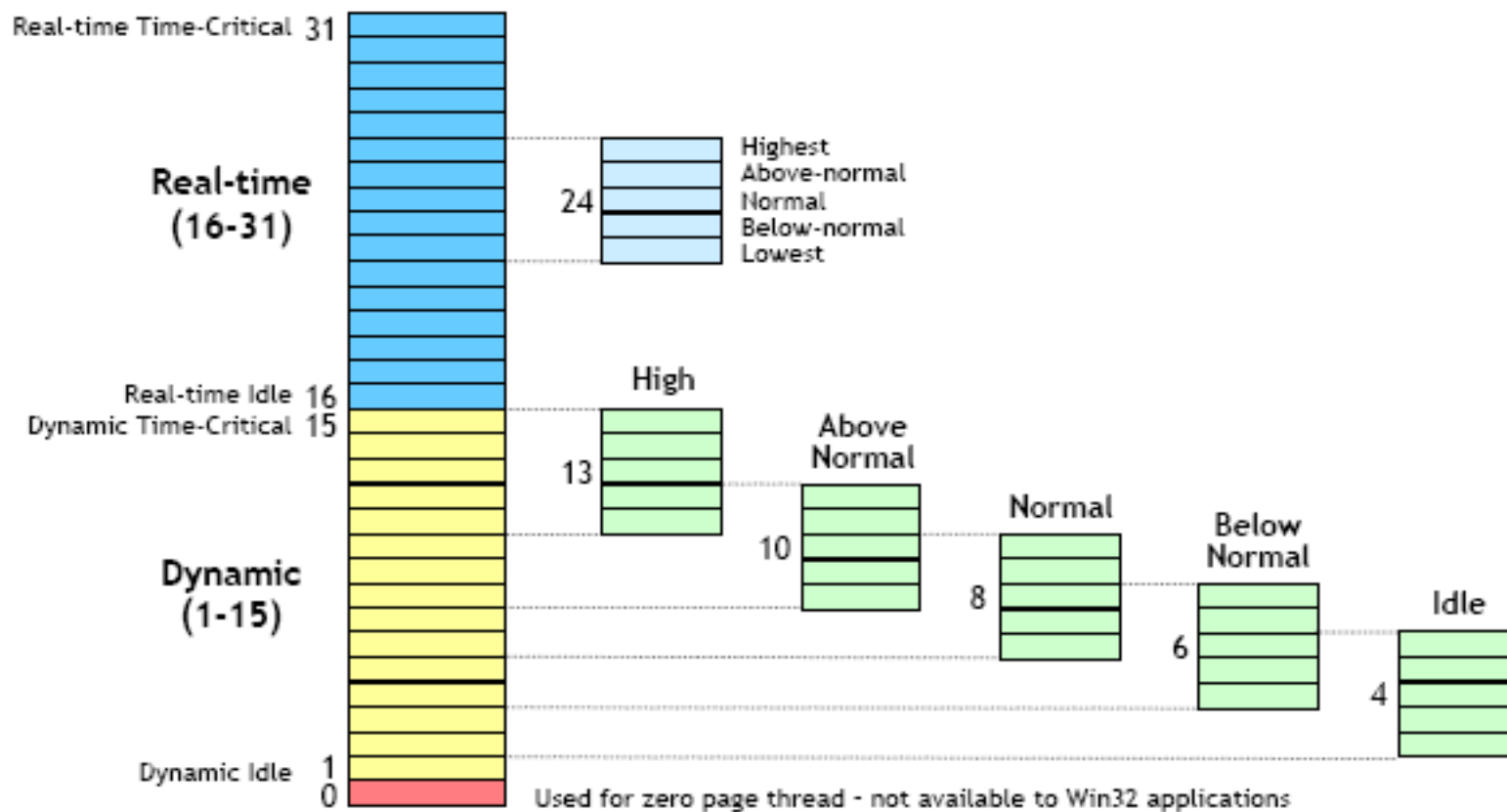
Dyspozytor (Dispatcher Database)

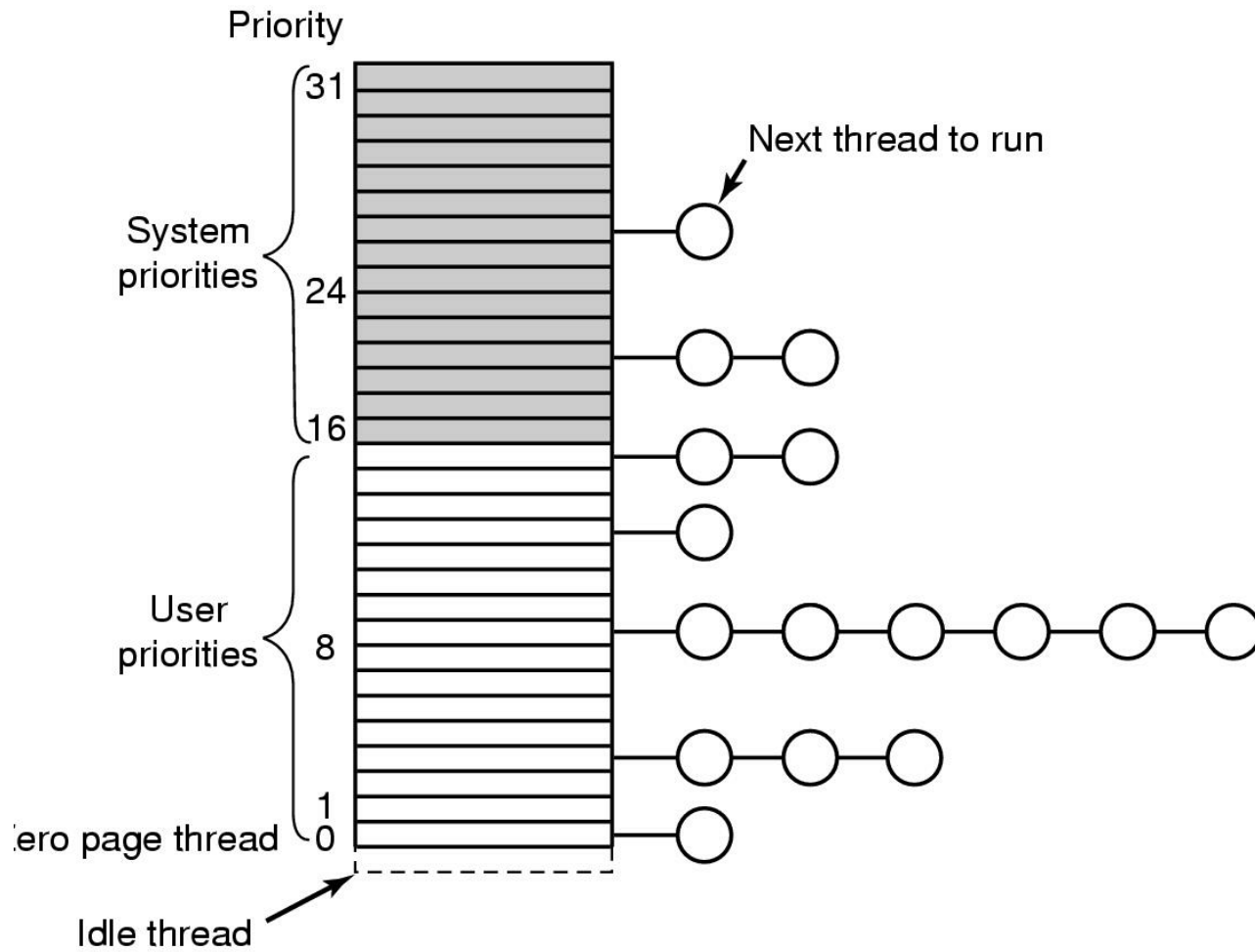




Poziomy priorytetu wątków

- Windows używa 32 poziomów priorytetu wątków (0-31)
 - szesnaście poziomów czasu rzeczywistego (16-31)
 - piętnaście poziomów zmiennych (1-15)
 - jeden poziom systemowy (0) zarezerwowany dla wątku podstawowego
- Nowy wątek dziedziczy priorytet procesu (staje się on jego priorytetem bazowym)
- Wątek ma dwa poziomy priorytetu:
 - bieżący
 - bazowy







- Kwant czasu jest jednostką czasu przez który działa wątek zanim system sprawdzi czy inny wątek o tym samym priorytecie może być uruchomiony
- W Windows 2000 Professional i Windows XP, wątki działają domyślnie przez 2 interwały czasowe, w Windows Server 12.
- Odmierzanie kwantu
 - Wartości kwantu są trzykrotną wielkością liczby przerwania zegarowych (6 lub 12).
 - Pozwala to na pewne rozbieżności tej wielkości w celu lepszego zarządzania czasem (np.. Boosting)

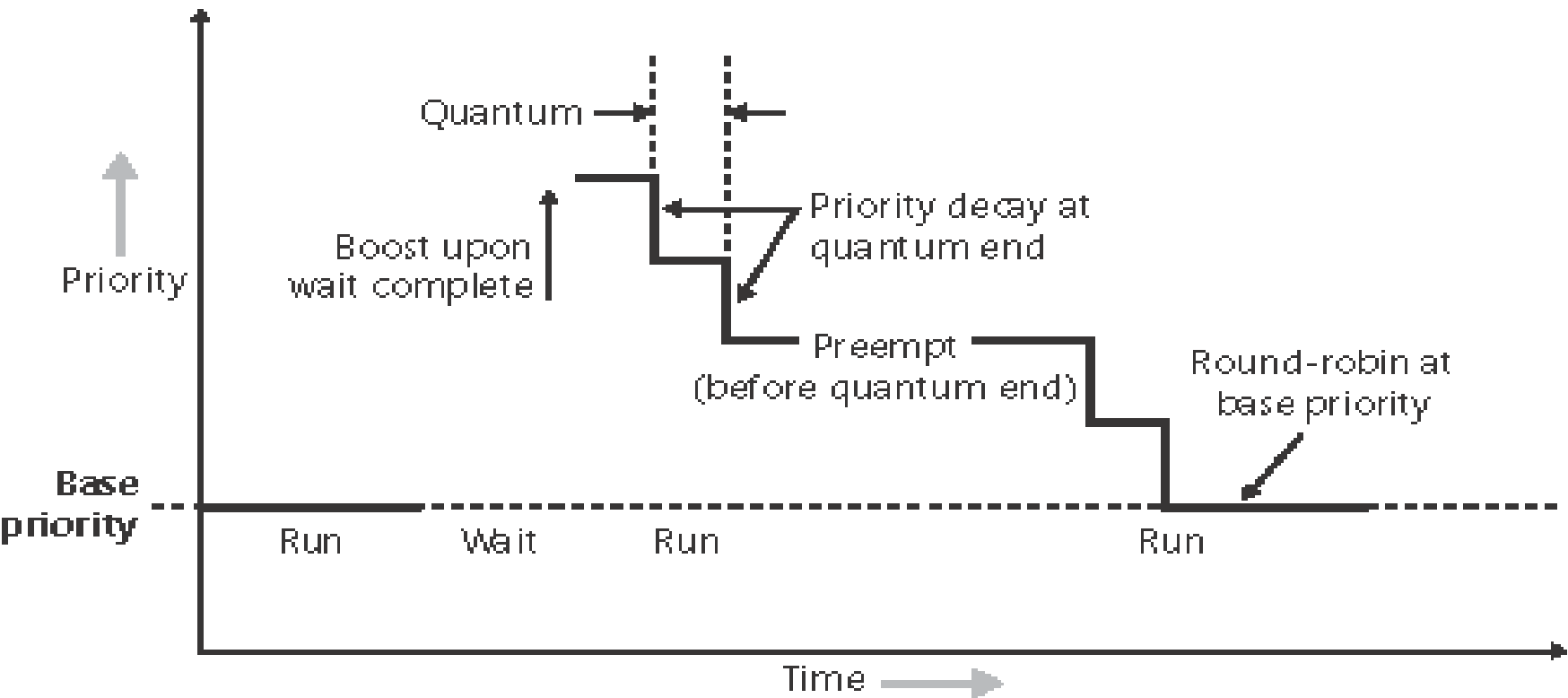


W pięciu przypadkach Windows może zwiększyć priorytet wątku:

- Po zakończeniu operacji I/O – wartość zwiększenia priorytetu zależy od sterownika (od +1 dla myszki do +8 dla obsługi dźwięku). Co 1 okres zmniejsza się o 1 aż powróci do bazowego
- Po odczekaniu na zdarzenie bądź semafor – priorytet zwiększa się o 1. Po wykorzystanym kwancie powraca do bazowego
- Gdy proces z pierwszego planu zakończył funkcję typu wait – priorytet zwiększony o wartość zmiennej jądra PsPrioritySeparation (0, 1 lub 2)
- Gdy wątki GUI budzą się z powodu aktywności okienek – priorytet zwiększa się o 2
- Proces gotowy do działania nie działał od dłuższego czasu (4s) – priorytet ustawiany na 15 i podwojenie wartości kwantu. Po wykorzystaniu czasu wraca do normy.

•

W żadnym z tych przypadków wartość procesu nie może przekroczyć 15.





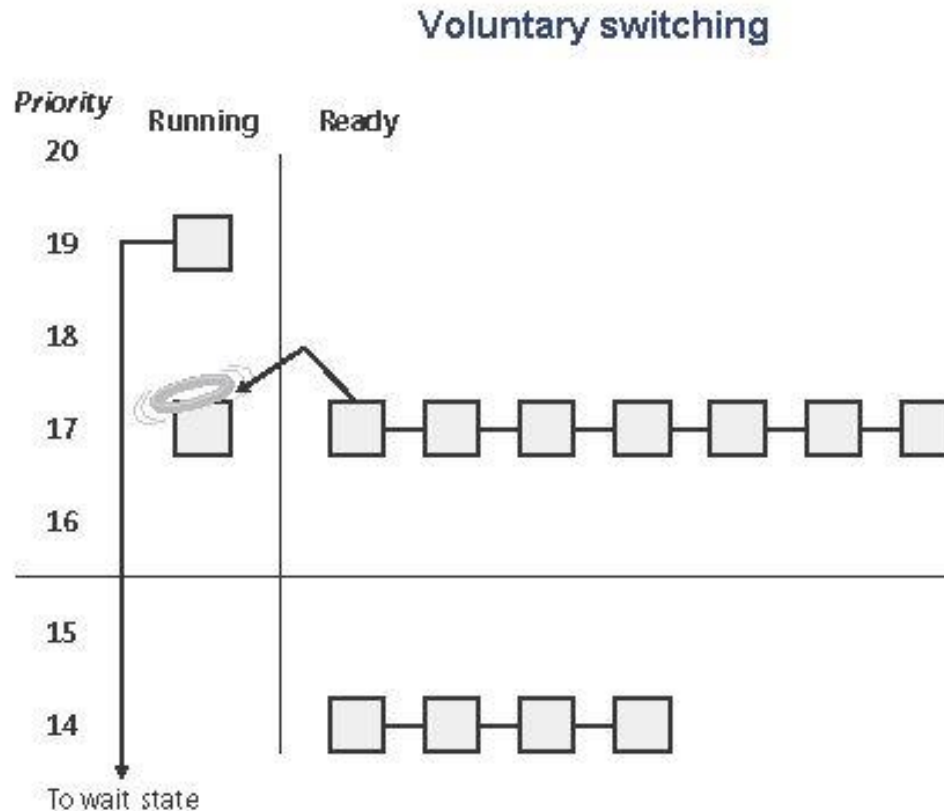
- Quantum Settings Registry Value
 - HKLM\SYSTEM\CurrentControlSet\Control\Priority\Win32PrioritySeparation





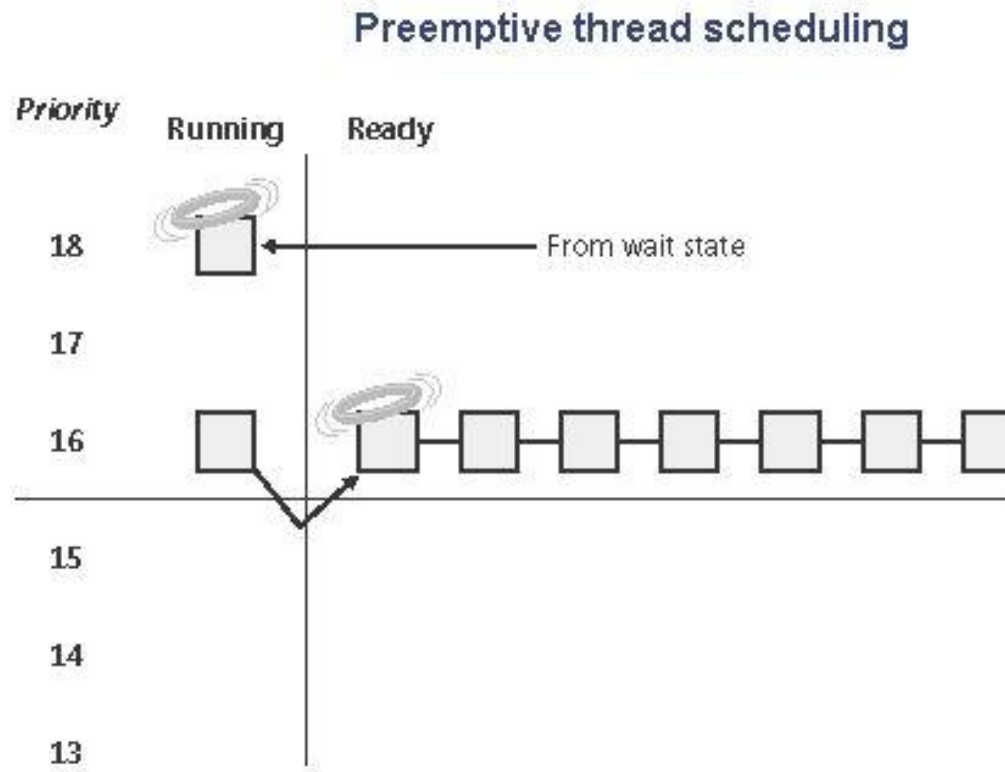
Taktyki szeregowania

- **Dobrowolne przełączenie** - Wątek może zwolnić dobrowolnie procesor przez przejście w stan oczekiwania na coś dzięki użyciu funkcji typu wait. Kwant czasu dla procesu nie jest resetowany tylko zmniejszany o 1/3 kwantu (tj. 1 jednostkę - pamiętamy, że wewnątrz kwantu składa się z 3 jednostek).





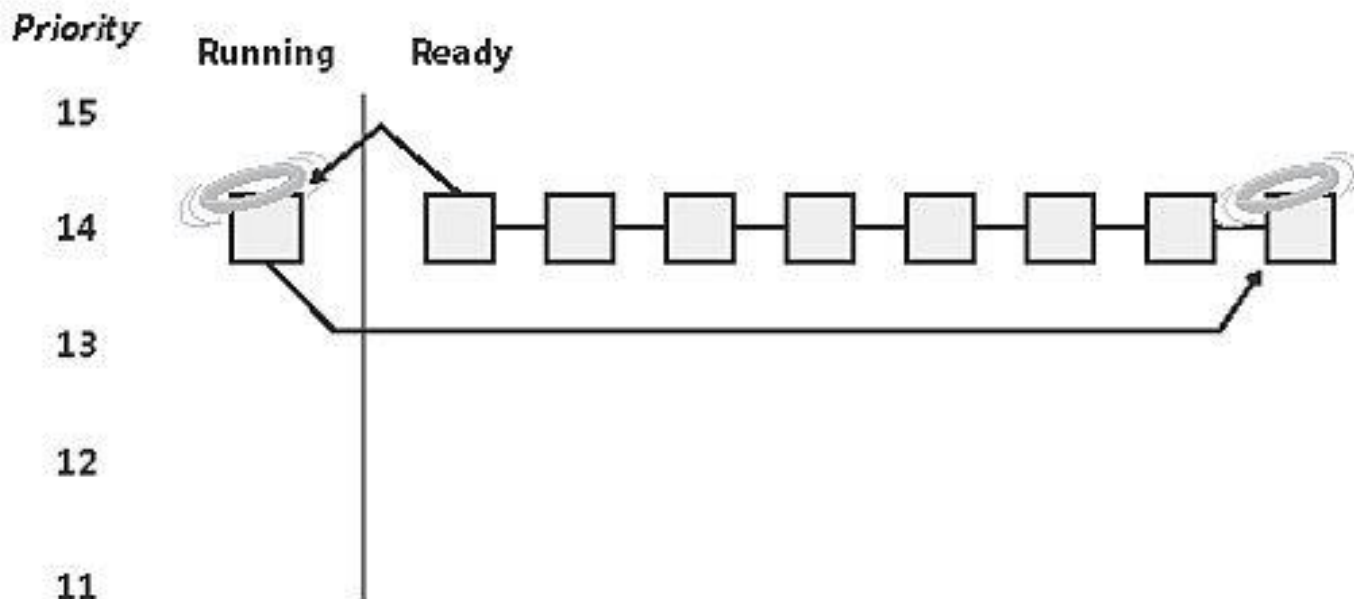
- **Wywłaszczenie** - W tym scenariuszu wątek jest wywłaszczany, gdyż skończyło się oczekiwanie wątku o wyższym priorytecie lub zostały dokonane odpowiednie zmiany w priorytetach wątków. Wywłaszczony wątek jest umieszczany na początku stosownej kolejki:





- **Koniec przydzielonego kwantu** - Gdy wątkowi aktywnemu skończy się kwant, Windows określa czy jego priorytet nie powinien ulec zmniejszeniu i czy inny wątek ma być zaszeregowany do procesora.

Quantum end thread scheduling



- **Zakończenie** - Gdy wątek kończy działanie jest usuwany z kolejki wątków. Jeśli nie ma otwartych dowiązań, to wszystkie obszary pamięci z nim związane są zwalniane.



CreateProcess – tworzy nowy proces

CreateProcessAsUser – tworzy proces jako użytkownik

OpenProcess – zwraca uchwyt do wybranego procesu

ExitProcess – zamyka proces

TerminateProcess – zabija proces

CreateThread – tworzy nowy wątek

ExitThread – kończy wątek

SuspendThread – zawiesza wątek

TerminateThread – zabija wątek

ConvertFiberToThread konwertuje włókno na wątek

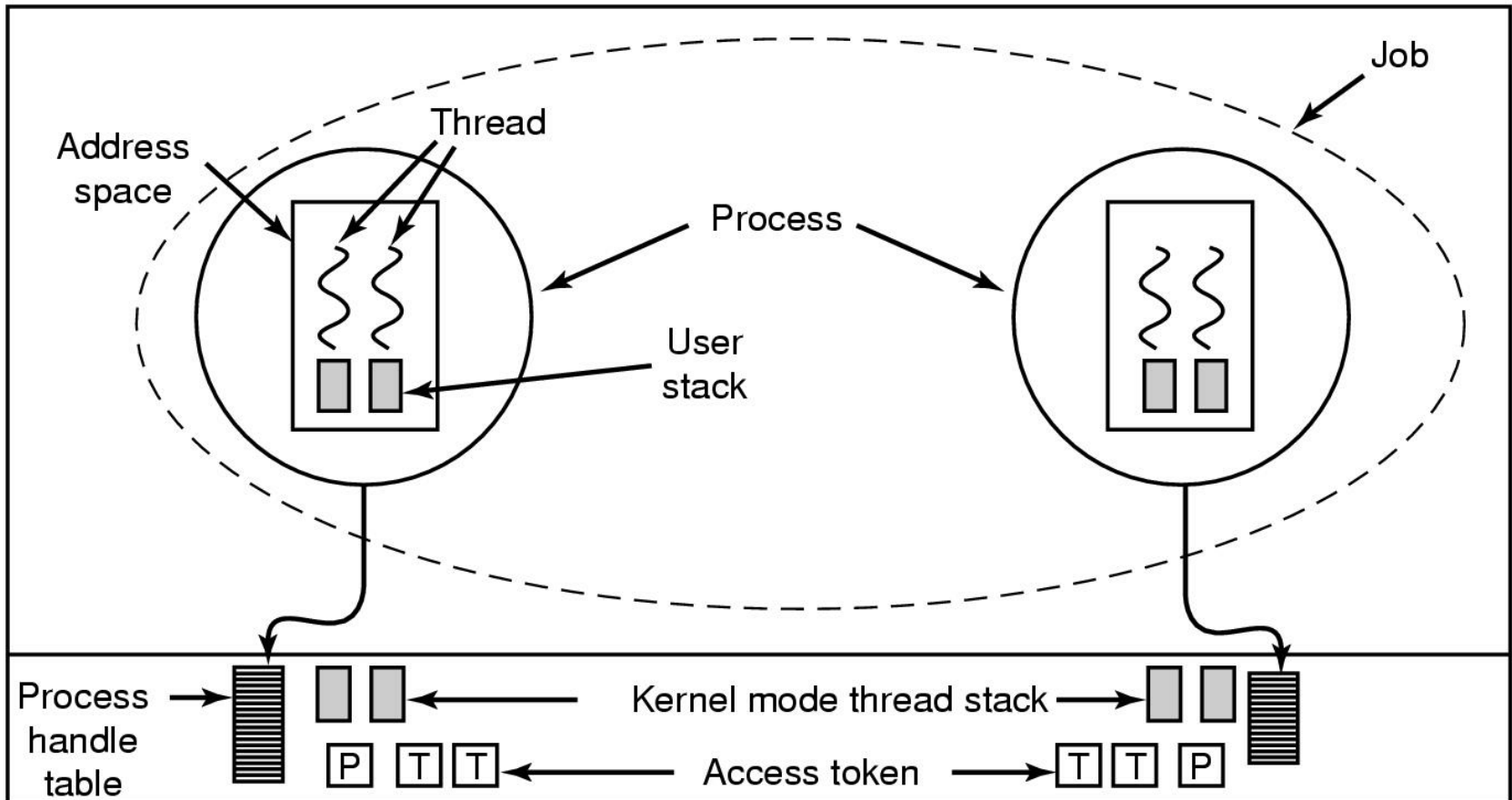
ConvertThreadToFiber – konwertuje wątek na włókno

CreateFiber – tworzy nowe włókno

DeleteFiber – usuwa włókno

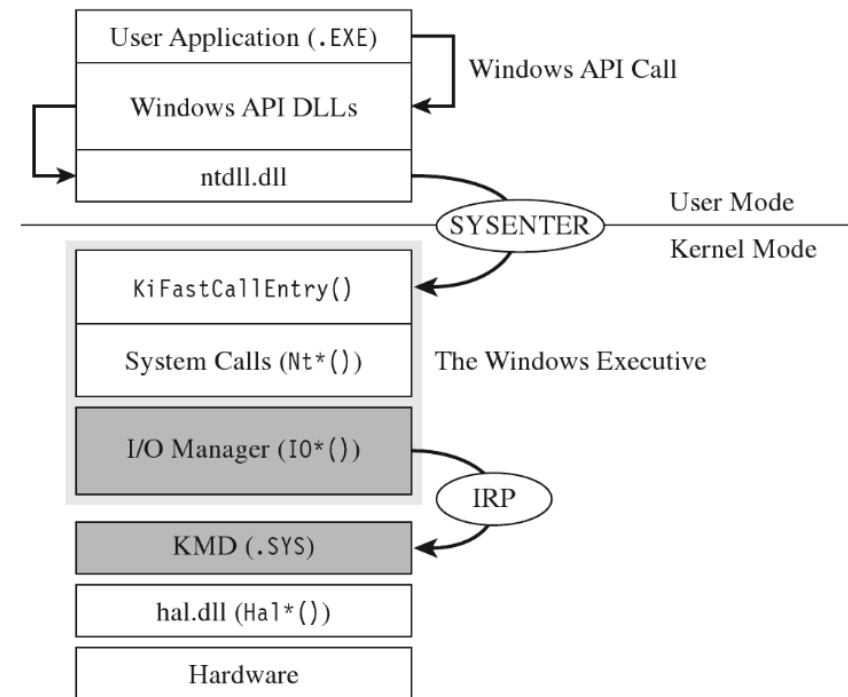
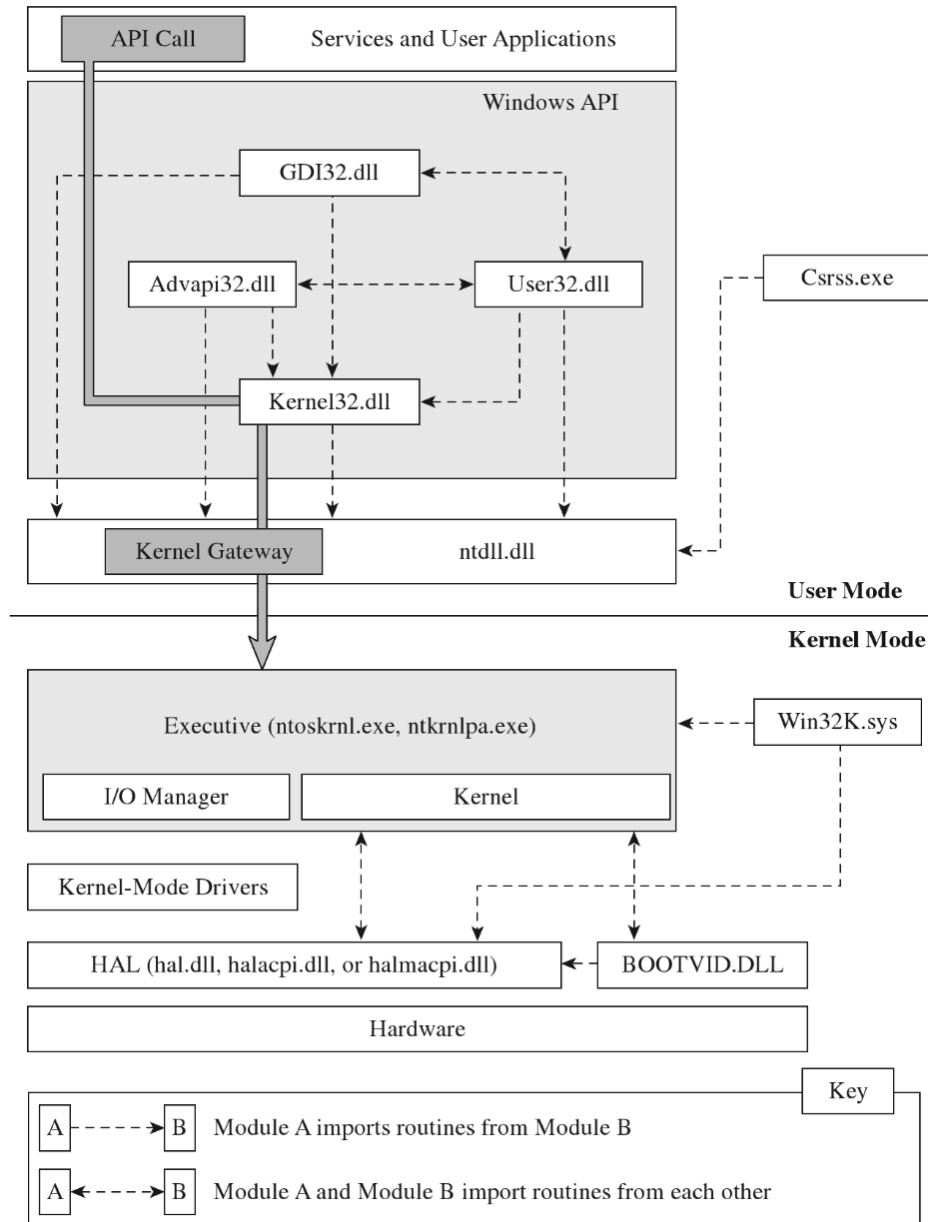


Zadania (jobs)



Sterowniki (sprzętowe ?)
w systemie Windows

Sterowniki systemu Windows KDM



Kroki do wykonania KDM

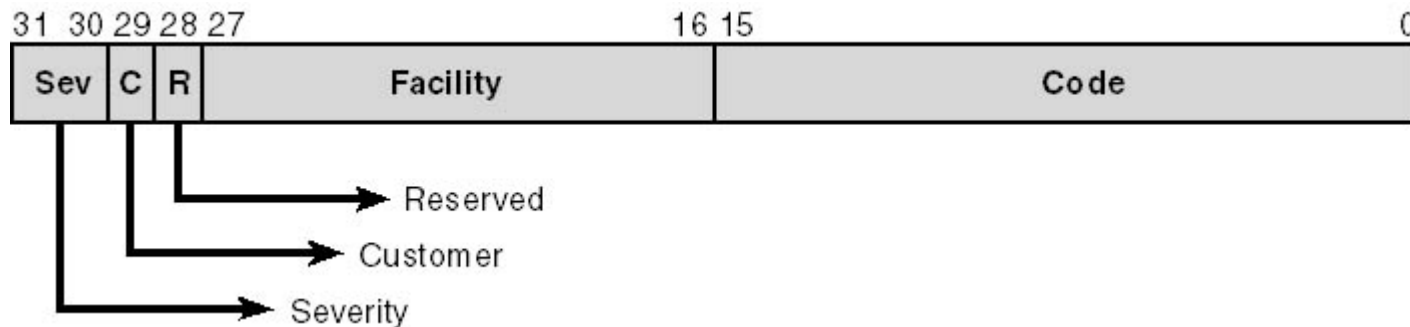
- Kod sterownika
- Kod aplikacji wywołującej funkcje sterownika
- Wymagany jest wspólny identyfikator, dzięki któremu aplikacja wyszuka sterownik (proces rejestracji obiektu sterownika w systemie)
- Sterownik trzeba zainstalować w systemie

Minimalny kod sterownika

```
#include "ntddk.h"
#include "dbgmsg.h"
VOID Unload (IN PDRIVER_OBJECT pDriverObject)
{
    DBG_TRACE("OnUnload", "Received signal to unload the driver");
    return;
}

NTSTATUS DriverEntry
(IN PDRIVER_OBJECT pDriverObject, IN PUNICODE_STRING regPath)
{
    DBG_TRACE("Driver Entry", "Driver has been loaded");
    (*pDriverObject).DriverUnload = Unload;
    return(STATUS_SUCCESS);
}
```

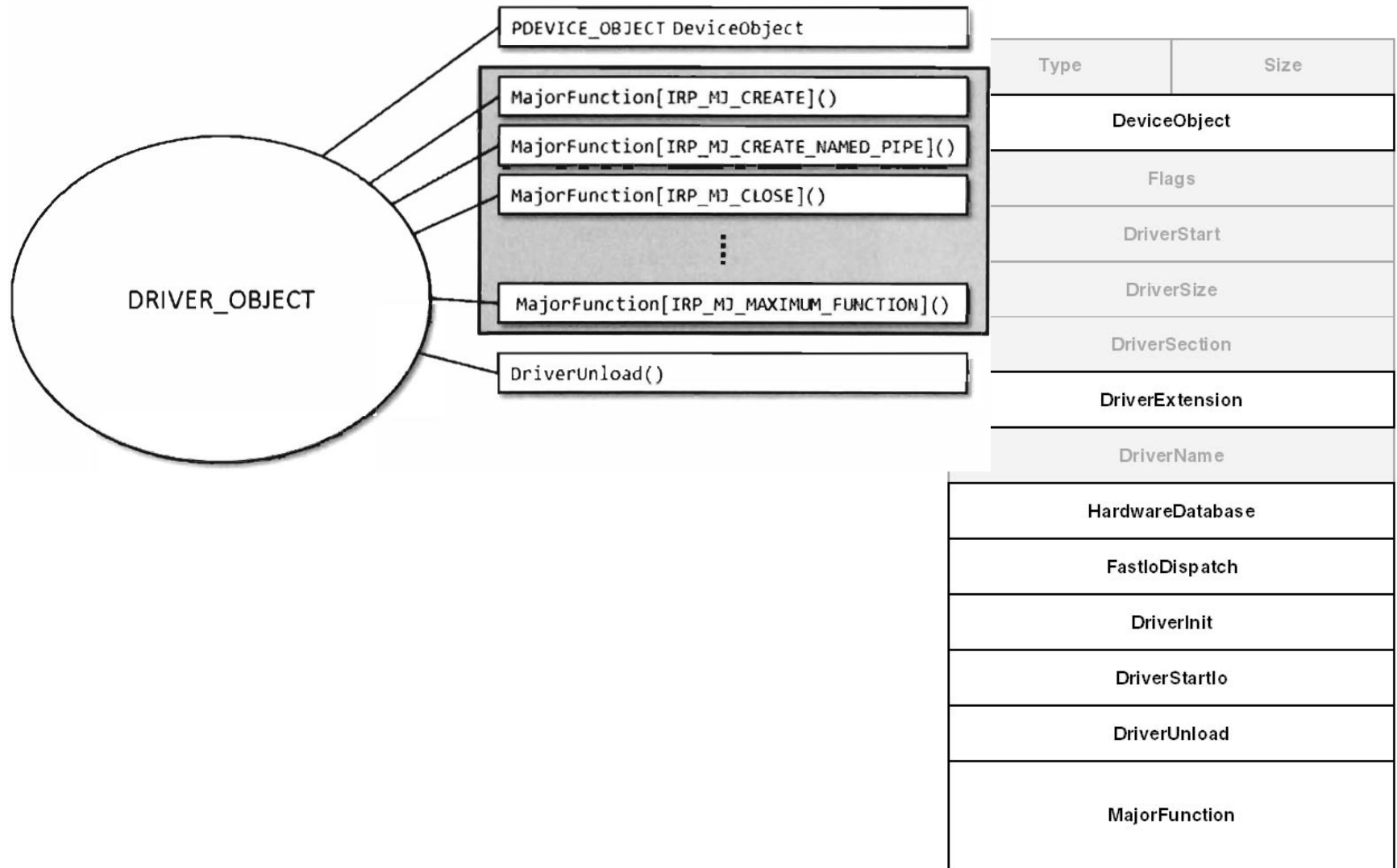
Kod powrotu



- **Sev** - kod powagi błędu
 - 00 - Success
 - 01 - Informational
 - 10 - Warning
 - 11 - Error
- **C** - jeśli wartość jest zdefiniowana przez użytkownika
- **Facility** - klasa błędu
- **Code** - Kod błędu

NT_SUCCESS(Status)
NT_INFORMATION(Status)
NT_WARNING(Status)
NT_ERROR(Status)

Struktura DRIVER_OBJECT



DeviceIoControl

```
(  
    hDeviceFile,  
    (DWORD)IOCTL_XXX_CMD,  
    (LPVOID)inBuffer,  
    nBufferSize,  
    (LPVOID)outBuffer,  
    nBufferSize,  
    &bytesRead,  
    NULL  
);
```

kernel32.dll

ntdll.dll

SYSENTER

User Mode

Kernel Mode

System Calls (Nt*())

I/O Manager (Io*())

IRP

```
inputBuffer      = (*pIRP).AssociatedIrp.SystemBuffer;  
outputBuffer     = (*pIRP).AssociatedIrp.SystemBuffer;  
irpStack         = IoGetCurrentIrpStackLocation(pIRP);  
inputBufferLength = (*irpStack).Parameters.DeviceIoControl.InputBufferLength;  
outputBufferLength = (*irpStack).Parameters.DeviceIoControl.OutputBufferLength;  
ioctrlcode       = (*irpStack).Parameters.DeviceIoControl.IoControlCode;
```

hal.dll (Hal*())

Hardware

Instalacja sterownika w systemie

- Wykorzystanie Service Control Manager'a
- Wykorzystanie wywołania `ZwSetSystemInformation()`
- Zapis do `\Device\PhysicalMemory`

SCM

```
@echo off
setlocal
set CREATE_OPTIONS= type= kernel start=
demand error= normal DisplayName=
driver3
sc.exe create srv3 binpath=
%windir%\System32\drivers\driver3.sys
%CREATE_OPTIONS%
sc.exe description driver3 "My OS Driver"
end local
```

ZwSetSystemInformation

- Wymagane jest pozyskanie adresu funkcji z ntdll.dll za pomocą `GetProcAddress`

```
ZwSetSystemInformation  
(LOAD_DRIVER_IMAGE_CODE,  
&DriverName,  
sizeof(DRIVER_NAME));
```

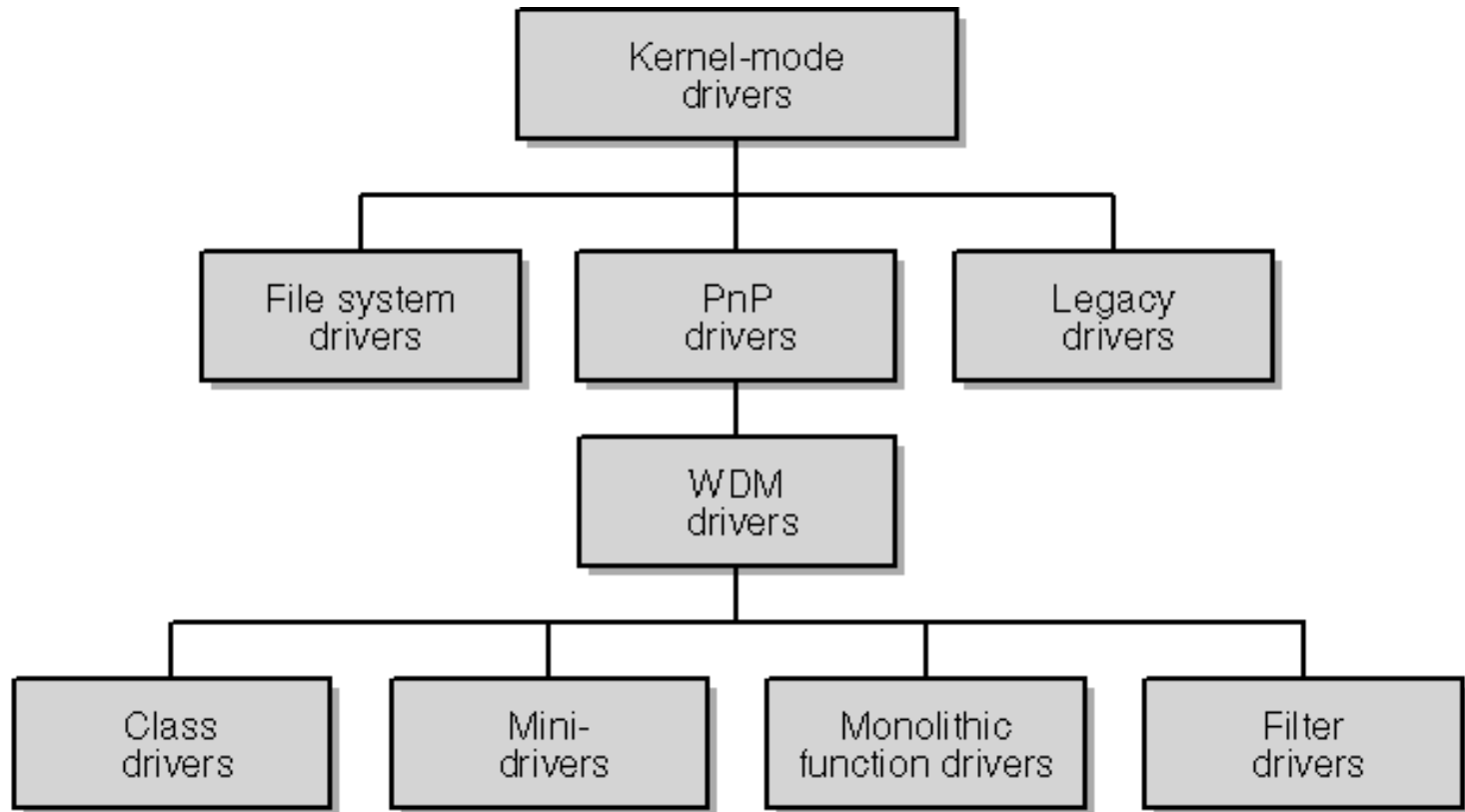
Ex - General executive routines
Exp - Executive private (not exported)
Cc - Cache Manager (Controller)
Mm - Memory Manager
Rtl - General runtime library
FsRtl - file system runtime library
Ob - object management
Io - I/O subsystem
Se - Security
Ps - Process structure
Po - Power management
Wmi - Windows Management Instrumentation
Zw - File and registry access
Ke - General Kernel
Ki - Kernel internal (not available outside of kernel)
Hal - hardware abstraction layer
READ_xxx, WRITE_xxx - I/O port and register access (HAL)

Nazwy obiektów

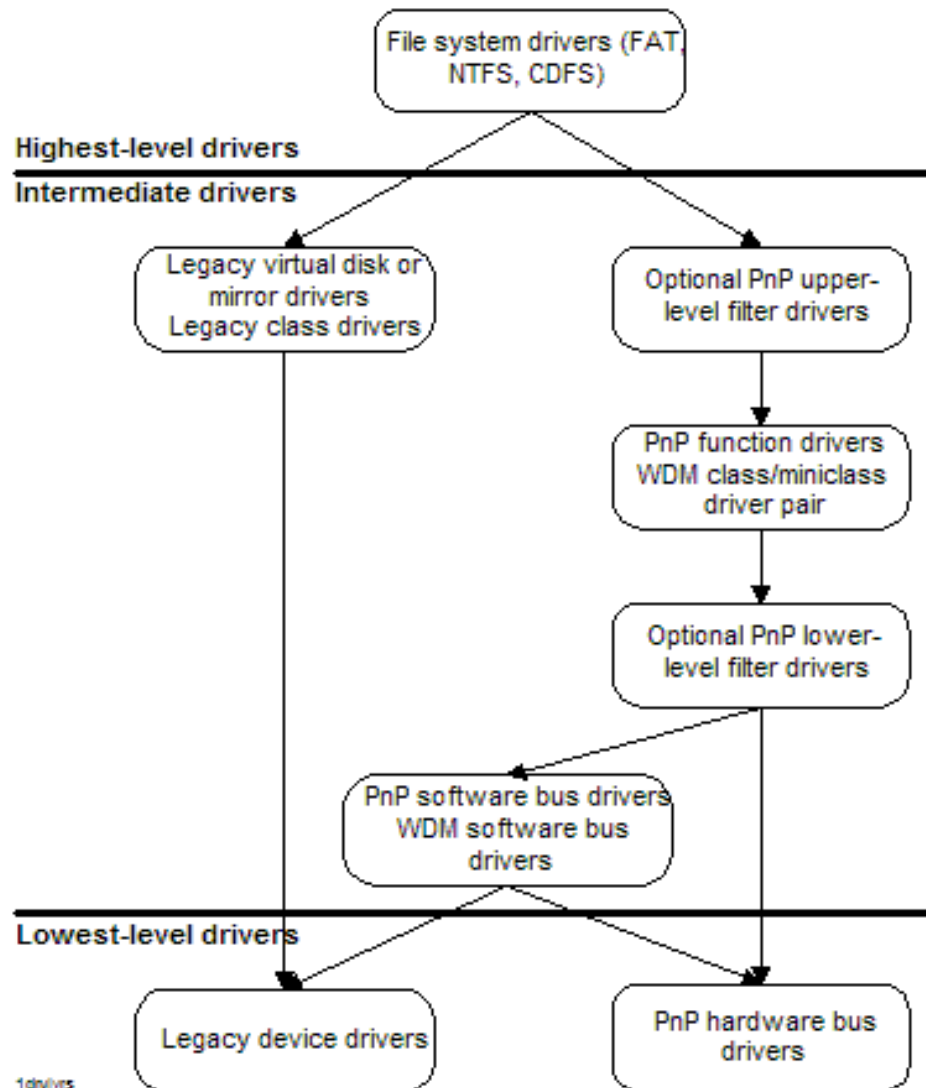
- Użytkownik widzi tylko obiektu w katalogach `\BaseNamedObjects` i `\??` (Global)
 - Obiekty rezydujące w `\BaseNamedObjects` są globalne i tworzone przez aplikacje użytkownika
 - KDM umieszcza łącza symboliczne w `\??`
- Katalog wszystkich obiektów
 - Struktura zarządzana przez Object Manager'a.
 - Tworzenie obiektów w katalogu z poziomu kernela za pomocą `ZwCreateDirectoryObject`
- Łącza symboliczne
 - `\??\A:` `\Device\Floppy0`
 - `\??\COM1` `\Device\Serial0`
 - Dostęp z poziomu użytkownika `QueryDosDevice`

Sterowniki w systemie Windows

Klasyfikacja sterowników dla Windows XP



Klasyfikacja warstwowa sterowników



Sterowniki WDM

- Sterowniki WDM (Windows Driver Model) to sterowniki Pnp, które dodatkowo implementują protokoły zarządzania energią i są zgodne zarówno z Windows 98/ME i Windows 2K/XP.
- To definicja. W praktyce sterowniki WDM nie zawsze oferują systemowi operacyjnemu wszystkie funkcje

Sterowniki WDM

- Sterowniki klas: Łączą wspólną funkcjonalność dobrze zdefiniowanych klas urządzeń (np. kart sieciowych - NDIS class drivers). Producenci sprzętu w większości nie piszą własnych sterowników klas, ale używają jednego z dostarczonych przez Microsoft.
- Minidriver: Pisane przez producentów jako wsparcie dla sterowników klas.
- Sterowniki Filtrów (filter driver): Dodają i/lub modyfikują zachowanie urządzenia
- Sterowniki monolityczne (Monolithic function driver): Jedne sterownik dla całego urządzenia (systemy typu embedded)

Klasyfikacja sterowników dla Windows XP

- Sterowniki systemu plików: implementują system plików dla dysku lokalnego lub podłączonego sieciowo.
- Sterowniki typu Legacy: Kontrolują sprzęt bez żadnego wsparcia od strony systemu operacyjnego (Sterowniki dla nieistniejącego sprzętu są dodane do systemu właśnie jako legacy drivers)
- Sterowniki Pnp: Implementują protokół Plug and Play.

Sterowniki urządzeń dla Windows

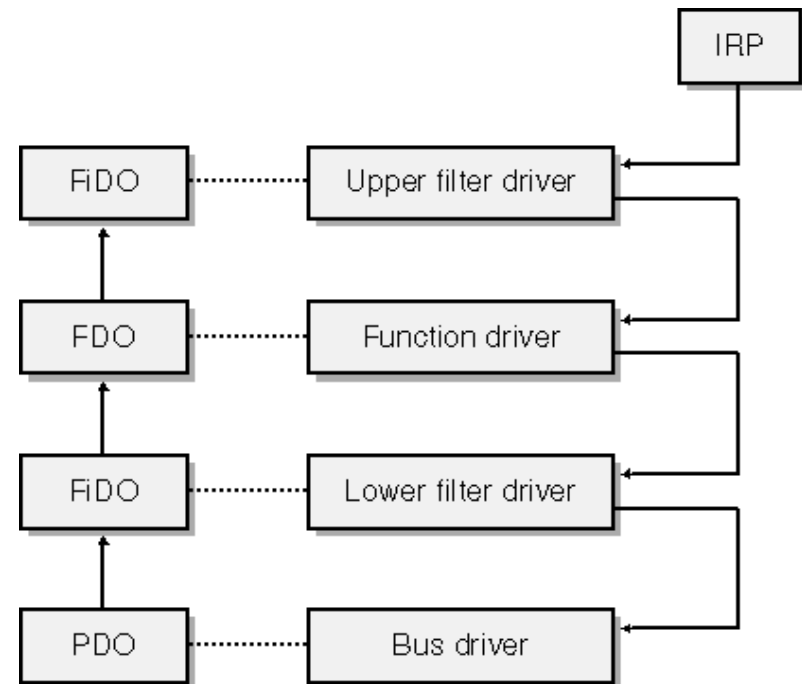
- Rzeczywisty plik sterownika, który zawiera realizację funkcjonalności urządzenia ma rozszerzenie .sys. Plik ten jest uzyskiwany po kompilacji kodu źródłowego przy użyciu pakietu DDK.
- Dla sterowników wymagany jest jeszcze plik .inf umożliwiający zainstalowanie pliku w systemie.
- Plik Inf: plik tekstowy z informacjami o urządzeniu (urządzeniach) i plikach, które należy dodać do systemu. Używany przez Win32 Installer.

Device extension i stos sterowników

- Dane specyficzne dla urządzenia (stan urządzenia, wykorzystane przez nie zasoby np. pamięć, uchwyty) są utrzymywane w strukturze Device extension.
- Warstwowe ułożenie sterowników: Przychodzące pakiety IRP są przetwarzane przez każdą pojedynczą warstwę (sterownik) i przekazywane do następnego sterownika. Jest to tzw. Stos komunikatów dla sterowników → IRP stack location

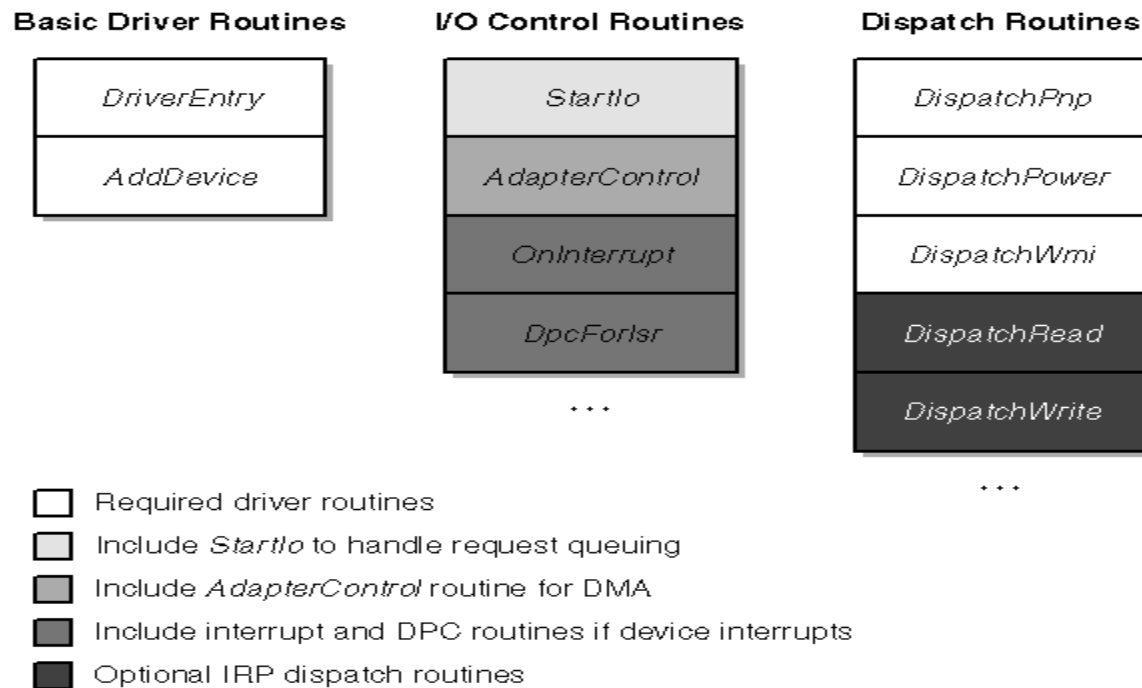
Warstwy sterowników

- Linie wykropkowane pokazują połączenia logiczne pomiędzy warstwami sterowników a strukturami device object.
- Sterowniki funkcji: Sterownik który faktycznie zarządza sprzętem.
- Sterownik magistrali (Bus driver): odpowiedzialny za zarządzanie połączeniem pomiędzy sprzętem a systemem.
- Sterowniki filtrów: modyfikują żądanie IRP (na dowolnej ścieżce)



Procedury sterownika WDM

- Architektura sterownika i obowiązkowe procedury sterownika zależą od typu urządzenia

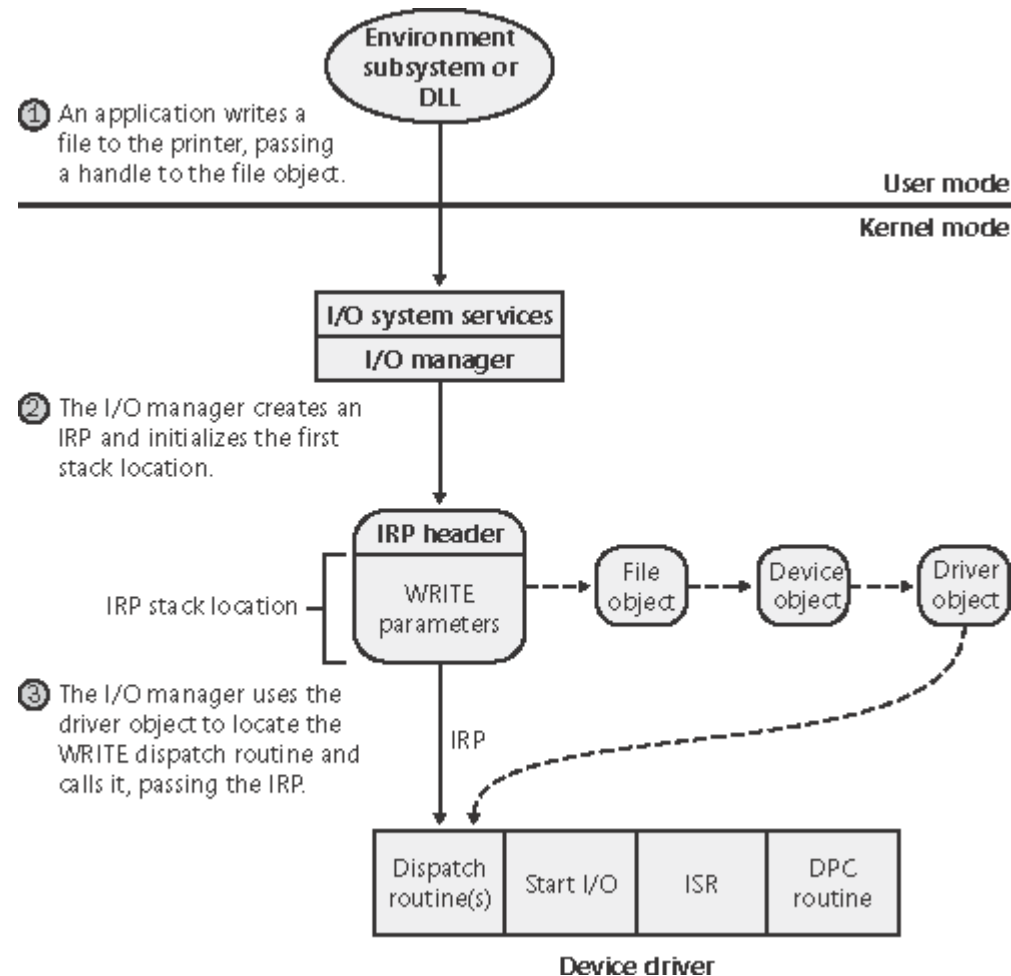


Procedury obowiązkowe

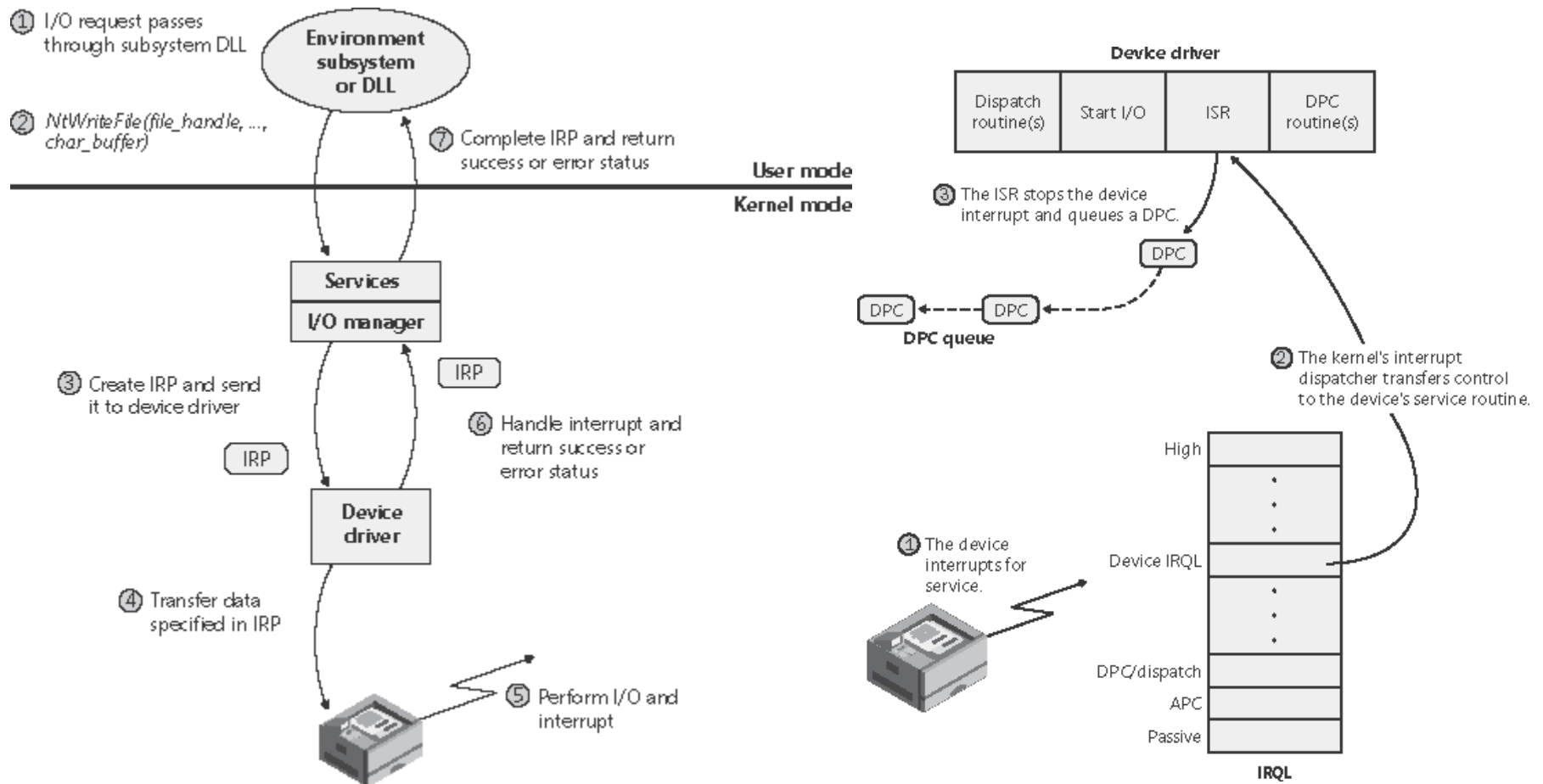
- **DriverEntry:** Funkcja inicjalizująca działanie sterownika. Jej zadanie to wskazanie SO, gdzie znajdują się niezbędne funkcje (np. AddDevice). W praktyce oznacza to wskazanie SO przez sterownik, które funkcje może użytkownik (a raczej jego aplikacja) wykonać.
- **AddDevice:**
 - Tworzy obiekt urządzenia (device object) oraz instancję rozszerzenia urządzenia (device extension)
 - Rejestruje jedno lub więcej interfejsów urządzeń
 - Wpisuje device extension i flagi (flags member) do obiektu urządzenia (device object).
 - Dołącza device object do stosu urządzeń (device stack).

Przetwarzanie pakietów IRP

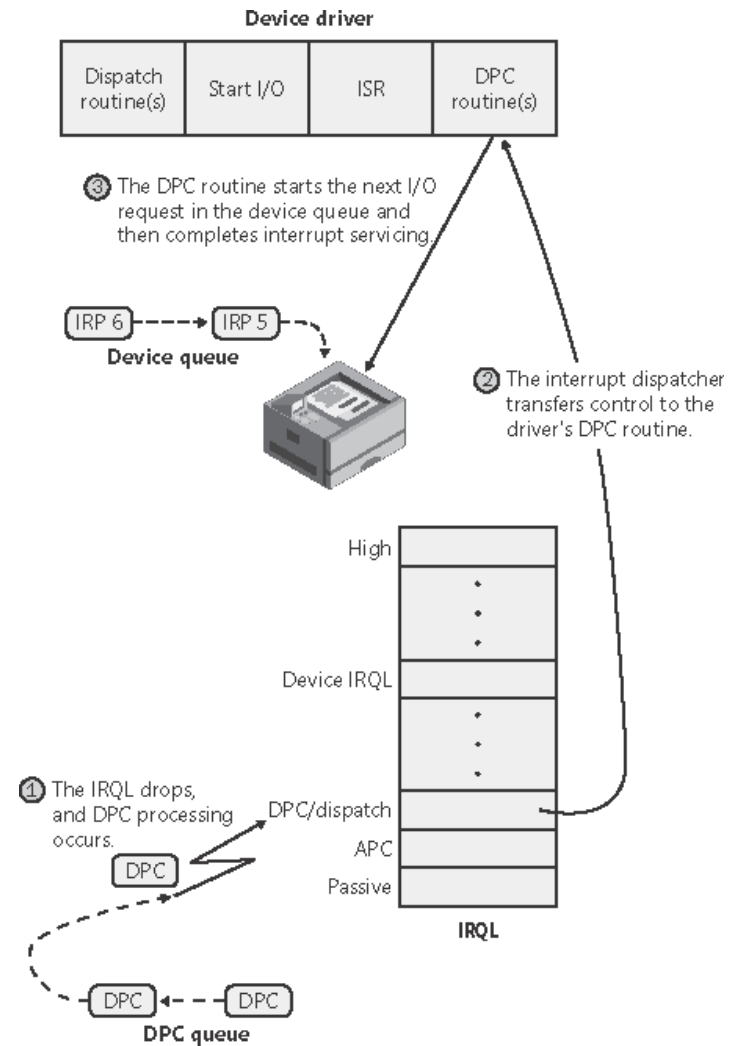
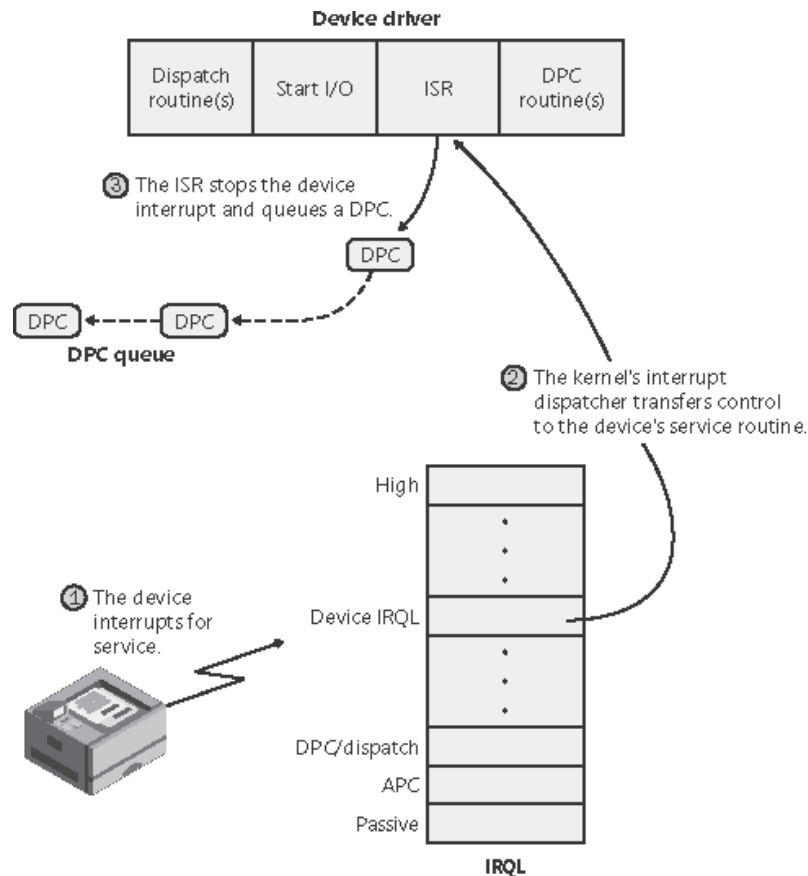
- Sterownik jednowarstwowy



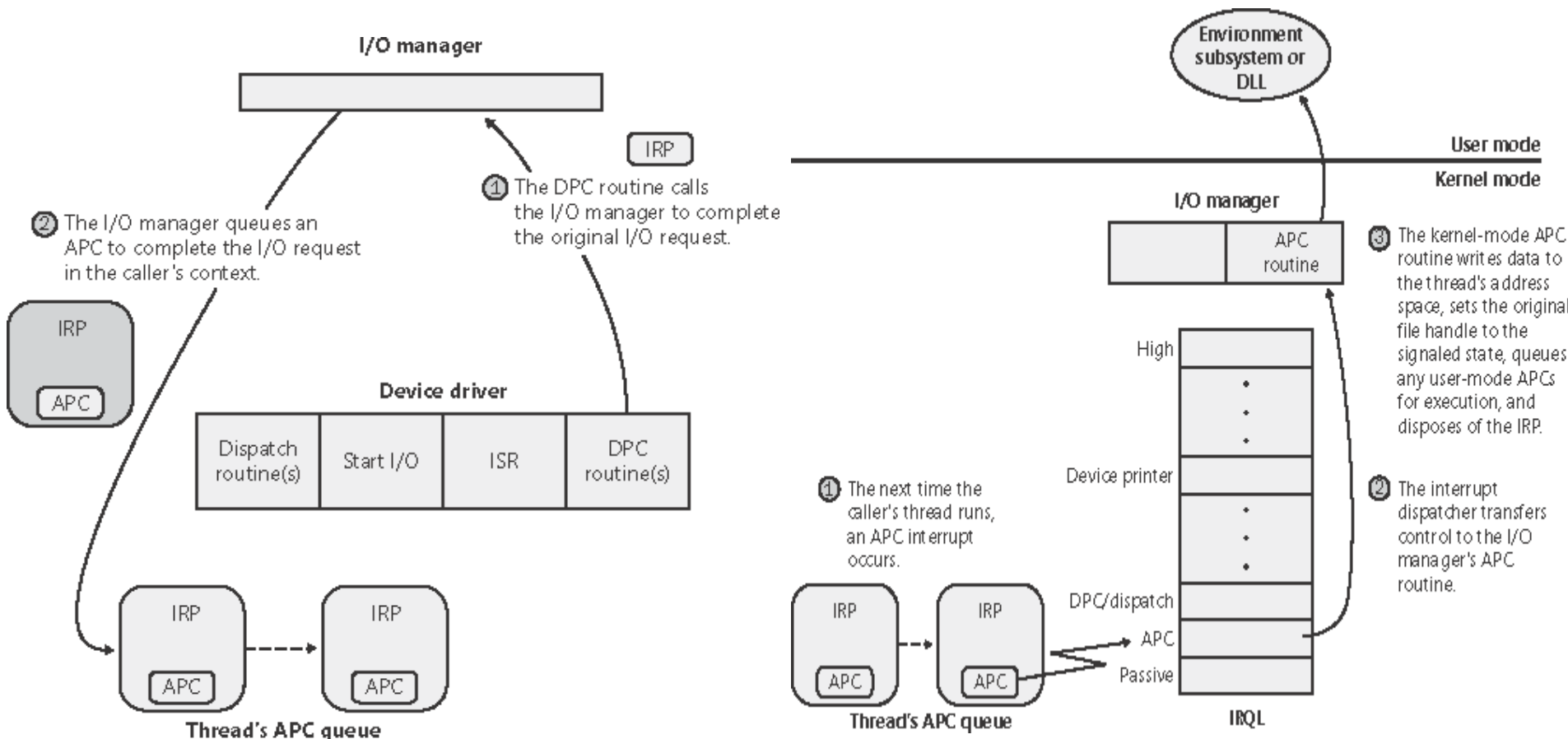
Przetwarzanie żądań synchronicznych



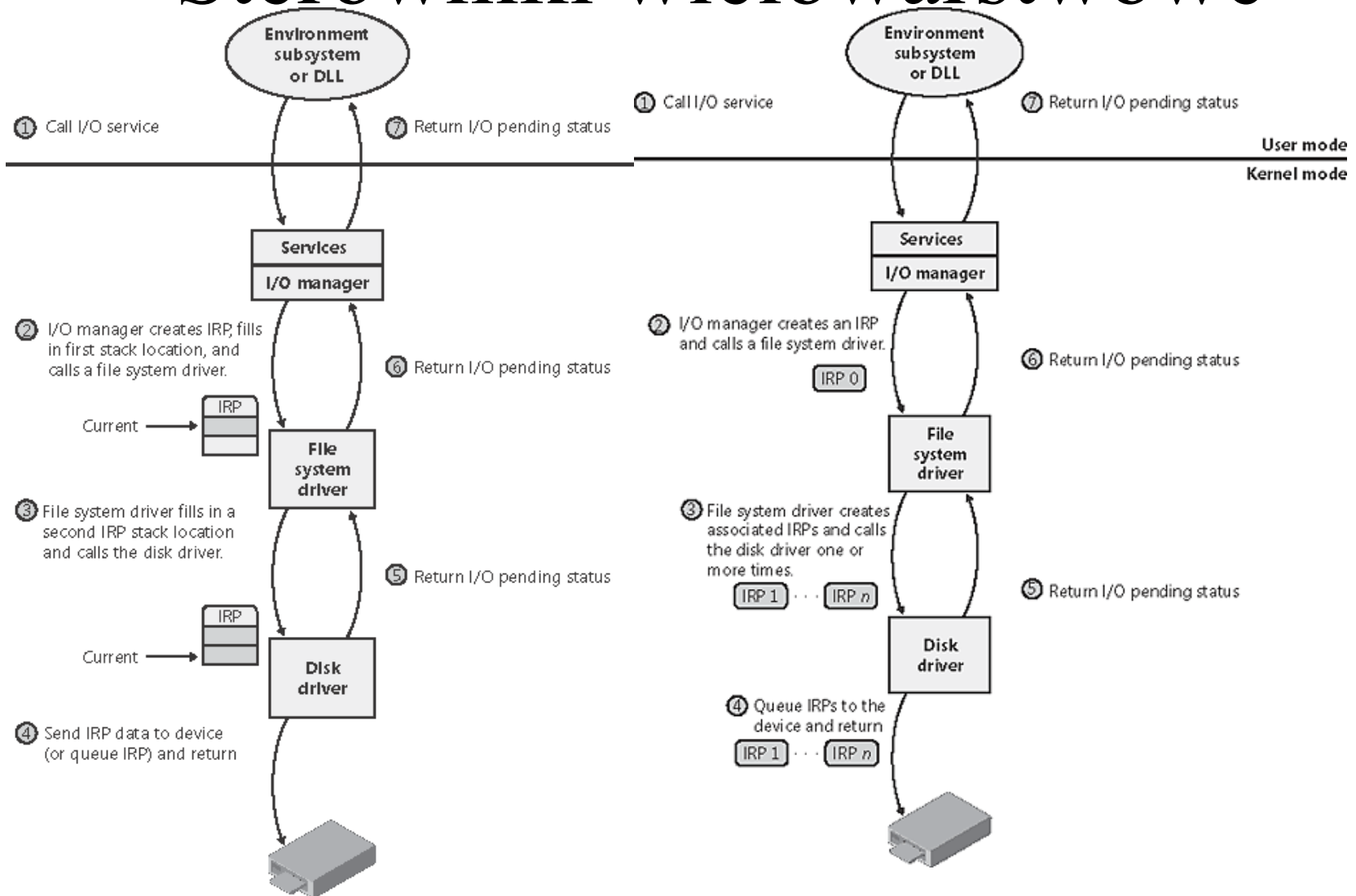
Obsługa przerwań



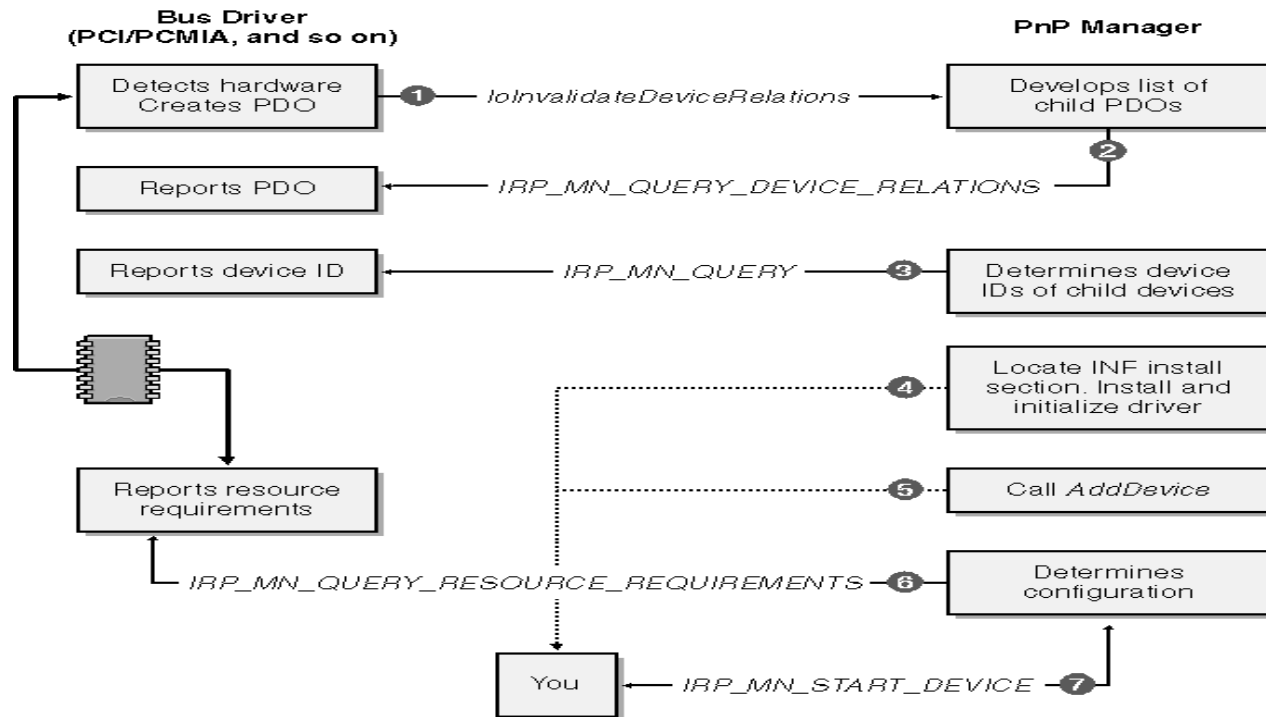
Kończenie obsługi IRP



Sterowniki wielowarstwowe



Instalacja i ładowanie sterownika PnP



Instalacja i ładowanie sterownika PnP

Sterownik magistrali (Bus driver) wykrywa hardware i pyta menadżera PnP o zaktualizowaną listę urządzeń.

Menadżer Pnp wysyła IRP (I/O request) do sterownika magistrali.

Sterownik magistrali zwraca listę znalezionych PDO (Device Object), które menadżer Pnp użył do określenia niezainicjalizowanych PDO.

Menadżer Pnp pyta sterownik magistrali o identyfikator urządzenia, który jest użyty przez menadżera PnP do lokalizacji klucza w Rejestrze systemowy

Jeśli sterownik jest już zainstalowany PnP Menadżer przechodzi do wywołania funkcji AddDevice (3 kroki dalej)

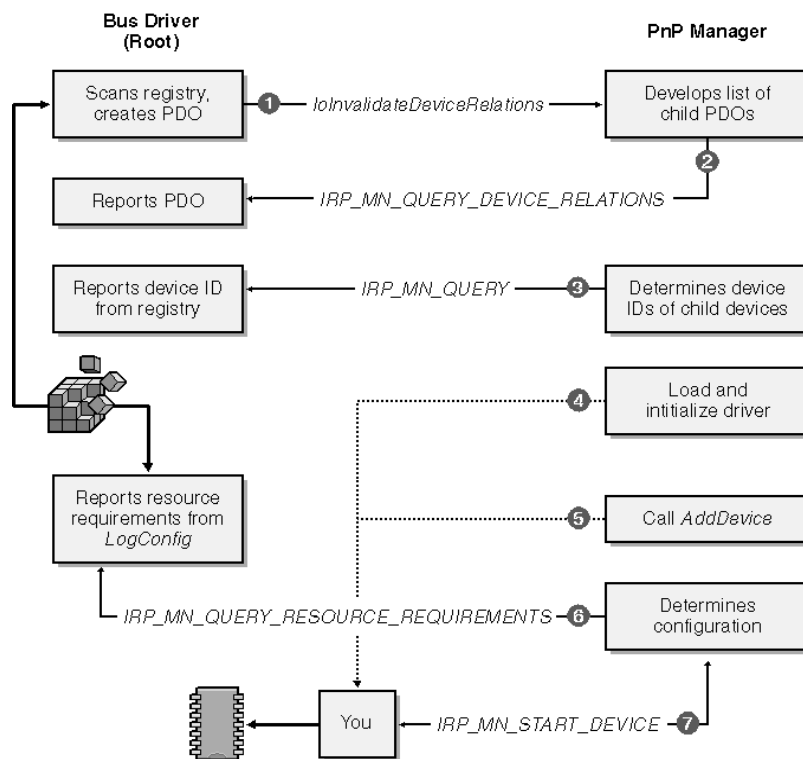
Menadżer Pnp instaluje sterownik używając właściwego pliku inf.

➔ Odtąd urządzenie jest znane przez system

Instalacja i ładowanie sterownika PnP

- Menadżer Pnp woła menadżera pamięci w celu mapowania pamięci zanim sterownik zostanie załadowany.
- Memory manager woła DriverEntry.
- Menadżer Pnp woła funkcję AddDevice sterownika, i w ten sposób załadowany sterownik jest powiadomiony o wykryciu nowego urządzenia.
- Menadżer Pnp wysyła IRP do sterownika magistrali aby pozyskać informacje o potrzebnych zasobach
- Menadżer Pnp konfiguruje sprzęt i wysyła IRP_MJ_PNP do sterownika. Urządzenie jest uruchomione.

Instalacja i ładowanie sterowników nie - PnP

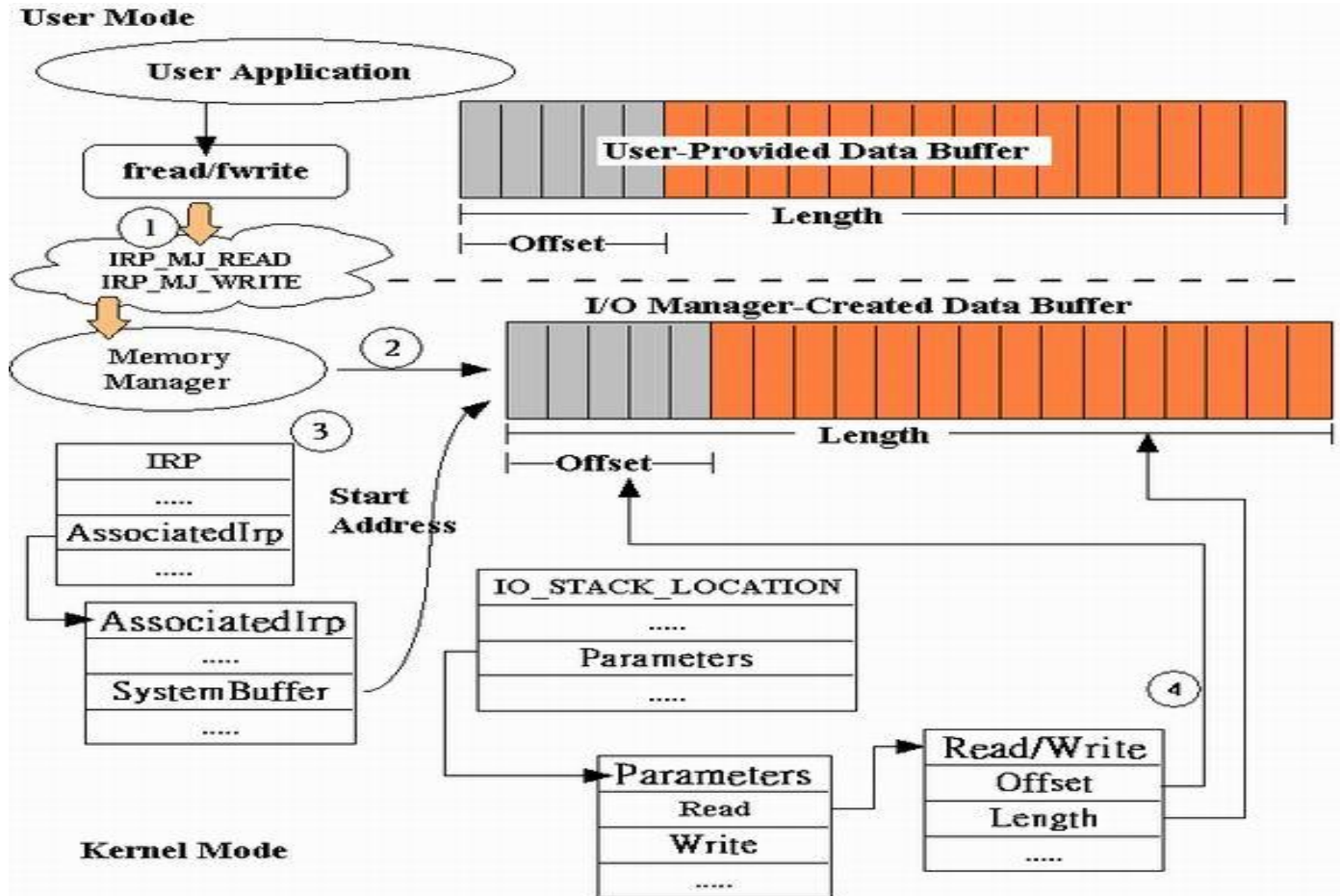


- Użytkownik instaluje sterownik (np.. Przez “Add new hardware”)
→ Zostaje utworzony właściwy wpis w rejestrze
- System jest restartowany.
- The root enumerator skanuje rejestr i znajduje nowo dodany sprzęt.
- Po tym, proces jest identyczny jak dla sterowników Pnp.

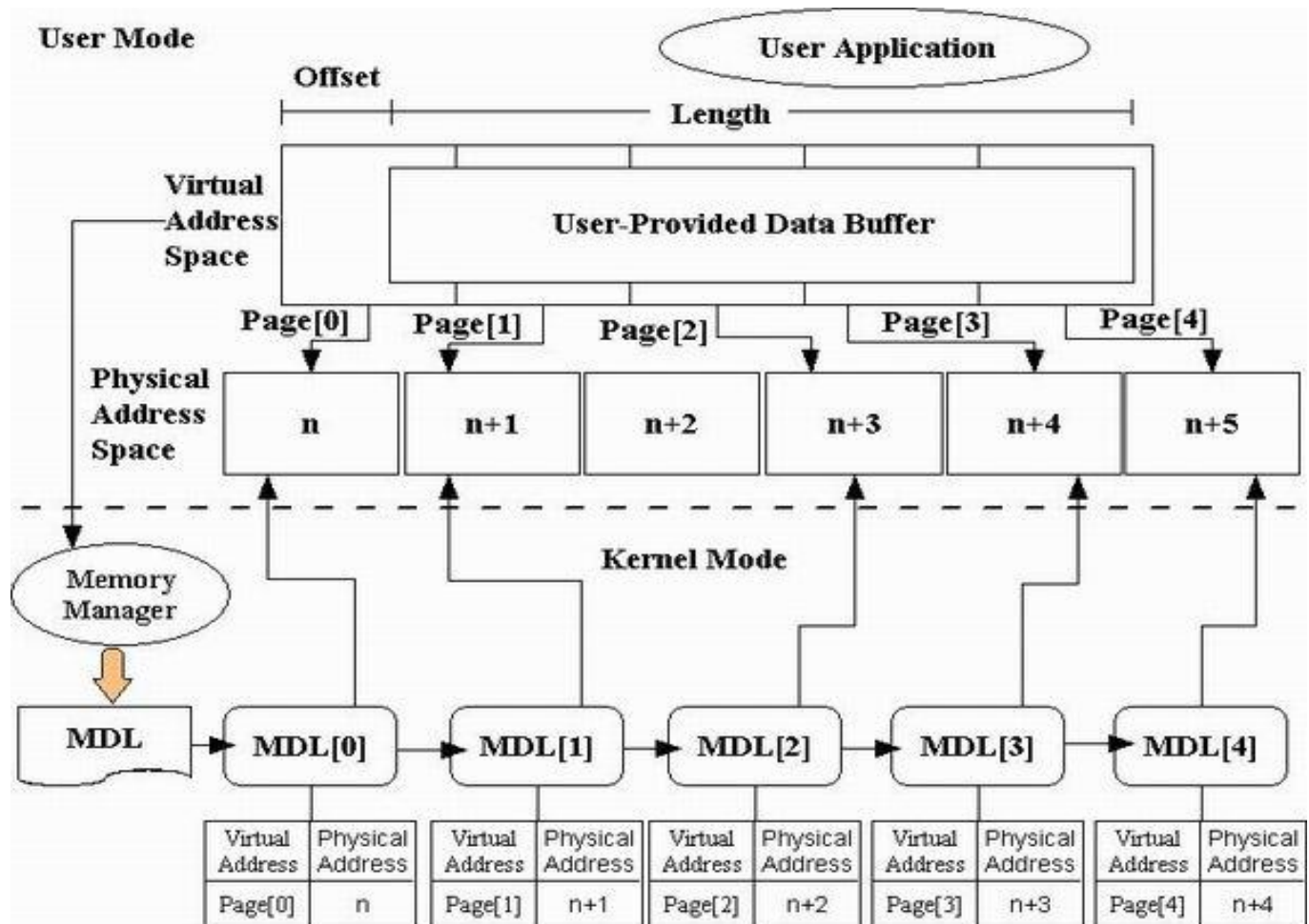
Funkcje odczytu/zapisu

- Istnieją trzy sposoby przekazywania danych pomiędzy user i kernel mode
 - Direct I/O (bezpośredni)
 - Buffered I/O (buforowany)
 - Neither (własny)

Buforowane operacje I/O



Direct I/O



Literatura

- W. Oney, *Programming the Windows Driver Model 2nd Edition*, Microsoft Press, 2003
- DDK documentation, <http://www.microsoft.com/whdc/devtools/ddk/default.msp>
- Introduction to Drivers: Part 1, <http://www.codeproject.com/system/driverdev.asp>
- Introduction to Drivers: Part 2: <http://www.codeproject.com/system/driverdev2.asp>
- Tips for Windows NT Driver Developers -- Things to Avoid;
<http://support.microsoft.com/default.aspx?scid=KB;EN-US;Q186775&>
- <http://www.codeproject.com/system/driverdev.asp>

Sterowniki WDF

Rola WDF

- Sterowniki modelu WDF (*Windows Driver Foundation*) spełniają tę samą rolę co sterowniki modelu WDM (*Windows Driver Model*): obsługują komunikację pomiędzy systemem operacyjnym Windows i urządzeniem.
- Reprezentują całkowicie nową budowę sterowników, powiązaną z WDM, umożliwiając łatwiejszą implementację.
- Istotną rolę odgrywa dobrze zdefiniowany model obiektowy, umożliwiający sterownikom WDF komunikację z obiektami poprzez spójny interfejs.

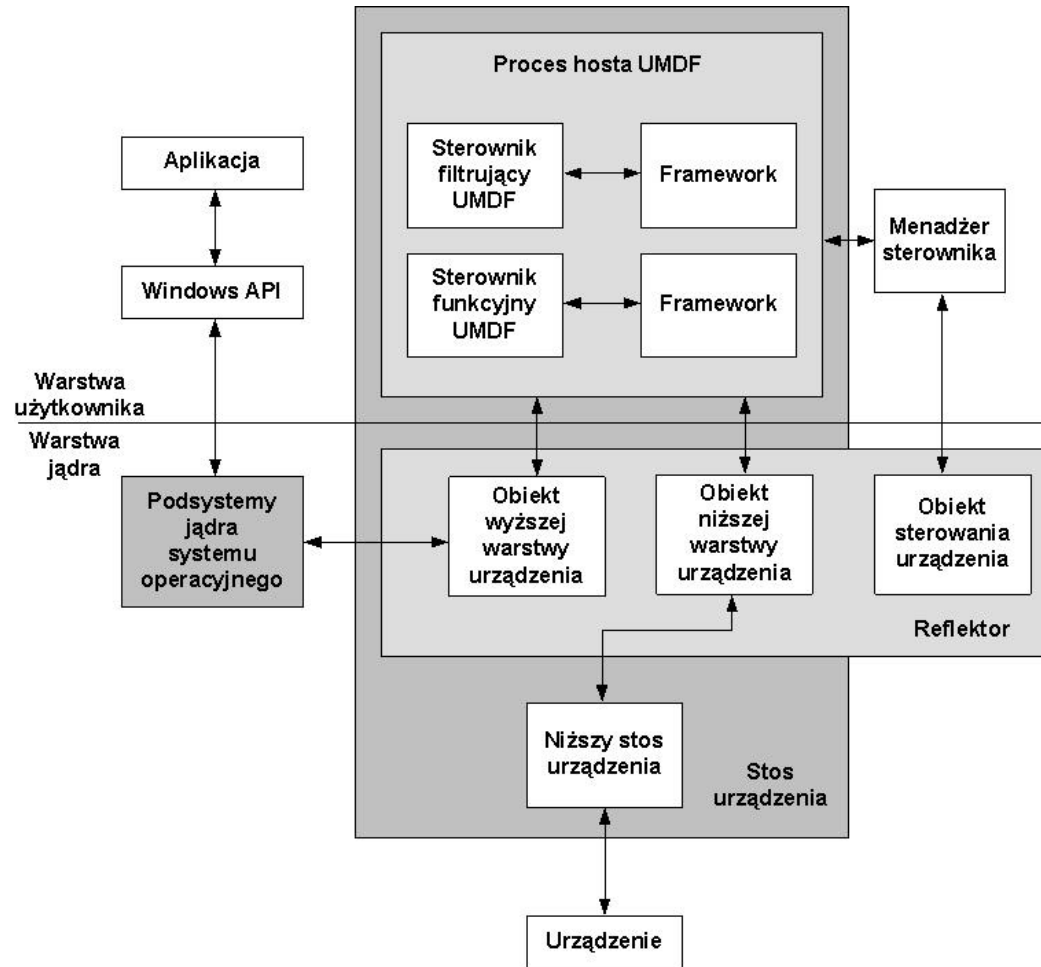
Rola WDF

- W skład modelu WDF wchodzi dwa frameworki: UMDF (User-Mode Driver Framework) i KMDF (Kernel-Mode Driver Framework), wspierające odpowiednio budowę sterowników warstwy użytkownika oraz jądra systemu operacyjnego
- Każdy z nich ma odmienną budowę.
- UMDF jest przeznaczony dla warstwy aplikacji (user mode) i ma postać biblioteki dll
- KMDF jest przeznaczony dla kernela SO i jego filozofia jest zgodna ze sterownikami WDM

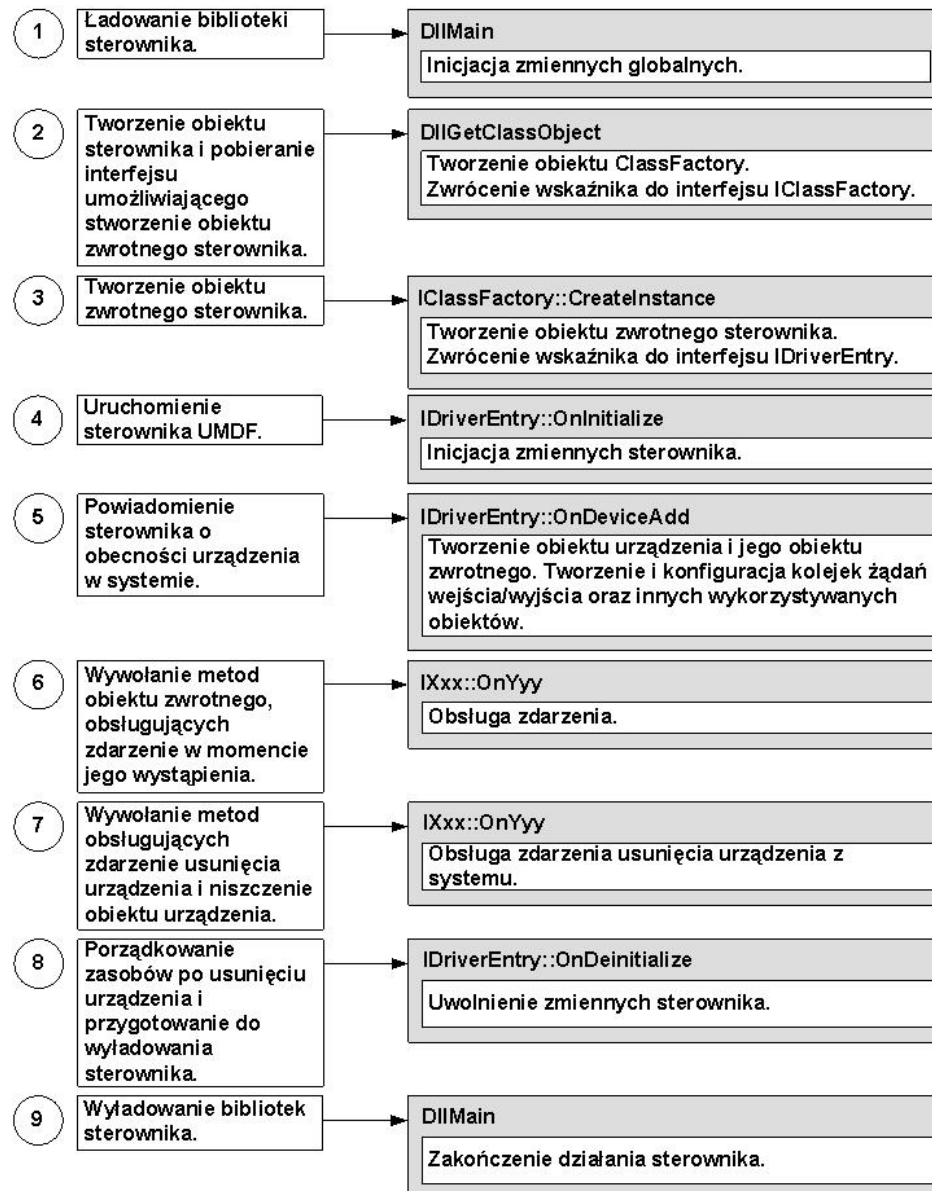
UMDF

- UMDF jest modelem programowania opartym na modelu COM.
- Umożliwiającym budowę sterowników funkcyjnych i filtrujących dla urządzeń bazujących na protokołach portów szeregowych i sieciowych, np. urządzeń USB.
- Dostarcza on wiele podstawowych obiektów, należących do dwóch głównych kategorii:
 - obiekty frameworka (ang. framework objects) - są to obiekty wbudowane w framework, reprezentujące: sterownik, obsługiwane przez niego urządzenie, kolejkę wejścia/wyjścia żądań, itp, które mogą być tworzone zarówno przez framework jak i sterownik. Każdy z obiektów posiada odpowiednie metody, pozwalające mu na wykonanie danej akcji, pobranie lub zapisanie wartości jego właściwości.
 - obiekty zwrotne (ang. callback objects) - są to obiekty implementowane przez sterownik i rejestrowane we frameworku. Umożliwiają one stworzenie własnej, specyficznej dla danego urządzenia obsługi zdarzenia, wywołanego przez obiekty frameworka. Udostępniają jeden lub więcej interfejsów, z których każdy posiada zbiór metod, obsługujących poszczególne zdarzenia. W momencie zaistnienia w systemie operacyjnym określonych zmian, np. zmiany stanu zasilania, framework wywołuje je w celu powiadomienia sterownika o ich wystąpieniu i przekazaniu do niego odpowiednich danych. Sterownik rejestruje obiekt zwrotny poprzez przekazanie do frameworka wskaźnika do niego.

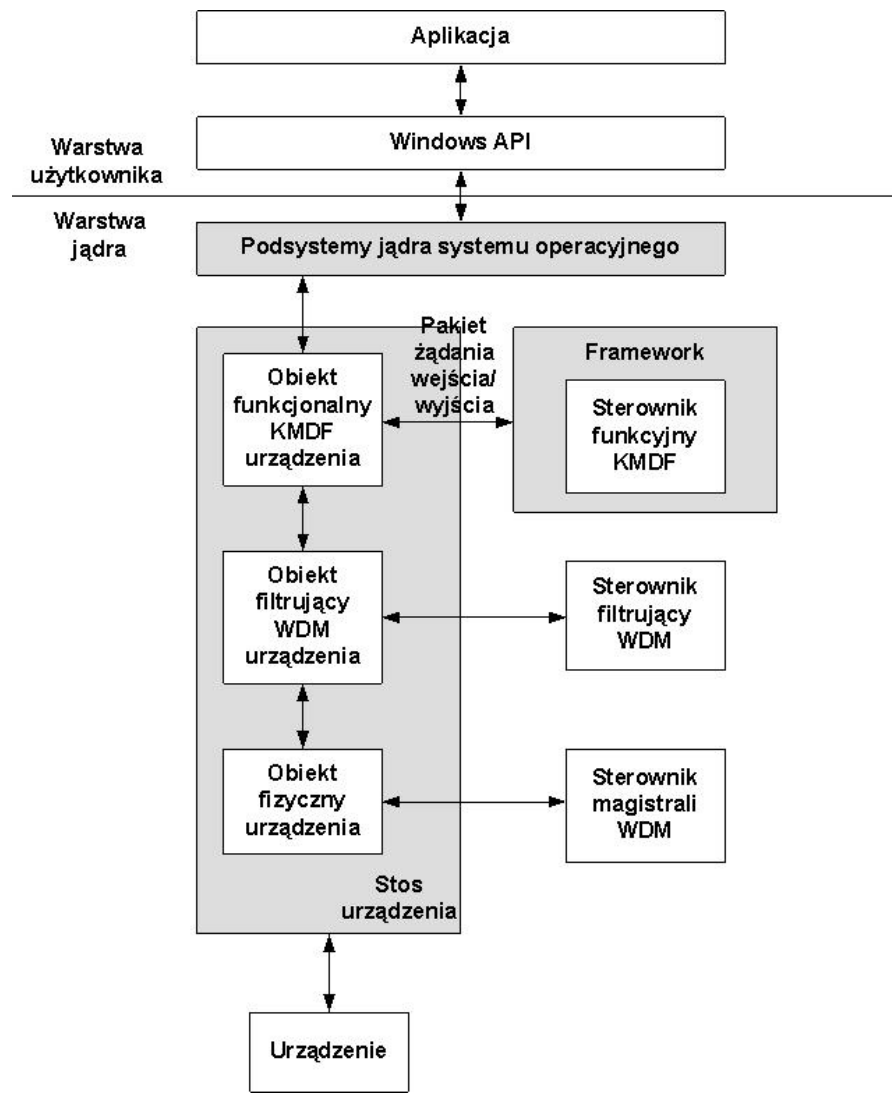
Architektura UMDF



Struktura sterownika UMDF



Architektura KMDF



Struktura sterownika KMDF

